TUM School of Computation, Information and Technology
Technical University of Munich



In collaboration with Agile Robots SE

Master's Thesis in Robotics, Cognition, Intelligence

Learning Long-Horizon Cyclic Nut Threading with a Dexterous Hand via Hybrid-Teacher Policy Distillation

Lernen langfristiger zyklischer Mutter-Gewindemontage mit einer geschickten Hand durch Policy-Destillation eines hybriden Lehrers

Supervisor	Prof. Dr.-Ing. habil. Alois C. Knoll
Advisor	Chi Zhang, M.Sc.
Industry supervisor	Antonio Gonzales Marin, M.Sc. Agile Robots SE
Author	Carsten Oertel
Date	November 15, 2026 in Munich

Disclaimer

I confirm that this Master's Thesis is my own work and I have documented all sources and material used.

Munich, November 15, 2026

(Carsten Oertel)

Acknowledgments

This thesis was conducted in collaboration with Agile Robots SE, Munich. The author was employed at Agile Robots SE during the course of this work, and the research was carried out using computational and hardware resources provided by Agile Robots SE, including the Franka Research 3 robot arm, the Inspire RH56DFX dexterous hand, and GPU infrastructure for reinforcement learning training. The author gratefully acknowledges Agile Robots SE for their support, in particular Antonio Gonzales Marin, M.Sc., for his supervision on the industry side.

The author further thanks Prof. Dr.-Ing. habil. Alois C. Knoll for supervising this thesis at the Chair of Robotics, Artificial Intelligence and Real-time Systems of the Technical University of Munich, and Chi Zhang, M.Sc., for his advising and for his work on the hybrid teacher, the distillation stack, and the hardware experiments this thesis builds on.

Abstract

Threaded joints are central to assembly because they can be released again and because their threads are standardized. This thesis asks how a robot with a dexterous hand can learn to thread a nut. Threading poses three difficulties at once. The skill has to be learned in simulation and transferred to hardware, although thread contact is hard to simulate. The hand has a high-dimensional action space and many ways to solve the task. Finally, threading is a long-horizon cyclic task: the range of the wrist and the fingers limits each turn, so the hand has to grasp, turn, release, and re-grasp the nut cycle after cycle while errors compound. The thesis decomposes the task during data generation and unifies it at deployment. A privileged reinforcement-learning teacher learns the pinch-grasp turn in a simplified thread simulation, and scripted release and return motions chain its turns into multi-cycle demonstrations. A single visuomotor student is distilled from these demonstrations by conditional flow matching and acts on a metric point cloud from a fixed RGB-D camera and on proprioception. It is trained on simulated data only. A calibrated camera model, visual randomization, and a Cartesian action path shared by teacher, student, and robot are designed to carry it zero-shot to a physical dexterous hand-arm system. In simulation, the student completes all ten threading cycles in each of ten trials, with a mean cyclic tracking error of $-1.77°$, a lag of less than two degrees per cycle. Each ablation, namely a single-cycle student, an image-space encoder, and two deterministic regression heads, fails before the tenth cycle in every trial. On hardware, the zero-shot student also completes all ten cycles in every trial on the training nut, with a mean cyclic tracking error of $-4.21°$. It completes all ten cycles on the M30 and M36 hexagonal nuts and on the unseen octagonal and round profiles of all three sizes, and it fails on the rectangular profile, whose face topology conflicts with the learned re-grasp.

Kurzfassung

Schraubverbindungen sind zentral für die Montage, weil sie wieder lösbar und ihre Gewinde genormt sind. Diese Arbeit untersucht, wie ein Roboter mit einer geschickten Hand das Aufschrauben einer Mutter lernen kann. Das Aufschrauben stellt drei Schwierigkeiten zugleich. Die Fähigkeit muss in der Simulation gelernt und auf die Hardware übertragen werden, obwohl sich der Gewindekontakt nur schwer simulieren lässt. Die Hand hat einen hochdimensionalen Aktionsraum und viele Möglichkeiten, die Aufgabe zu lösen. Schließlich ist das Aufschrauben eine zyklische Aufgabe mit langem Horizont: Der Bewegungsbereich von Handgelenk und Fingern begrenzt jede Drehung, sodass die Hand die Mutter Zyklus für Zyklus greifen, drehen, loslassen und erneut greifen muss, während sich Fehler aufsummieren. Die Arbeit zerlegt die Aufgabe bei der Datenerzeugung und vereint sie beim Einsatz. Ein privilegierter Lehrer lernt mit Reinforcement Learning die Drehung im Spitzgriff in einer vereinfachten Gewindesimulation, und skriptgesteuerte Loslass- und Rückkehrbewegungen

verketten seine Drehungen zu Demonstrationen über mehrere Zyklen. Aus diesen Demonstrationen wird mittels bedingtem Flow Matching eine einzige visuomotorische Schülerpolicy destilliert, die auf einer metrischen Punktwolke einer festen RGB-D-Kamera und auf Propriozeption arbeitet. Sie wird ausschließlich mit simulierten Daten trainiert. Ein kalibriertes Kameramodell, visuelle Randomisierung und ein von Lehrer, Schüler und Roboter geteilter kartesischer Aktionspfad sollen sie ohne Nachtraining (zero-shot) auf ein reales geschicktes Hand-Arm-System übertragen. In der Simulation vollendet die Schülerpolicy in jedem von zehn Versuchen alle zehn Schraubzyklen mit einem mittleren zyklischen Nachführfehler von $-1{,}77°$, also einem Rückstand von weniger als zwei Grad pro Zyklus. Jede Ablation, nämlich eine Einzelzyklus-Schülerpolicy, ein bildbasierter Encoder und zwei deterministische Regressionsköpfe, scheitert in jedem Versuch vor dem zehnten Zyklus. Auf der Hardware vollendet die Schülerpolicy ohne Nachtraining ebenfalls alle zehn Zyklen in jedem Versuch auf der Trainingsmutter, mit einem mittleren zyklischen Nachführfehler von $-4{,}21°$. Sie vollendet alle zehn Zyklen auch auf den Sechskantmuttern M30 und M36 sowie auf den ungesehenen achtkantigen und runden Profilen aller drei Größen. Am rechteckigen Profil, dessen Flächentopologie dem gelernten Umgreifen widerspricht, scheitert sie.

Contents

Acknowledgments **v**

List of Abbreviations **xix**

List of Symbols **xxi**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem: How Can a Robot Learn Threading? . . . 2
1.2.1 Simulation . . . 2
1.2.2 Threading with a Dexterous Hand . . . 3
1.2.3 Threading as a Long-Horizon Cyclic Task . . . 3
1.3 Approach . . . 4
1.4 Research Questions . . . 5
1.5 Contributions . . . 5
1.6 Outline . . . 6

2 State of the Art **7**
2.1 Theoretical Background . . . 7
2.1.1 Threaded Joints and Thread Geometry . . . 7
2.1.2 Reinforcement Learning . . . 7
2.1.3 Imitation Learning . . . 9
2.1.4 Action Chunking . . . 10
2.1.5 Generative Action Heads: Diffusion and Flow Matching . . . 11
2.1.6 Operational-Space Control . . . 12
2.2 Related Work . . . 15
2.2.1 Robotic Assembly and Threading . . . 15
2.2.2 Dexterous Manipulation with Learned Policies . . . 16
2.2.3 Visual Representations for Manipulation . . . 16
2.2.4 Long-Horizon Tasks and Policy Chaining . . . 17
2.2.5 Teacher–Student and Visuomotor Distillation . . . 18
2.2.6 Summary and Research Gap . . . 18

3 Methodology **21**
3.1 Problem Formulation . . . 22
3.2 Simulated and Physical System . . . 22
3.2.1 Platform . . . 23
3.2.2 Simplified Thread Simulation . . . 23
3.2.3 Action Contract and Controller . . . 25
3.3 Privileged Teacher Training . . . 26
3.4 Hybrid Teacher and Dataset Collection . . . 29

3.4.1 Cyclic Teacher Rollout in Simulation . . . 29
3.4.2 Disturbance Augmentation . . . 30
3.4.3 Vision Data Collection . . . 31
3.4.4 Dataset Format . . . 32
3.5 Student Policy Distillation by Conditional Flow Matching . . . 33
3.5.1 Conditional Flow Matching . . . 33
3.5.2 Student Architecture . . . 33
3.5.3 Offline Training . . . 35

4 Experiments **37**
4.1 Evaluation Metrics . . . 37
4.1.1 Evaluation Ladder . . . 37
4.1.2 Metrics . . . 38
4.2 Experimental Setup . . . 38
4.2.1 Zero-Shot Transfer Protocol . . . 39
4.3 Teacher and Demonstrations . . . 41
4.4 Ablations . . . 42
4.5 Results . . . 43
4.6 Generalization across Nut Sizes and Geometries . . . 44
4.7 Long-Horizon Stress . . . 45
4.8 Summary . . . 45

5 Discussion **47**
5.1 Interpretation per Research Question . . . 47
5.2 Action and Observation Contracts . . . 49
5.3 What This Means for Robotic Assembly . . . 49
5.4 Limitations . . . 50

6 Conclusion and Future Work **51**
6.1 Conclusion . . . 51
6.2 Future Work . . . 52

A Implementation Details **55**
A.1 Simulation and Controller . . . 55
A.2 Robot Assets . . . 55
A.3 Finger Kinematics of the Simulated Hand . . . 56
A.3.1 Method . . . 57
A.3.2 Results . . . 58
A.3.3 Fitted Model in Simulation . . . 61
A.4 Teacher Reward . . . 62
A.5 Hybrid Teacher . . . 63
A.6 Teacher Hyperparameters . . . 63
A.7 Student Hyperparameters . . . 63
A.8 Dataset Summary . . . 63

B Sim-to-Real Measures in the Arm Controller **65**
B.1 Task-Wrench Dead Zone in Training . . . 65
B.2 Controller Additions for Deployment . . . 66
B.3 Rollouts in ros-sim with and without the Dead Zone . . . 67
B.4 Step-Aligned Comparison of the Return-Dead-Zone Student . . . 70
B.5 Static Dead-Zone and Friction Probe . . . 71

B.6 Lost Wrench against Speed 72
B.7 Why the Return Stalls 74
B.8 Transition Clip 75
B.9 Target Accuracy and Repeatability 76
B.10 Joint Friction Fitted from Rollouts 79
B.11 Summary 80

Bibliography **81**

List of Figures

1.1 Overview of the approach: decompose during data generation, unify at deployment. Top: a privileged RL teacher is trained in parallel simulated workcells. During data collection its rollouts are chained with scripted release-and-reset controllers into long-horizon demonstrations $\mathcal{D}_{\text{sim}}$. A conditional flow-matching student is distilled from them and acts from RGB and depth. Bottom: the phases of one cycle on the physical robot, approach, grasp, rotation, goal reaching, and release-and-reset, after which the hand returns for the next cycle. 4

2.1 Diffusion and conditional flow matching for a bimodal action distribution in a two-dimensional toy action space (u_1, u_2). Generation time runs from left to right, from Gaussian noise p_0 to the demonstrated actions p_1, and each slice shows the action space at one point in time. (a) A diffusion model adds noise to demonstrated actions (gray path) and learns to reverse this process over many denoising steps along curved trajectories. The point clouds on the slices are intermediate denoising samples, colored by the mode they reach. (b) Conditional flow matching regresses a velocity field onto the straight path between a Gaussian sample and a data sample (dashed). Integrating the learned field (gray arrows on the slices at $\lambda = 0.25$, 0.5, and 0.75) from noise reaches either mode in a few ODE steps. 13

2.2 Impedance control, illustrated on a planar arm with three joints. (a) A virtual spring and damper with stiffness k_1, k_2, k_3, the diagonal entries of K_q, act directly on each joint, pulling it toward its reference q_{ref}. (b) A virtual spring and damper with stiffness k_x, k_y, k_φ and damping d_x, d_y, d_φ, the diagonal entries of K_p and K_d, act on the controlled end-effector frame along each task direction instead. The resulting wrench (F_x, F_y, M_z) is mapped to the joint torques τ_1, τ_2, τ_3 through J^{T}. 15

3.1 The pipeline in detail. Left: the privileged RL policy executes grasp, rotation, and goal reaching and a scripted controller executes release and reset. Chained in a loop, they form the hybrid teacher that generates the simulated demonstrations $\mathcal{D}_{\text{sim}}$. Right: the student encodes the aligned RGB-D observation into a 64-D point-cloud feature, fuses it with proprioception, the normalized global progress, and the process-phase encoding into a 256-D condition, and a conditional flow-matching Transformer predicts an eight-step action chunk. 21

3.2 The robot platform, rendered from the simulation models used in this work. In the assembled system the hand is mounted on the arm flange. 23

3.3 How a physics engine resolves contact, shown for PhysX. (a) Body A moves toward body B with velocity $\boldsymbol{v}$. Once their gap falls below the contact offset, the engine creates a contact. (b) Within one time step Δt, A sinks into B by the penetration depth d along the contact normal $\boldsymbol{n}$. (c) The solver pushes both bodies apart along $\boldsymbol{n}$ with a separating velocity $\boldsymbol{v}_{\text{sep}}$ until their separation reaches the rest offset. (d) The separating velocity grows in proportion to $d/\Delta t$ and is capped at the maximum depenetration velocity $v_{\max}$. 24
3.4 How interpenetration can be exploited in thread simulation, shown in an axial section of bolt and nut. (a) The nut thread sits in the bolt groove and advances by one pitch p per turn. (b) A push along the axis with velocity $\boldsymbol{v}$ sinks the nut flanks into the bolt flanks. The contact normal $\boldsymbol{n}$ of the loaded flank points back out of the groove. (c) Once the nut thread has been pushed past the bolt crest, the nearest separating direction is the opposite flank. The solver resolves the overlap with $\boldsymbol{v}_{\text{sep}}$ along the axis and pushes the nut into the next groove. (d) Axial progress z over nut rotation θ. On the helix the nut gains one pitch per full turn. The shortcut gains the same progress, and the same reward for axial progress, almost without turning. 24
3.5 Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t through $d_t = k\theta_t$ with $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. The finger–nut contacts remain fully simulated. 25
3.6 Data-collection loop of the hybrid teacher over K threading cycles. The learned policy (blue) owns approach, grasp, turn, and hold. A physical release gate on the nut rotation hands over to the scripted transitions (orange): four waypoints, which open the pinch and retreat, and the gated return. A completed return rebases the turning target and starts the next cycle, and a return that times out rejects the episode. Table A.6 lists the gate thresholds. Every policy, waypoint, and return segment receives one random arm impulse, and every executed command is recorded in the common 9-D label space. 30
3.7 Three draws of the texture randomization in the simulated workcell. The dome light and the textures of the floor and the tabletop change between draws, while the robot keeps its nominal appearance. 32
3.8 Simulated depth with and without augmentation, next to real depth. Left: the noise-free depth of one frame of a simulated demonstration, rendered by the 320×180 head camera. Middle: the same depth image after the full augmentation used in student training, with depth noise, quantization, and dropout of edge pixels, correlated holes, and isolated pixels. The augmentation raises the fraction of invalid pixels in this frame from 21.4 % to 47.3 %. Right: a depth frame streamed from the head camera of the physical robot, shown for comparison. All three images use the same color map, from about 0.8 m (dark blue) to 2 m (dark red), and black marks invalid pixels. 32
3.9 Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature. The 35-D proprioception is encoded by an MLP. Late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time. 34

4.1 Evaluation ladder. Each mode closes one more part of the control loop. A failure in a mode points to the causes listed below it. 37

4.2 Hardware setup. Left: the FR3 arm with the Inspire RH56DFX hand, the bolts on their holders in front of the robot, and the fixed RealSense D415 on its stand. Right: the RGB image and the metric depth image of the D415 as the student receives them, after center-cropping and downscaling to 320×180. . . 39
4.3 The test-object family. The hexagonal nuts of the three sizes M24, M30, and M36 sit on their bolts and holders. In front of each bolt lie the round, octagonal, and rectangular nuts of the same size. The angular scale of Figure 4.4 is printed on the sheet under the holders. 40
4.4 One threading cycle on the physical robot. Top, from left to right: approach, grasp, rotation, goal reaching, and release with return to the reset pose. Bottom: top-down views of the angular scale under the bolt during grasp, rotation, and goal reaching. The scale is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors. 41
4.5 Point cloud of the physical D415 as the encoder receives it. Upper rows: the 4096 sampled points (blue) over the RGB image, with crosses for the nut centre, the nut top, and the bolt base of the training scene and for the grasp point recorded during the trial. Lower rows: the same points in the x–z side view of the robot base frame, with the training nut as a red box and the bolt as a yellow bar. 42

A.1 Thumb abduction sweep on the physical hand, six stops from open (top left, register 979) to fully abducted (bottom right, register 242). Magenta outlines show the vendor model and cyan outlines the fitted model at the register of each stop, both projected into the camera image. Each label gives the fingertip distance of the vendor and the fitted model from the camera fit. Mean over the stops: 18.3 mm for the vendor model, 3.7 mm for the fitted model. 59
A.2 Thumb flexion sweep at abduction register 250, six stops from open (register 994) through closed and back to open (register 991). Magenta outlines show the vendor model and cyan outlines the fitted model. Mean fingertip distance from the camera fit over the stops: 8.9 mm for the vendor model, 2.1 mm for the fitted model. 60
A.3 Index flexion sweep, six stops from register 800 through register 401 and back to open (register 999). Magenta outlines show the vendor model and cyan outlines the fitted model. Mean fingertip distance from the camera fit over the stops: 6.9 mm for the vendor model, 1.4 mm for the fitted model. 62

B.1 Observations per cycle on hardware and in ros-sim 69
B.2 Grasp position of the return-dead-zone student 70
B.3 Lost wrench against speed 74
B.4 Transition clip on hardware 77

List of Tables

3.1 Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. Joint torques are divided by the effort limits of 87 N m (joints 1–4) and 12 N m (joints 5–7). 27

3.2 Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling. Multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability. Joint torques are in units of the joint effort limits. Quantities marked † start at their nominal value and widen to the listed range with the initial-yaw curriculum. 28

4.1 Training statistics of the M24 teacher over its last ten epochs, with the full randomization of Table 3.2 active. 41

4.2 Simulation and real-world zero-shot ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target. It is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot. Ablations (c) and (d) are evaluated in simulation only. 44

4.3 Real-world zero-shot transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation. The octagonal, rectangular, and round profiles and the M30 and M36 sizes are absent from the demonstrations. 45

A.1 Simulation settings, thread model, and simulated hand of Section 3.2. 55

A.2 Action decoding and controller of the shared Cartesian contract. The values are nominal. The per-episode randomization of the simulated controller is listed in Table 3.2. In the physical column, "same" marks values shared with simulation. 56

A.3 Finger kinematics of the vendor model and of the model fitted to the physical hand. The register maps give the joint angle as a function of $\tilde{r} = (1 - r/1000)\, q_{\max}$, with $q_{\max}$ the URDF upper limit of the joint. The vendor maps are those of the hand driver before the fit. The residual is the standard deviation of the coupling fit. Angles in rad. 61

A.4 Fingertip distance from the camera fit in mm, mean / maximum over n stops. Upper part: full models, each driven by the commanded registers, at the six stops of Figures A.1 to A.3. Lower part: couplings only, applied to the measured leading joints, on the thumb flexion stops at abduction register 250, the index stops with $q_{i0} \leq 0.85$ rad, and the pinch stops of the fitting sweep and of the held-out sweep. 61

A.5 Reward terms of the threading teacher (3.8). Progress terms pay only for newly achieved progress. The weight of P^{wrench} rises from zero at the first initial-yaw curriculum level to one at its full range. 63

A.6 Thresholds and timings of the scripted transitions of the hybrid teacher (Section 3.4.1). 63
A.7 PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic). 64
A.8 Hyperparameters of the conditional flow-matching student. 64
A.9 Demonstration data of the M24 ten-cycle collection used for the reported students. 64

B.1 Settings of the deployment Cartesian impedance controller 65
B.2 Per-cycle metrics with and without the dead zone in ros-sim 68
B.3 Observation distances to the hardware mean 68
B.4 Grasp height and vertical error of the return-dead-zone student 70
B.5 Rest band of the static probe 72
B.6 Fitted loss curve per axis 73
B.7 Joint torques at a return stall 75
B.8 Transition phases with the transition clip 76
B.9 Accuracy and repeatability in ros-sim 77
B.10 Accuracy and repeatability of the physical arm 78
B.11 Joint Coulomb friction fitted from rollouts 79

List of Abbreviations

ACT	Action Chunking with Transformers
ADR	Automatic Domain Randomization
BC	Behavior Cloning
CFM	Conditional Flow Matching
DAgger	Dataset Aggregation
DoF	Degree of Freedom
DP3	3D Diffusion Policy
DR	Domain Randomization
EMA	Exponential Moving Average
FR3	Franka Research 3
GAE	Generalized Advantage Estimation
IL	Imitation Learning
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multilayer Perceptron
ODE	Ordinary Differential Equation
OSC	Operational-Space Control
PD	Proportional-Derivative
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Color and Depth Image
RL	Reinforcement Learning
RMSE	Root Mean Square Error
SDF	Signed Distance Field
TCP	Tool Center Point

List of Symbols

Unless a section states otherwise, the symbols below keep the following meanings in all chapters.

Symbol	Meaning
Task	
$\mathcal{S}$	ordered phase sequence of one threading cycle
$m_t \in \mathcal{M} = \mathcal{M}_{\mathrm{RL}} \cup \mathcal{M}_{\mathrm{script}}$	active phase at step t; learned phases $\mathcal{M}_{\mathrm{RL}}$, scripted phases $\mathcal{M}_{\mathrm{script}}$
K	number of cycles per trial ($K = 10$)
$\Delta\varphi^*$	desired clockwise nut rotation per cycle (60°)
φ_k	unwrapped nut angle after the goal-reaching phase of cycle k
e_k, e_{ep}	cyclic and episodic (accumulated) tracking error
$C(\tau)$, $E(\tau)$	completed cycles and tracking cost of a trajectory τ
ρ	thread pitch
θ_t, d_t	revolute and prismatic coordinate of the simplified thread joint
Teacher	
$\boldsymbol{x}_t$	full simulator state
$\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$	actor observation with observation noise $\boldsymbol{\xi}_t$
$\boldsymbol{h}_t$	recurrent hidden state of the actor
$\pi_{\mathrm{T}}(\boldsymbol{a} \mid \boldsymbol{z}^{\pi}, \boldsymbol{h})$, $\bar{\pi}_{\mathrm{T}}$	privileged RL teacher and its deterministic mean
κ_m	scripted controller of transition phase $m \in \mathcal{M}_{\mathrm{script}}$
π_{HT}	hybrid teacher
$\boldsymbol{a}_t \in [-1, 1]^9$	normalized Cartesian-controller action (teacher); rescaled filtered target (student label)
r_t, $R_{i,t}$, $P_{j,t}$	shaped reward, reward terms, penalty terms
Student	
$\boldsymbol{y}_t = (\boldsymbol{O}_t^{\mathrm{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$	student observation: RGB-D image, proprioception, progress, phase encoding
$\mathcal{P}_t$, $\boldsymbol{v}_t \in \mathbb{R}^{64}$	point cloud and visual feature
$\boldsymbol{c}_t \in \mathbb{R}^{256}$	fused condition vector
H	action horizon (chunk length, $H = 8$)
H_{e}	execution horizon (actions executed per policy query)
$\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}]$	demonstrated action chunk
$\boldsymbol{U}_t^0 \sim \mathcal{N}(\boldsymbol{0}, I)$	Gaussian source sample
$\lambda \in [0, 1]$	flow time
$\boldsymbol{U}_t^{\lambda}$	interpolant between source and target chunk
$\boldsymbol{v}_{\theta}(\boldsymbol{U}^{\lambda}, \lambda \mid \boldsymbol{c})$	learned conditional velocity field
N	number of ODE integration steps
β	probability of teacher control during online DAgger
$\mathcal{D}_{\mathrm{sim}}$, $\mathcal{D}$	simulated demonstration set and the training set formed from it
Control	
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	joint positions and velocities
$\boldsymbol{\tau}$	joint torques
J, M, Λ	geometric Jacobian, joint-space inertia, task-space inertia
K_p, K_d	proportional and derivative gains

Chapter 1

Introduction

1.1 Motivation

Threaded fasteners such as a nut on a bolt join parts so that the joint can be released again, which makes them the standard choice wherever products are assembled in the field, maintained, adjusted, or taken apart [1], [2]. They appear in almost every industry. The halves of a cast engine housing are bolted together, airplane seats are mounted with threaded bolts, and the injector of a rocket engine is bolted onto the combustion chamber. Many valve and pipe connections consist of two threaded halves that are screwed together with a sealing material, so that the liquid stays inside, from the plumbing of a house to the water cooling loop of a gaming PC. These applications have the same needs. The product has to be manufacturable in modules, assembled in a defined process, and opened again without damage for inspection, maintenance, repair, and development.

The vast majority of people have also met threaded fasteners in everyday life, for example when assembling IKEA furniture or fixing household objects and appliances. For a human, the process is intuitive. You pick up a screw, find the hole, set the tip of a Philips screwdriver into the screw head, and turn. If the screw does not bite, you back off, straighten it in the hole, and turn again. Industry solves the same task differently. In industrial production, turning the fastener along the thread is done by dedicated equipment. Handheld and stationary nutrunners drive the fastener with a powered spindle, multi-spindle stations tighten several fasteners of one part at once, and robots carry screwdriving end effectors that feed and drive fasteners. None of these machines has to find the hole. A fixture clamps the part in a known pose, so the hole is always at the same place. The screw or nut arrives from a feeder in a known orientation, and the tool is matched to that one fastener type. This setup pays off in high-volume production. It stops working where parts vary, where the cell is not built around one product, and where batches are small. It also stops working in maintenance and disassembly, where the hole is wherever the part happens to be, as it is for a person kneeling next to a half-built wardrobe.

People handle the cases that dedicated equipment cannot. They pick up a nut, place it on a bolt, and turn it with their fingers, and they use the same hands to operate a wrench or a screwdriver. A robot with a dexterous, multi-fingered hand could in principle do the same. It could work with the parts and tools that were designed for people, in workcells that were designed for people, without a dedicated tool for every fastener [3], [4]. Learning-based control has made in-hand manipulation with such hands practical in simulation and on hardware [4], [5], [6]. Robots have fastened screws with manual screwdrivers [7], [8], and a single learned policy has manipulated a range of everyday tools with a dexterous hand [9]. The vision behind this thesis is a dexterous robot that helps in manufacturing and assembly in a world built for humans. Threading a nut by hand is one of the basic skills such a robot

needs. For humans the skill is trivial, but it is not for robots. This is an instance of Moravec's paradox: it is comparatively easy to make computers perform well on tasks that people find intellectually demanding, and difficult to give them the perception and mobility skills of a small child [10]. Industrial fastening shows both sides of this paradox. A nutrunner on a fixture drives a fastener without difficulty, because the cell design has already solved the hard part: the part is held in a known pose and the fastener is presented in a known orientation. Without this structure, a robot has to find the hole, align the fastener, and engage the thread, which a person does without thinking. If a dexterous robot is to thread, it has to learn the skill, and the next section explains why that is hard.

1.2 Problem: How Can a Robot Learn Threading?

Teaching a robot to thread a nut with a dexterous hand raises three difficulties at once. First, the skill has to be learned in simulation and transferred to hardware, although thread contact is very hard to simulate. Second, the dexterous hand has a high-dimensional action space and countless ways to solve the task. Third, threading is a long-horizon cyclic task. The following subsections treat the three difficulties in turn, and each ends with the requirement it puts on the method.

1.2.1 Simulation

Learning a contact-rich skill directly on hardware is slow and unsafe. Reinforcement learning (RL) needs many trials, every trial costs robot time and wear, and exploration produces collisions and excessive forces [11]. Simulation removes these limits. GPU-parallel simulators run thousands of environments at once and have made RL feasible for dexterous manipulation and for contact-rich assembly [6], [12], [13].

Thread contact, however, is among the hardest interactions to simulate. Engaged threads produce thousands of simultaneous contacts, and simulating them faithfully needs dedicated contact methods, such as the signed-distance-field contacts with which Factory simulates 1000 nut-and-bolt assemblies in real time on one GPU [13].

The GPU-parallel engines that make this scale possible, such as PhysX and MuJoCo, resolve contact by letting bodies overlap slightly and then pushing them apart again. For large bodies this overlap is harmless. For threads it is not, because the thread geometry is small: the pitch is 3 to 4 mm for the nuts used in this thesis. In simulation, a nut that is pressed along the bolt can therefore slip through the threads into the next groove instead of following the helix (Section 3.2.2).

A learning policy finds such shortcuts. If the reward pays for axial progress of the nut, the policy can collect it by pushing the nut through the threads rather than by turning it. The resulting behavior solves the simulated task and fails on hardware, where the threads are rigid and the nut advances only by rotating. Faithful thread contact therefore needs dedicated contact methods and is easy to exploit, and any simplification of the thread opens a gap in contact between simulation and reality. A deployed policy also has to perceive the nut through a camera, and rendered images differ from real ones, which opens a second gap in perception. Collecting real data to close these gaps would bring back the cost that simulation was meant to avoid.

Requirement: a thread model that is cheap enough for massively parallel RL and cannot be exploited through interpenetration, and a transfer recipe that carries the learned skill to hardware without real-world data.

1.2.2 Threading with a Dexterous Hand

A multi-fingered hand on an arm has far more actuated joints than a parallel-jaw gripper, and its action space is correspondingly high-dimensional [5], [14]. The task also has a large solution space. People thread a nut with a single finger, with a pinch between thumb and index finger, or with a grasp of several fingers, depending on how strongly the thread resists. A policy that may choose among all of these strategies has to explore a vast space of finger motions, and a reward that steers it toward one reliable strategy is hard to design [11].

A dexterous hand also has to acquire the grasp, keep it under load while the nut turns, and release it again. Learned assembly with parallel-jaw grippers does not face this part of the problem. There, the threading or insertion policy starts every episode with the part already held, so it learns to move a part that is grasped, and grasping, holding, and re-grasping are not part of what it learns [13], [15], [16]. Learned dexterous threading has so far addressed the spinning of an already engaged fastener [17] and goal-conditioned screwing of furniture parts [18].

Requirement: a learned policy that controls the hand and the arm together, with the solution space narrowed to one grasp strategy, the pinch grasp, which the policy has to acquire, hold, and release.

1.2.3 Threading as a Long-Horizon Cyclic Task

A long-horizon cyclic task repeats a fixed sequence of phases many times while the task state advances from one repetition to the next. Threading with a pinch grasp is such a task. One turn is limited by the range of the wrist and the fingers, so the hand has to approach the nut, grasp it, turn it, release it, and return for the next grasp, cycle after cycle. Meanwhile the nut advances along the thread by a fraction of a pitch in every cycle, so no two cycles start from the same state. Many tasks in assembly and everyday manipulation share this structure, for example driving a screw, turning a crank, winding a cable, or wiping a surface.

Three properties make such tasks hard. First, errors compound across cycles. A grasp that lands slightly off the intended faces, a release that disturbs the nut, or a return that brings the fingers back at the wrong height carries over into the next cycle and accumulates. For a learned policy this is worse, because small deviations lead it to states it was not trained on, where its errors grow further [19]. Second, failures concentrate at the transitions between phases, where the contact between hand and nut is made or broken, rather than within a single phase. Third, a single RL policy for the whole task needs a reward that covers every phase and every transition, and it has to assign the outcome of a late cycle to decisions made many steps earlier. Reward shaping over many phases and credit assignment over the full horizon break down for such a policy [11].

The common answer is to decompose the task into skills, train one policy per skill, and chain the policies at deployment [14]. Chaining moves the difficulty to the transitions. The final state of one policy has to lie in the region where the next policy succeeds, which needs extra machinery that fine-tunes the sub-policies for their neighbors [14], and several policies together with a switching logic have to be deployed. A second answer is to imitate complete long-horizon demonstrations with one generative policy [20], [21]. This avoids the reward problem, but it needs demonstrations of the whole task, and teleoperated demonstrations of forceful dexterous manipulation are hard to collect [9].

Requirement: a training signal that is easy to specify phase by phase, and one deployable policy that sustains many cycles without switching between separate policies.

Taken together, the three requirements point to one approach: decompose the task where the training signal is generated, and unify it where the policy is deployed.

1.3 Approach

This thesis decomposes the task during data generation and unifies it at deployment, as Figure 1.1 summarizes. Each part of the approach answers one of the difficulties of Section 1.2. A simplified kinematic thread model couples the axial travel of the nut to its rotation while keeping the finger–nut contacts, which makes massively parallel training with proximal policy optimization (PPO) feasible (Section 1.2.1). A privileged RL teacher, which observes the nut pose from the simulator, learns the pinch-grasp turn: approach, grasp, rotation, and goal reaching (Section 1.2.2). Scripted controllers release the nut and return the hand. Chained with the teacher, they form a hybrid teacher that generates complete multi-cycle demonstrations whose phase boundaries are triggered by physical events. A single visuomotor student is then distilled from these demonstrations by conditional flow matching (CFM) [22]. This teacher–student structure follows earlier work that distills privileged policies into sensor-based students [23], [24]. The student cannot observe the true nut pose, so it replaces the teacher's privileged nut state with perception. It acts from proprioception and a fixed RGB-D camera whose model is calibrated against the physical cell. The depth image is converted into a metric point cloud [25], and the rendering is randomized during data generation [23], so that the student sees similar inputs in simulation and on hardware. Teacher, student, and robot share one Cartesian action path. The student is therefore meant to run the whole cycle on hardware without privileged state, without switching between policies, and without real-world data.

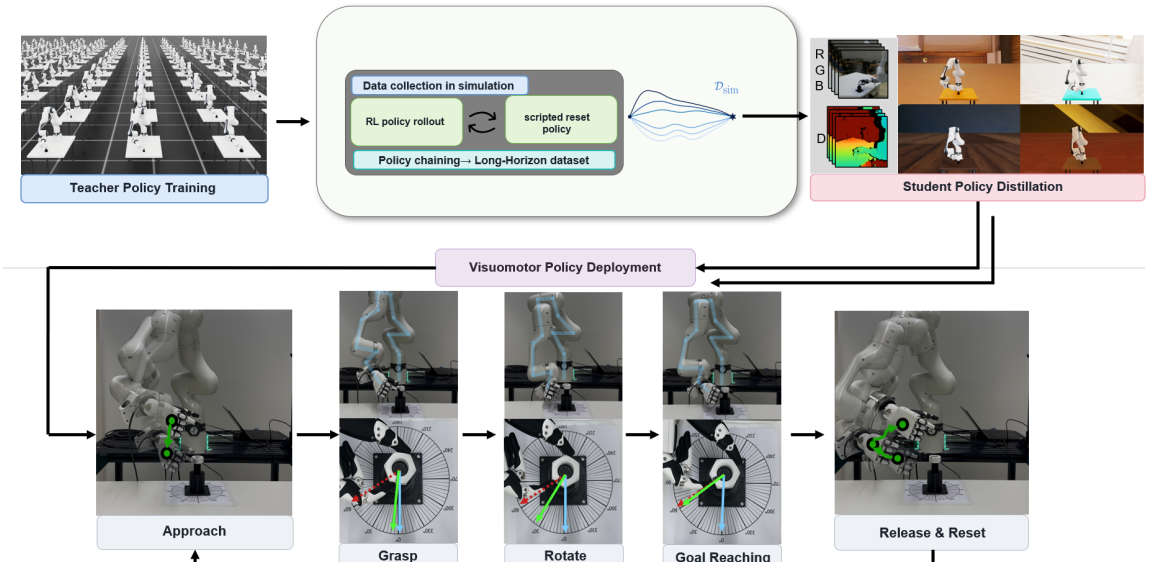


Figure 1.1: Overview of the approach: decompose during data generation, unify at deployment. Top: a privileged RL teacher is trained in parallel simulated workcells. During data collection its rollouts are chained with scripted release-and-reset controllers into long-horizon demonstrations $\mathcal{D}_{\text{sim}}$. A conditional flow-matching student is distilled from them and acts from RGB and depth. Bottom: the phases of one cycle on the physical robot, approach, grasp, rotation, goal reaching, and release-and-reset, after which the hand returns for the next cycle.

1.4 Research Questions

The approach rests on two claims, that one student can replace a chain of policies and that it can be trained on simulated data alone, and on one design choice, the student that performs the unification. Three research questions test them.

RQ1 (long horizon). Does one student distilled from complete multi-cycle demonstrations sustain consecutive threading cycles better than a single-cycle policy that is invoked repeatedly?

RQ2 (student design). Which student design distills the hybrid-teacher demonstrations into a policy that sustains the cyclic task? The ablations designed for it replace the flow-matching action head with deterministic regression on long short-term memory (LSTM) and Transformer backbones, and the point-cloud encoder with an image-space encoder.

RQ3 (sim-to-real). Can a student trained on simulated demonstrations only transfer zero-shot to the physical hand–arm system?

RQ1 tests the answer to the long-horizon difficulty, RQ2 asks which student makes the unification work, and RQ3 tests the answer to the transfer difficulty. Chapter 4 answers all three, RQ1 and RQ2 in simulation and RQ3 on hardware. The answer to the dexterity difficulty, the privileged teacher, is tested by the teacher training that Chapter 4 reports before the student experiments.

1.5 Contributions

The thesis makes the following contributions.

1. A formulation of cyclic nut threading as an ordered sequence of phases with explicit cyclic and episodic tracking errors, and a hybrid teacher that combines a privileged RL policy for approach, grasp, rotation, and goal reaching with scripted controllers for release and return. The hybrid teacher generates coherent multi-cycle demonstrations with physically gated phase boundaries.

2. A unified visuomotor student distilled by conditional flow matching from a point-cloud representation of an aligned RGB-D view, proprioception, a global trajectory-progress scalar, and a process-phase encoding. Ablations compare it with a single-cycle student, deterministic regression heads, and an alternative vision encoder. In simulation, the full student completes all ten cycles in every trial, and every ablation fails earlier.

3. A zero-shot sim-to-real recipe for the student: a camera model calibrated against the physical cell, a metric point cloud cropped to the workspace, visual randomization of the rendered demonstrations, and a Cartesian action path shared by teacher, student, and robot.

4. An evaluation of the student in simulation on the hexagonal training nut and on hardware across three nut sizes and four nut profiles. Both are reported in Chapter 4. On hardware, the student completes all ten cycles in every trial on the hexagonal nuts of all three sizes and on the held-out octagonal and round profiles, and it fails on the rectangular profile.

This work was carried out together with the advisor, who developed substantial parts of the hybrid teacher, the distillation stack, and the hardware experiments. The acknowledgments state this attribution.

1.6 Outline

This thesis comprises six chapters: Introduction, State of the Art, Methodology, Experiments, Discussion, and Conclusion and Future Work. This chapter motivated threading as a skill for dexterous robots, named the difficulties of learning it, and stated the approach and the research questions. Chapter 2 reviews the background and related work, Chapter 3 presents the method, Chapter 4 reports the experiments in simulation and on hardware, Chapter 5 interprets the results, and Chapter 6 concludes the thesis. The remainder of this section outlines these chapters in more detail.

Chapter 2 provides the background that the method builds on: the thread geometry that the thread model uses, RL with privileged information, imitation learning and compounding error, generative action heads, and operational-space control. It then reviews related work on robotic assembly and threading, dexterous manipulation, visual representations for manipulation, long-horizon tasks and policy chaining, and teacher–student distillation.

Chapter 3 presents the method, with each part answering one difficulty of Section 1.2: the problem formulation, the simulated system with its thread model and controller, the privileged teacher, the hybrid teacher and the generation of demonstrations, the distillation of the student by conditional flow matching, and the evaluation protocol.

Chapter 4 reports the experiments. It defines the evaluation metrics and the experimental setup, including the physical system and the zero-shot transfer protocol. It then reports the ablations for RQ1 and RQ2 in simulation and on hardware, and the real-world evaluation for RQ3 on the training nut, across nut sizes and geometries, and under long-horizon stress.

Chapter 5 interprets the results with respect to the research questions, discusses what they mean for robotic assembly, and states the limitations of the work.

Chapter 6 returns to the assembly motivation of this chapter, summarizes what a dexterous robot can now do in threading and what is still missing, and outlines future work.

Chapter 2

State of the Art

This chapter provides the background that the method of Chapter 3 builds on and places the thesis within related work. Section 2.1 introduces the theoretical concepts in the order in which the method uses them: the geometry of threaded joints that the thread model reproduces, reinforcement learning for the teacher, imitation learning for the distillation, action chunking and generative action heads for the student, and operational-space control for the action path that teacher, student, and robot share. Section 2.2 reviews related work along the three difficulties that threading poses for a robot: it covers robotic assembly and threading, dexterous manipulation with learned policies, visual representations for manipulation, long-horizon tasks and policy chaining, and teacher–student distillation, and it closes with the gap that this thesis addresses.

2.1 Theoretical Background

2.1.1 Threaded Joints and Thread Geometry

Screws are divided into fastening screws and motion screws, and a nut on a bolt is a fastening joint [2]. Threaded fasteners make a joint nonpermanent, so that it can be released again for field assembly, disassembly, maintenance, and adjustment [1]. Their threads are standardized, for example in the ISO metric M series [1]. The thread converts a large circumferential motion into a small axial motion, and a small circumferential force into a large axial force [2]. Assembling a threaded joint and securing it are treated as separate design topics [2]. This thesis addresses the assembly motion, turning the nut along the thread, and not the securing of the joint.

The quantity that links rotation and axial travel is the pitch ρ, the distance between adjacent threads [1]. For a single-start thread, adjacent threads are successive turns of one helix. A nut that turns by an angle $\Delta\theta$ about the bolt axis therefore advances along the axis by $\rho\,\Delta\theta/(2\pi)$, that is, by one pitch per revolution. Because a partial turn advances the nut by only a fraction of a pitch, the axial position at which a hand must grasp the nut changes with every turn.

2.1.2 Reinforcement Learning

In RL, an agent learns a behavior from interaction and a scalar reward, not from demonstrations [11], [26]. At every step an agent observes its environment, chooses an action, and receives a reward that rates the outcome. The agent has to discover the actions that lead to

high reward by trying them, and it has to attribute a reward to the earlier actions that made it possible.

The interaction is formalized as a Markov decision process (MDP) $(\mathcal{X}, \mathcal{A}, P, r, \gamma)$ with state space $\mathcal{X}$, action space $\mathcal{A}$, transition kernel $P(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$, reward $r(\boldsymbol{x}_t, \boldsymbol{a}_t)$, and discount factor $\gamma \in [0, 1)$ [26]. The Markov property states that the next state depends only on the current state and action, not on the earlier history. A stochastic policy $\pi(\boldsymbol{a} \mid \boldsymbol{x})$ maps states to distributions over actions. Executing it generates a trajectory $\tau = (\boldsymbol{x}_0, \boldsymbol{a}_0, r_0, \boldsymbol{x}_1, \dots)$, and the goal is a policy that maximizes the expected discounted return

$$J(\pi) = \mathbb{E}_{\tau \sim p_\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]. \tag{2.1}$$

The discount factor sets how far ahead the agent plans. A reward that lies t steps in the future is weighted by γ^t, so rewards beyond roughly $1/(1-\gamma)$ steps contribute little.

Value functions measure how good a state or an action is under a given policy. The state value $V^\pi(\boldsymbol{x}) = \mathbb{E}_\pi[\sum_{l \geq 0} \gamma^l r_{t+l} \mid \boldsymbol{x}_t = \boldsymbol{x}]$ is the expected return when the policy starts in $\boldsymbol{x}$. The action value $Q^\pi(\boldsymbol{x}, \boldsymbol{a})$ is the expected return when the agent first takes $\boldsymbol{a}$ and then follows π. Their difference is the advantage

$$A^\pi(\boldsymbol{x}, \boldsymbol{a}) = Q^\pi(\boldsymbol{x}, \boldsymbol{a}) - V^\pi(\boldsymbol{x}), \tag{2.2}$$

which is positive for actions that are better than the policy's average behavior in that state and negative for worse ones. The state value satisfies the Bellman equation $V^\pi(\boldsymbol{x}_t) = \mathbb{E}_\pi[r_t + \gamma V^\pi(\boldsymbol{x}_{t+1})]$, which relates the value of a state to the reward of one step and the value of its successor [26]. This recursion lets a value function be learned from single transitions instead of complete trajectories.

Value functions rate states and actions, but the policy itself can be improved directly by gradient ascent on the expected discounted return J of (2.1). This requires a differentiable policy π_θ, here a neural network with parameters θ, and a gradient of J that can be estimated from sampled trajectories. The likelihood-ratio form of the policy gradient,

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim p_{\pi_\theta}}\left[\sum_t \nabla_\theta \log \pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{x}_t)\, \Psi_t\right], \tag{2.3}$$

estimates this gradient from sampled trajectories alone, without a model of the transition kernel [26], [27]. The weight Ψ_t scores the action $\boldsymbol{a}_t$. With the return that follows step t as Ψ_t, as in REINFORCE [27], the estimate is unbiased but has high variance, because the return accumulates the randomness of every later action and transition.

Actor–critic methods reduce this variance by learning a value function alongside the policy. The actor is the policy π_θ, and the critic is a value network V_ϕ that is regressed onto the observed returns. The critic replaces the return in Ψ_t by an estimate of the advantage (2.2), that is, of how much better an action turned out than the critic expected. After one step, the critic compares its expectation $V_\phi(\boldsymbol{x}_t)$ with the observed reward plus its estimate of the next state. The difference is the temporal-difference residual $\delta_t = r_t + \gamma V_\phi(\boldsymbol{x}_{t+1}) - V_\phi(\boldsymbol{x}_t)$, which is positive when the step went better than expected. Generalized advantage estimation (GAE) [28] averages these residuals over the following steps with exponentially decaying weights,

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda_{\mathrm{GAE}})^l\, \delta_{t+l}, \tag{2.4}$$

where $\lambda_{\mathrm{GAE}} \in [0, 1]$ trades the bias of the critic against the variance of the observed returns.

The critic is needed only to compute $\hat{A}_t$ during training and is discarded afterwards. It may therefore receive information that the actor will not have at deployment. In an asymmetric

actor–critic [29], the critic observes the full, noise-free simulator state, while the actor acts only on the observations available to it. The privileged critic gives more accurate value estimates and therefore better advantages for the actor, but the actor never depends on the privileged state.

Proximal policy optimization (PPO) [30] is an on-policy actor–critic algorithm. On-policy methods estimate the gradient from data collected by the current policy, so every batch becomes outdated as soon as the policy changes. PPO reuses each batch for several epochs of minibatch updates and keeps the updated policy close to the one that collected the data. It maximizes the clipped surrogate objective

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\eta_t(\theta)\hat{A}_t,\ \text{clip}(\eta_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \qquad \eta_t(\theta) = \frac{\pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{x}_t)}{\pi_{\theta_{\text{old}}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)}, \tag{2.5}$$

where $\eta_t(\theta)$ is the probability ratio between the new and the old policy and ϵ is the clipping parameter. Once the ratio leaves the interval $[1-\epsilon, 1+\epsilon]$ in the direction that the advantage favors, the objective stops growing, and the gradient no longer pushes the policy further. The full loss adds a squared-error loss for the critic and an optional entropy bonus that keeps the policy stochastic,

$$L(\theta, \phi) = -L^{\text{clip}}(\theta) + c_V\, \mathbb{E}_t\Big[\big(V_\phi(\boldsymbol{x}_t) - \hat{R}_t\big)^2\Big] - c_H\, \mathbb{E}_t\big[\mathcal{H}\big(\pi_\theta(\cdot \mid \boldsymbol{x}_t)\big)\big], \tag{2.6}$$

with the return target $\hat{R}_t = \hat{A}_t + V_\phi(\boldsymbol{x}_t)$, the critic coefficient c_V, the entropy coefficient c_H, and the policy entropy $\mathcal{H}$ [30]. One training epoch collects a rollout of fixed length from every environment, computes the advantages (2.4), and then runs several passes of minibatch gradient descent on (2.6). On-policy learning needs many samples, and GPU-parallel simulators supply them by stepping thousands of environments at once. On such simulators, RL has trained in-hand reorientation that transferred to hardware [6] and regrasping and throwing with simulated hand–arm systems [12].

The reward is the only specification of the task that the agent receives, and its design largely decides whether learning succeeds [11]. A sparse reward that pays only on success is easy to specify, but in a high-dimensional action space random exploration almost never reaches success, so the agent receives no learning signal. Reward shaping adds dense terms that pay for intermediate progress, such as moving the hand toward the object or turning it toward the goal. Shaping can, however, change which policy is optimal. Ng et al. [31] show that a shaping term of the form $F(\boldsymbol{x}_t, \boldsymbol{x}_{t+1}) = \gamma\Phi(\boldsymbol{x}_{t+1}) - \Phi(\boldsymbol{x}_t)$, with an arbitrary potential function Φ over states, leaves the optimal policy unchanged, and that other shaping terms carry no such guarantee. A term that is not of this form may be exploited. The agent optimizes the reward as written rather than the task as intended. A term that pays for every reduction of a distance, for example, pays again each time the agent moves away and comes back, so a policy can collect it by oscillating. In simulation the agent can also exploit inaccuracies of the physics, as the interpenetration through the thread flanks of Section 3.2.2 illustrates.

RL thus produces a policy from a reward alone, at the cost of many simulated trials and a carefully shaped reward. When an expert policy or recorded demonstrations are available, a policy can instead be learned by supervised imitation, which the next subsection introduces.

2.1.3 Imitation Learning

Imitation learning (IL) is the problem of learning a policy from the demonstrations of an expert rather than from a reward. IL methods differ in how they use the demonstrations.

The simplest method, behavior cloning (BC), reduces IL to supervised learning [19]. It treats every demonstrated pair of observation and expert action as a labeled training example and fits the learned policy, called the student, to these labels. Given a dataset $\mathcal{D} = \{(\boldsymbol{y}_t, \boldsymbol{u}_t^*)\}$, BC solves the regression problem

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(\boldsymbol{y},\boldsymbol{u}^*)\sim\mathcal{D}}[\ell(\pi_\theta(\boldsymbol{y}), \boldsymbol{u}^*)], \tag{2.7}$$

with a per-sample loss ℓ such as the squared error. BC is trained on the states that the expert visits but evaluated on the states that the student visits. Small prediction errors move the student into states that are not covered by the demonstrations, and the errors compound. Ross et al. [19] show that the expected cost of a BC policy with per-step error ε can grow as $O(T^2\varepsilon)$ over a horizon T. For a cyclic task, small deviations in grasp height or nut angle carry over from one cycle into the next.

Dataset aggregation (DAgger) [19] counters this by querying the expert on the states that the student itself visits. The environment is driven by a mixture that executes the expert's action with probability β and the student's action otherwise, the expert labels every visited state, and the student is retrained on the aggregated dataset. The expected cost then grows only linearly in the horizon. DAgger requires an expert that can be queried at arbitrary states. A policy in simulation satisfies this requirement, a recorded demonstration does not.

2.1.4 Action Chunking

A policy that predicts one action per observation is queried at every control step. An action-chunking policy instead predicts a sequence of H future actions, the action chunk $[\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}]$, from the observation $\boldsymbol{y}_t$ [32]. It executes the first $H_\mathrm{e} \leq H$ actions of the chunk without querying the policy again and then predicts a new chunk from the current observation. The policy thus plans further ahead than it acts, which keeps consecutive actions consistent while the policy still responds to new observations at every query [20]. Chunking has three benefits for imitation learning and one cost.

First, chunking counters the compounding errors of BC (Section 2.1.3). A policy that executes H_e actions per query makes only T/H_e decisions over a task of T steps. The effective horizon of the task shrinks by the factor H_e, and fewer errors can compound [32].

Second, chunking handles demonstrations that are not Markovian in the observation. Human demonstrators pause in the middle of a demonstration, for example during teleoperation, and the observation does not show how long the pause has lasted. A single-step policy learns that it stands still in such a state, but not for how long, and it can get stuck [20], [32]. A chunk covers the pause together with the motion that follows it, so the policy can learn to resume. Similarly, when the demonstrations contain several valid behaviors, a policy that samples every action independently can switch between them from one step to the next and produce jittery motion. A chunk is sampled as a whole and follows one behavior [20].

Third, chunking decouples inference from the control rate. A large policy, such as a vision-language-action model, can need longer for one inference than one control period. A chunk supplies actions for several control periods, so the robot keeps moving while the next chunk is computed [33]. In this way, a vision-language-action model controls robots at up to 50 Hz [21].

The cost of chunking is reactivity. While the actions of a chunk execute, the policy does not observe their effect and cannot respond to a disturbance before its next query. In tasks with uncertain contact, a long execution horizon therefore trades robustness against compounding errors for responsiveness to the contact. Two remedies keep the chunk prediction and restore feedback. Temporal ensembling queries the policy at every step and executes an

exponentially weighted average of all predictions for the current step from the chunks that are still valid [32]. Real-time chunking generates the next chunk while the current one executes. It fixes the actions that will execute before the new chunk is ready and fills in the rest of the new chunk consistently with them, which avoids discontinuities at the switch between chunks [33]. Predicting a chunk requires a model of the distribution over action sequences, which the generative action heads of the next subsection provide.

2.1.5 Generative Action Heads: Diffusion and Flow Matching

A generative model learns a probability distribution from a set of samples and produces new samples from it. Diffusion models [34] do so by transforming Gaussian noise step by step into a data sample with a learned network. Conditioned on a text prompt, latent diffusion models generate high-resolution images that match the prompt [35].

In robot learning, the same principle generates actions instead of images. A generative action head learns the conditional distribution $p(\boldsymbol{u} \mid \boldsymbol{y})$ of the demonstrated actions given the observation and draws an action from it at run time. A regression head instead predicts a single action per observation, and the Gaussian policy of Section 2.1.2 represents only a single mode. The generative action head has two advantages. First, when the demonstrations contain several valid actions for the same observation, a sample takes one of them rather than their average [20]. The average of two valid actions need not be valid itself, for example: if an obstacle can be passed on either side, the average drives into it. Second, the head can generate a whole action chunk (Section 2.1.4) as one sample, so that all actions of the chunk follow the same mode [20]. Generative action heads are also used in large generalist policies, such as the vision-language-action model π_0, which is trained on data from many robots and tasks [21]. Two families of generative action heads are common in robot learning, diffusion and flow matching. Both start from Gaussian noise and transform it step by step into an action, but they learn this transformation differently.

Diffusion Policy [20] learns to remove noise. During training, Gaussian noise is added to a demonstrated action, and a network that is conditioned on the observation predicts the added noise. The magnitude of the noise depends on a randomly drawn iteration index through the noise schedule, a hand-designed sequence of noise variances that fixes how much noise each iteration adds or removes. At run time, sampling starts from pure noise and applies the network over a fixed number of denoising iterations, each of which removes part of the predicted noise. The number of iterations sets the inference cost. Diffusion Policy predicts action chunks and executes several actions of a chunk before it predicts again.

Flow matching [22], [36] learns the transformation from noise to action as a velocity field. Each noise sample can be pictured as a particle that moves during a flow time λ from 0 to 1. A network $\boldsymbol{v}_\theta(\boldsymbol{u}, \lambda \mid \boldsymbol{y})$ predicts the velocity of the particle at position $\boldsymbol{u}$ and time λ, and integrating this velocity from a noise sample $\boldsymbol{u}^0$ at $\lambda = 0$ yields an action $\boldsymbol{u}^1$ at $\lambda = 1$. The velocity field that maps the noise distribution onto the demonstrated actions is unknown, but a training target can be built from pairs of a noise sample and a demonstrated action. The simplest construction connects the two by a straight line,

$$\boldsymbol{u}^\lambda = (1-\lambda)\boldsymbol{u}^0 + \lambda \boldsymbol{u}^1, \qquad \boldsymbol{u}^0 \sim \mathcal{N}(\boldsymbol{0}, I), \tag{2.8}$$

along which the velocity is the constant $\boldsymbol{u}^1 - \boldsymbol{u}^0$. Training regresses the network onto this velocity at random points of random lines,

$$L(\theta) = \mathbb{E}_{(\boldsymbol{y},\boldsymbol{u}^1)\sim\mathcal{D},\, \boldsymbol{u}^0\sim\mathcal{N}(\boldsymbol{0},I),\, \lambda\sim U[0,1]} \left\| \boldsymbol{v}_\theta(\boldsymbol{u}^\lambda, \lambda \mid \boldsymbol{y}) - \left(\boldsymbol{u}^1 - \boldsymbol{u}^0\right) \right\|_2^2. \tag{2.9}$$

Many lines pass through the same point, so the network learns the average of their velocities. Lipman et al. [22] show that this average is exactly the unknown velocity field, so the simple loss (2.9) trains the correct model. At run time, an action is generated by drawing $\boldsymbol{u}^0 \sim \mathcal{N}(\mathbf{0}, I)$ and following the learned field with a few explicit Euler steps $\boldsymbol{u}^{\lambda+\Delta\lambda} = \boldsymbol{u}^{\lambda} + \Delta\lambda\, \boldsymbol{v}_\theta(\boldsymbol{u}^{\lambda}, \lambda \mid \boldsymbol{y})$.

Diffusion and flow matching are closely related. A diffusion model corresponds to flow matching with a particular Gaussian path between noise and data, whose trajectories are curved [22]. Figure 2.1 contrasts the two. The straight-line path of (2.8) gives flow matching three advantages. First, training is simpler. The regression target is the difference of two samples, and the path fixes how noise and data are mixed over the flow time, so no noise schedule has to be designed. Second, sampling is faster. A straight path can be followed with a few large integration steps, whereas a curved path needs many small ones. On image generation, flow matching reaches the sample quality of diffusion with fewer network evaluations [22]. Third, sampling is the deterministic integration of an ordinary differential equation (ODE) from a given noise sample. The number of integration steps can therefore be chosen at run time, which trades accuracy against inference time without retraining. Fast sampling matters for robot control, because a new action chunk must be ready within the control period. π_0 uses flow matching to generate action chunks for control at up to 50 Hz [21].

In a policy, the flow-matching action head sits on top of a backbone. The backbone encodes the observation, for example camera images, the robot state, or a language instruction, into a condition vector $\boldsymbol{c}$, which takes the place of $\boldsymbol{y}$ in (2.9). The head is the velocity network $\boldsymbol{v}_\theta(\boldsymbol{u}^{\lambda}, \lambda \mid \boldsymbol{c})$. It receives the noisy action or action chunk $\boldsymbol{u}^{\lambda}$, the flow time λ, and the condition, and it outputs a velocity of the same dimension as the action. The flow time is a scalar and is usually embedded with sinusoidal features before it enters the network [21]. The velocity network is typically an multilayer perceptron (MLP) or a Transformer. The condition enters it by concatenation with the input, by a feature-wise scaling and shifting of the hidden layers [20], or as additional tokens that the action tokens attend to [21]. The backbone is evaluated once per generated action chunk, the head once per integration step. Each additional step therefore costs only one evaluation of the head. π_0 follows this structure: a second set of Transformer weights, called the action expert, generates action chunks by flow matching and attends to the features of a pre-trained vision-language model [21]. Section 3.5.2 describes the backbone and the head of the student in this thesis.

2.1.6 Operational-Space Control

A policy controls a robot arm through its action space, and the choice of this space decides how much of the motion the policy has to learn itself. The most direct choice is the joint space. On a seven-DoF arm, the policy outputs seven joint position targets at every step, and a joint position controller tracks them in simulation and on the hardware. The policy then learns the complete motion of the arm, including the coordination of the joints that moves the end effector to where the task needs it. Joint-space actions impose no structure on this motion and leave the policy free to explore. Grasping pipelines that train arm and hand together with RL use them. For example, a humanoid with two seven-DoF arms learns to grasp, lift, and hand over objects with joint position targets for its arm and hand joints [37].

The alternative moves part of the motion into a controller that runs both during training and at deployment. The policy then commands a target in the operational space, the space in which the task is described [38]. For assembly, this is the six-dimensional pose of the end effector, its position and orientation in Cartesian space, and the controller computes the joint torques that move the end effector toward the commanded pose. The action space shrinks from the number of joints to six dimensions, and it is expressed in the coordinates of the task,

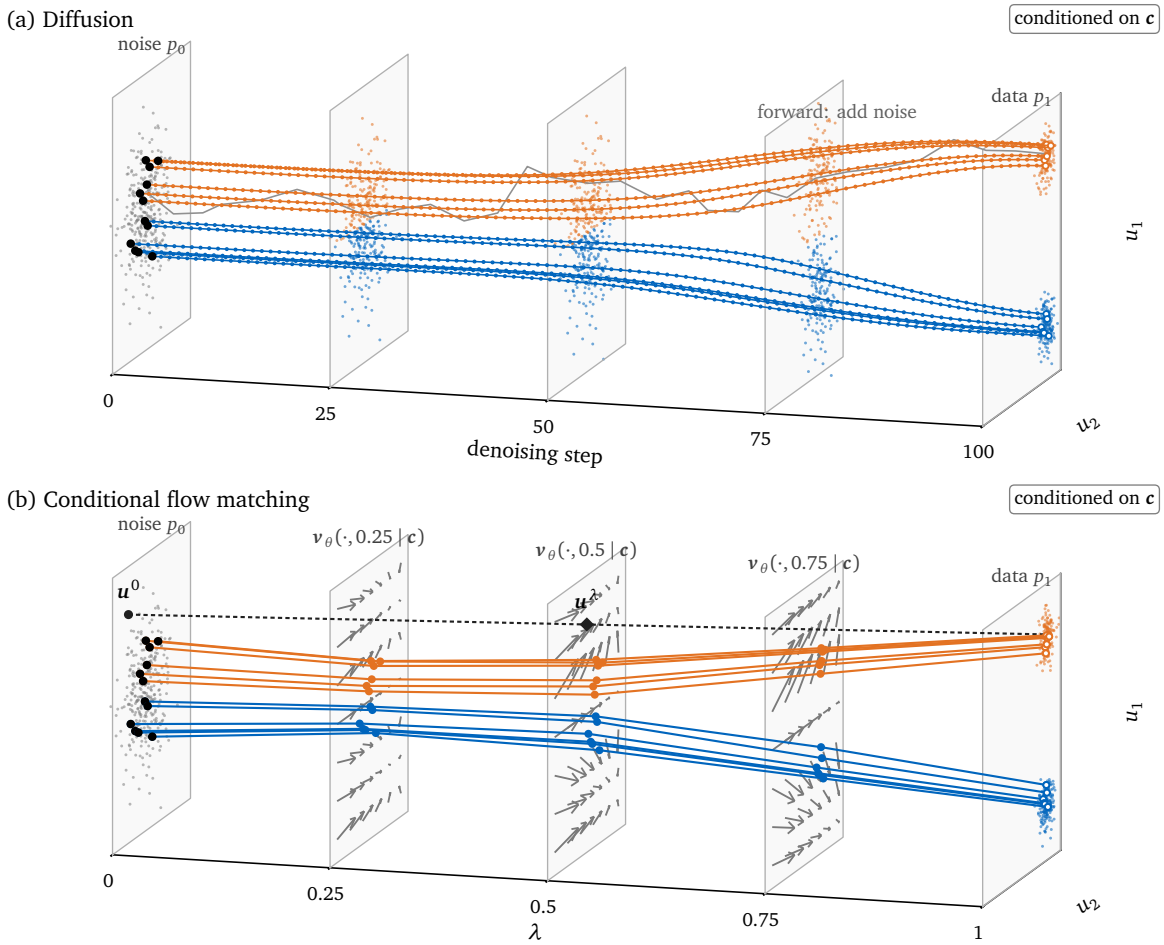


Figure 2.1: Diffusion and conditional flow matching for a bimodal action distribution in a two-dimensional toy action space (u_1, u_2). Generation time runs from left to right, from Gaussian noise p_0 to the demonstrated actions p_1, and each slice shows the action space at one point in time. (a) A diffusion model adds noise to demonstrated actions (gray path) and learns to reverse this process over many denoising steps along curved trajectories. The point clouds on the slices are intermediate denoising samples, colored by the mode they reach. (b) Conditional flow matching regresses a velocity field onto the straight path between a Gaussian sample and a data sample (dashed). Integrating the learned field (gray arrows on the slices at $\lambda = 0.25$, 0.5, and 0.75) from noise reaches either mode in a few ODE steps.

for example relative to a bolt. With an impedance controller in end-effector space, RL learns contact-rich tasks with fewer samples than with joint-space actions [39].

Impedance is the second reason for this choice. A stiff position controller drives through an unexpected contact and can produce forces that damage the robot or the parts. An impedance controller instead makes the end effector behave as if it were attached to the target by a spring and a damper [40]. In steady contact, the force on the environment equals the stiffness times the deviation of the end effector from its target, so a small target offset produces only a small force. This protects the robot and the environment. It also gives the policy a way to apply force: a target that lies a small distance behind a contact surface presses the end effector against the surface with the stiffness times that distance. The policy thus regulates contact forces by placing targets, without commanding forces directly.

Task-space motion control with torque inputs and impedance control are treated in standard references [40], [41], [42]. Impedance control is simplest in joint space. Assuming that

gravity is compensated, the law

$$\boldsymbol{\tau} = K_q(\boldsymbol{q}_{\text{ref}} - \boldsymbol{q}) - D_q\dot{\boldsymbol{q}} \tag{2.10}$$

makes every joint behave like a spring with stiffness K_q and a damper with damping D_q around its reference $\boldsymbol{q}_{\text{ref}}$, where $\boldsymbol{q}$ are the joint positions (Figure a). The spring then acts on the joints, not on the end effector, and the stiffness felt at the end effector changes with the arm configuration. Operational-space impedance control instead defines the spring and the damper directly in the six-dimensional task space and maps them to the joints with the Jacobian. Joint space and operational space are related by the forward kinematics

$$\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{q}), \tag{2.11}$$

which gives the pose $\boldsymbol{x}$ of a frame on the robot for the joint positions $\boldsymbol{q}$. The Jacobian is the derivative of this map, $J(\boldsymbol{q}) = \partial \boldsymbol{f}/\partial \boldsymbol{q} \in \mathbb{R}^{6\times 7}$. By the chain rule, it relates the velocities in the two spaces,

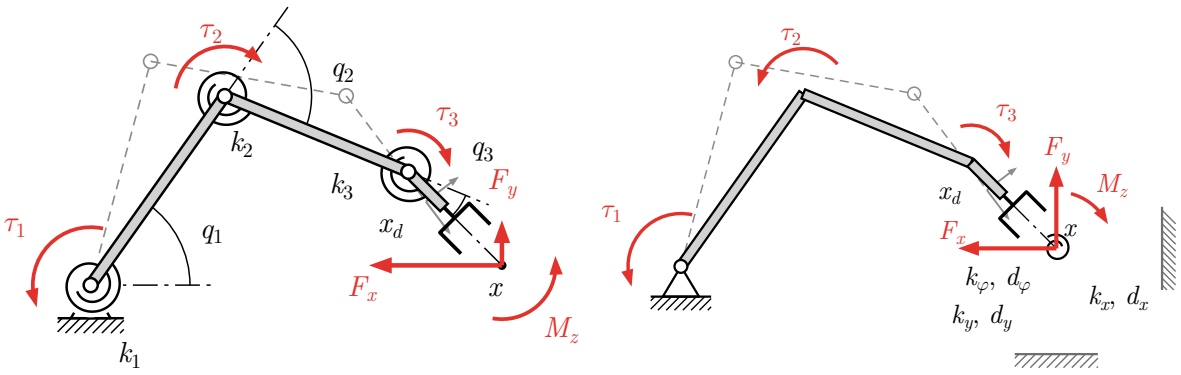
$$\dot{\boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{q}} \frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}t} = J(\boldsymbol{q})\,\dot{\boldsymbol{q}}. \tag{2.12}$$

The relation is linear in the joint velocities, but the Jacobian itself changes with the configuration. For the orientation, the time derivative of a parameterization such as Euler angles is not the angular velocity of the frame. Controllers therefore use the geometric Jacobian, which maps the joint velocities to the twist $\dot{\boldsymbol{x}}$ of the frame, its linear and angular velocity [41]. By the principle of virtual work, its transpose maps a wrench $\boldsymbol{F}$ at the frame to the joint torques that produce it, $\boldsymbol{\tau} = J^{\mathrm{T}}\boldsymbol{F}$ [41]. The frame can be any point on the robot, for example the point between the fingertips at which the hand holds the nut. A virtual spring and damper at this frame (Figure b), $\boldsymbol{F} = K_p\boldsymbol{e} - K_d\dot{\boldsymbol{x}}$ with the position and axis-angle orientation error $\boldsymbol{e} \in \mathbb{R}^6$ between target and current pose, therefore translates directly into joint torques, and the controller computes them at every step from the current Jacobian. On the seven-DoF arm, the six task dimensions leave one redundant degree of freedom, which a second term controls. The complete law, in the form that the thesis uses, is

$$\boldsymbol{\tau} = J^{\mathrm{T}}\big(K_p\boldsymbol{e} - K_d\dot{\boldsymbol{x}}\big) + N^{\mathrm{T}}M\big(K_{p,0}(\boldsymbol{q}_0 - \boldsymbol{q}) - K_{d,0}\dot{\boldsymbol{q}}\big), \tag{2.13}$$

where K_p and $K_d = 2\sqrt{K_p}$ are diagonal stiffness and critical-damping gains, and the second term pulls the redundant degree of freedom toward a rest posture $\boldsymbol{q}_0$ through a nullspace projector N with posture gains $K_{p,0}$ and $K_{d,0}$. The joint-space inertia M converts the posture term from a joint acceleration into a torque.

The nullspace projector decides how strongly the posture term interferes with the task. It is dynamically consistent when $N^{\mathrm{T}} = I - J^{\mathrm{T}}\bar{J}^{\mathrm{T}}$ with the inertia-weighted pseudoinverse $\bar{J} = M^{-1}J^{\mathrm{T}}(JM^{-1}J^{\mathrm{T}})^{-1}$ and joint-space inertia M [43], [44]. Torques in the nullspace then produce no acceleration of the controlled frame. A projector built from a pseudoinverse of the Jacobian without inertia weighting needs no inertia model, but it decouples the nullspace from the task only statically, so that posture torques can still disturb the task during motion [43]. The controllers in simulation and on the physical robot both use the dynamically consistent projector (Section 4.2). Equation (2.13) is the Factory and FORGE controller [13], [15]. Scaling the spring and damper term by the task-space inertia $\Lambda = (JM^{-1}J^{\mathrm{T}})^{-1}$ and compensating the nonlinear dynamics yields the full operational-space formulation, in which the closed loop becomes a decoupled unit-mass system in every task direction [41].



(a) Joint-space impedance. **(b)** Operational-space impedance.

Figure 2.2: Impedance control, illustrated on a planar arm with three joints. (a) A virtual spring and damper with stiffness k_1, k_2, k_3, the diagonal entries of K_q, act directly on each joint, pulling it toward its reference q_{ref}. (b) A virtual spring and damper with stiffness k_x, k_y, k_φ and damping d_x, d_y, d_φ, the diagonal entries of K_p and K_d, act on the controlled end-effector frame along each task direction instead. The resulting wrench (F_x, F_y, M_z) is mapped to the joint torques τ_1, τ_2, τ_3 through J^{T}.

2.2 Related Work

The related work is ordered along the difficulties of threading. Section 2.2.1 reviews robotic assembly and threading, the application itself. Section 2.2.2 reviews learned dexterous manipulation (difficulty (b)), Section 2.2.3 reviews visual representations for manipulation, and Sections 2.2.4 and 2.2.5 review long-horizon tasks and the teacher–student distillation with which this thesis addresses them (difficulty (c)). Section 2.2.6 states the gap.

2.2.1 Robotic Assembly and Threading

Robotic assembly, for example peg-in-hole insertion, gear meshing, connector mating, and nut–bolt fastening, requires contact-rich interaction with high precision and accuracy [16]. In mass production, dedicated fixtures position the parts precisely and thereby reduce the uncertainty of the hole pose [45]. Learning-based methods train assembly policies in simulation. Factory [13] combines signed-distance-field collision detection, contact reduction, and a Gauss–Seidel solver to simulate 1000 nut-and-bolt assemblies in real time on a single GPU, and it learns nut-and-bolt assembly with RL through separate pick, place, and screw subpolicies. FORGE [15] trains force-guided exploration under pose uncertainty and demonstrates on a real robot a multistage planetary-gear assembly that includes nut threading, and IndustReal [16] transfers simulation-trained assembly policies to a physical arm. All three use a parallel-jaw gripper.

Screwdriving with a manual screwdriver has been addressed with analytical models and force control, first with the tool rigidly attached to a robot arm [7] and then with the tool held by a Shadow Hand [8]. For an industrial screwdriving tool on an impedance-controlled arm, a physics-informed learned model describes the motion of the screw tip [45]. With multi-fingered hands, twisting the lids off bottle-like objects has been learned with two hands by sim-to-real RL [46], and unscrewing a bottle lid with one hand has been learned from teleoperated demonstrations with a visuomotor diffusion policy [47]. Play2Perfect [18] pretrains a goal-conditioned dexterous manipulation policy through play and fine-tunes it for precise assembly, including screwing a furniture leg into a table fixture. It thus covers goal-directed screwing of a furniture leg, but not of small fasteners such as nuts. The only reviewed work

that fastens nuts with a dexterous hand is DexScrew [17]. It mounts a 12-DoF hand on a robot arm and replaces the thread geometry by a revolute-joint proxy to learn a rotational finger gait with RL. The gait assists teleoperation and supports the collection of real demonstrations for tactile behavior cloning. The RL policy is rewarded for the angular velocity of the fastener rather than for reaching a target angle, its episodes end on failure or after a fixed time limit rather than on reaching a goal, and the nut is already installed on the bolt. Goal-directed threading over many grasp-and-release cycles, which this thesis addresses, is left open by these works.

2.2.2 Dexterous Manipulation with Learned Policies

A multi-fingered hand is a general-purpose alternative to task-specific tooling, but it brings high-dimensional control and complex contact dynamics with it [3], [4]. According to a recent survey, analytical models of motion and contact have failed to provide practical solutions for in-hand manipulation because they cannot cope with complex environments and uncertainty, and research has therefore shifted to learning [4]. Model-free RL in simulation learns in-hand reorientation of more than two thousand objects [5]. DeXtreme [6] transfers agile in-hand reorientation from simulation to a comparatively affordable Allegro hand with a vision-based pose estimator, and DexPBT [12] learns regrasping and grasp-and-throw for simulated hand–arm systems with population-based training. DextrAH-G [24] and DextrAH-RGB [23] grasp a wide range of objects with a hand–arm system from depth and RGB images, respectively, and SimToolReal [9] manipulates tool-like objects to random goal poses and generalizes zero-shot to real tools. Without learning, an arm-driven hand has executed a finger gait on a tool whose tip pivots on a supporting plane, using hybrid position/impedance control with reference finger forces [48].

Several works name the high-dimensional action space of the hand or of the hand–arm system as a key challenge of dexterous manipulation [5], [14], [24]. Most of the works above learn one skill, such as grasping, reorienting an object, or moving a tool to goal poses. In-hand reorientation chains goals and counts consecutive successes [6], but the object stays in the hand. Threading additionally requires the hand to release the part and acquire it again while the bolt constrains it, and the grasp must hold under the turning load in between. This thesis narrows the solution space to one grasp strategy, a thumb–index pinch, and lets the learned policy control this pinch together with the arm (Section 3.2).

2.2.3 Visual Representations for Manipulation

A visuomotor policy must extract the task-relevant geometry from camera data, in threading the pose of the nut relative to the hand. Two families of encoders, and combinations of them, are used for this. Image-space encoders apply a convolutional backbone such as ResNet-18 [49] to the RGB image, possibly with depth or other per-pixel channels added, and pool the feature map into a vector. They can start from ImageNet-pretrained weights, but from RGB alone they have to infer metric geometry from appearance, and when they are trained on rendered images they must be made robust to texture and lighting, for example by visual domain randomization [50]. Point-cloud encoders back-project the depth image through the calibrated camera model into a set of 3D points, optionally attach the pixel color, and encode the set with a shared point network followed by global max pooling in the spirit of PointNet [51]. The 3D Diffusion Policy (DP3) encoder [25] is such a compact encoder, a three-layer MLP with max pooling and a projection head, and it uses the point coordinates without color. In real-robot tests with one trial per condition, DP3 generalizes to unseen

object colors and slightly changed camera views on which image- and depth-image-based Diffusion Policy [20] fail. The authors attribute the robustness to appearance to the omission of color and the spatial generalization to the 3D representation [25]. Adapt3R [52] combines the two families. Dense 2D features from a pretrained backbone are localized in 3D by the depth map and expressed in the end-effector frame before attention pooling. In simulation, this lets a policy transfer zero-shot to unseen robot arms and camera viewpoints, although its success rate on unseen arms is lower than on the training arm. The student of this thesis uses a point-cloud encoder in the style of DP3 whose points carry their pixel color, and Chapter 4 sets up its comparison with an image-space encoder and with an Adapt3R-style encoder.

2.2.4 Long-Horizon Tasks and Policy Chaining

A long-horizon task can be learned by one policy over the whole horizon or decomposed into skills, and each route has its own difficulties. One policy trained with RL needs a reward that covers every phase, and it has to assign the outcome of a late phase to decisions made many steps earlier. AlphaNIST [53] shows in simulation that a sparse binary task reward can suffice. It trains one policy from scratch on the 20 assembly tasks of the NIST Assembly Task Board #1, which require precise, contact-rich control of small, tight-fitting parts, and reaches 98.2 % overall training success without demonstrations, progress shaping, or scripted primitives. The same framework trains policies that complete ten assembly pairs in one episode without a prescribed order, rewarded only when all ten pairs are complete. This thesis takes a different route. It applies RL only to the pinch-grasp turn, whose reward covers a single cycle (Section 3.3), and reaches the long horizon by imitation.

Imitation avoids the reward problem but moves the difficulty to the data. One policy that imitates the whole task needs demonstrations of the whole task, and teleoperated demonstrations of forceful dexterous manipulation are hard to collect [9].

Decomposing the task into skills shortens the horizon that each skill covers, but it creates difficulties of its own. If one network learns the skills one after another, fine-tuning it on later skills can erase what it learned on earlier ones. This catastrophic forgetting affects supervised and reinforcement learning alike [54]. Separate policies per skill avoid forgetting, but chaining them introduces a transition problem. The states in which one skill terminates may lie outside the initial-state distribution on which the next skill was trained, and the next skill can then fail [55]. Adversarial skill chaining [55] regularizes the terminal-state distribution of each skill toward the initial-state distribution of its successor. Sequential Dexterity [14] learns a transition-feasibility function that fine-tunes each sub-policy toward states from which its successors succeed, and it uses the same function to switch between sub-policies autonomously, which lets it recover from failures and skip redundant stages. A cyclic task sharpens the transition problem. The same boundaries recur in every cycle while the task state advances, so a task of many cycles crosses each boundary many times, and an error at one crossing carries into the following cycle.

Generative action heads make it practical for one imitation policy to cover a multi-stage task. Such a head samples one of several demonstrated actions for similar observations instead of their average (Section 2.1.5). The flow-matching model π_0 [21] is trained on data from several dexterous robot platforms and applied to multi-stage tasks such as laundry folding, table cleaning, and box assembly. Its demonstrations are collected by teleoperation [21], which is the kind of data that is hard to obtain for forceful dexterous manipulation. Section 2.2.5 returns to these action heads.

The method of this thesis decomposes the task into separately controlled phases only during data generation. The privileged RL teacher executes the pinch-grasp turn, scripted controllers execute release and return, and together they generate complete multi-cycle demon-

strations in simulation without teleoperation (Section 3.4). One visuomotor student with a flow-matching head is distilled from these demonstrations and deployed. It learns all phases in one training run, so no phase is fine-tuned after another and catastrophic forgetting does not arise. The phase enters the student only as a conditioning signal derived from physical events, not as a switch between networks, and the student predicts the transition actions itself (Section 3.5.2). The transitions are thus part of what the student learns rather than a problem to be solved between separate policies. RQ1 tests whether this unified student sustains consecutive cycles better than a single-cycle policy that is invoked repeatedly.

2.2.5 Teacher–Student and Visuomotor Distillation

Learning by Cheating [56] trains a privileged agent on ground-truth simulator state by imitating an expert and then uses it to supervise a sensorimotor agent that acts from a forward-facing camera image, the vehicle speed, and a high-level command. In dexterous manipulation, DextrAH-G [24] and DextrAH-RGB [23] distill a privileged RL grasping teacher into a depth-based and an RGB-based student, respectively, with online DAgger. UniGraspTransformer [57] distills 3200 object-specific RL teachers into one Transformer student, and Lin et al. [37] train separate per-object RL specialists on a humanoid and distill their successful rollouts offline into one generalist policy per task that acts from egocentric depth, the object position, and the joint states.

Two further design choices of visuomotor policies matter for this thesis, action chunking and the generative action head. Action chunking (Section 2.1.4) was introduced to imitation learning by Action Chunking with Transformers (ACT) [32], which learns fine-grained bimanual tasks on low-cost arms from teleoperated demonstrations. ACT predicts action chunks with a Transformer-based conditional variational autoencoder and averages the overlapping predictions for the current step with exponential weights, which it calls temporal ensembling. Diffusion Policy [20] predicts action sequences with a generative head and executes a fixed number of their actions before it replans. Its authors attribute to sequence prediction that the policy commits to one mode of multimodal demonstrations and does not get stuck on pauses in them. Chunking has since become a standard component of generalist policies. The vision-language-action model π_0 [21] generates chunks of 50 actions with ten flow-matching integration steps and controls robots at up to 50 Hz. It executes the first 16 or 25 actions of each chunk open loop, because temporal ensembling reduced its performance in early experiments [21]. Executing chunks without pauses for inference has become a research topic of its own. Real-time chunking generates the next chunk while the current one executes and constrains it to the actions that are already committed [33].

A generative model of the action distribution can represent multimodal demonstrated behavior, for which explicit policies trained by direct regression are not suitable [20]. Diffusion Policy [20] models multimodal action sequences by iterative denoising, and DP3 [25] conditions it on compact point-cloud features and the robot pose. Flow matching [22] regresses a continuous vector field that transports noise to data, and samples are generated by integrating an ODE. The student of this thesis combines a point-cloud encoder, action chunking with temporal ensembling, and a flow-matching head. The generative head suits its training data, which mix nominal teacher actions with the teacher's recoveries from disturbances.

2.2.6 Summary and Research Gap

The related work addresses each difficulty of threading separately. Simulation-based assembly learning makes thread contact tractable [13] and transfers insertion and nut threading

to hardware [15], [16], but with a parallel-jaw gripper, a part that is already held when contact begins, and no regrasping during threading [13], [15]. Learned dexterous screwing covers a spinning gait for an installed nut [17] and goal-conditioned screwing of a furniture leg [18]. Policy chaining reaches long horizons at the cost of transitions between separately trained policies [14], [55], and teacher–student distillation yields deployable visuomotor policies for single skills [23], [24]. None of the reviewed works threads a nut over many grasp-and-release cycles with a dexterous hand. Chapter 3 addresses this gap by combining the concepts of Section 2.1: a simplified thread model makes RL in simulation feasible, a privileged teacher learns the pinch-grasp turn, scripted transitions chain it into multi-cycle demonstrations, and one flow-matching student distills these demonstrations for zero-shot deployment.

Chapter 3

Methodology

This chapter presents the method. Section 3.1 formalizes cyclic nut threading. Section 3.2 describes the simulated and physical system: the hand–arm platform, the simplified thread model that makes learning in simulation tractable, and the action contract with its controller. Section 3.3 presents the privileged reinforcement-learning teacher that learns to grasp and turn the nut with the dexterous hand. Section 3.4 explains how the hybrid teacher chains this skill with scripted transitions into complete multi-cycle demonstrations, and how the visual observations of these demonstrations are collected and randomized. Section 3.5 develops the central contribution, the distillation of these demonstrations into a single visuomotor student by conditional flow matching. Figure 3.1 gives an overview.

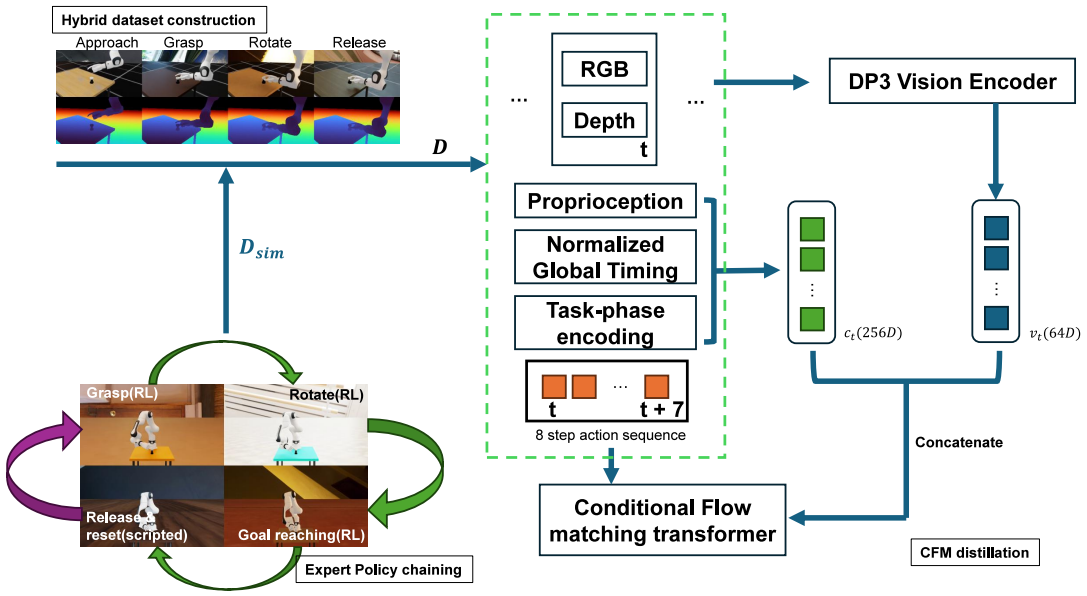


Figure 3.1: The pipeline in detail. Left: the privileged RL policy executes grasp, rotation, and goal reaching and a scripted controller executes release and reset. Chained in a loop, they form the hybrid teacher that generates the simulated demonstrations $\mathcal{D}_{\text{sim}}$. Right: the student encodes the aligned RGB-D observation into a 64-D point-cloud feature, fuses it with proprioception, the normalized global progress, and the process-phase encoding into a 256-D condition, and a conditional flow-matching Transformer predicts an eight-step action chunk.

3.1 Problem Formulation

Cyclic nut threading is decomposed into the ordered phase sequence

$$\mathcal{S} = \big(\underbrace{\text{approach, grasp, rotate, goal reaching}}_{\mathcal{M}_{\mathrm{RL}}:\,\pi_{\mathrm{T}}},\ \underbrace{\text{release, return and reset}}_{\mathcal{M}_{\mathrm{script}}:\,\kappa}\big). \tag{3.1}$$

The active phase at control step t is $m_t \in \mathcal{M} = \mathcal{M}_{\mathrm{RL}} \cup \mathcal{M}_{\mathrm{script}}$. The four phases in $\mathcal{M}_{\mathrm{RL}}$ are executed by a learned teacher policy π_{T}. The two transition phases in $\mathcal{M}_{\mathrm{script}}$ are executed by scripted controllers κ_m. The split is a property of the data-generation stage only: the deployed student receives neither the phase label as a switch nor separate networks per phase.

Let K be the required number of cycles, $\Delta\varphi^*$ the desired clockwise rotation per cycle, and φ_k the unwrapped nut angle about the bolt axis after the goal-reaching phase of cycle k, with φ_0 the angle at the start of the trial. The cyclic tracking error, the episodic tracking error, and the tracking cost of a trajectory τ are defined as

$$e_k = \varphi_k - \varphi_{k-1} - \Delta\varphi^*, \qquad e_{\mathrm{ep}} = \varphi_K - \varphi_0 - K\Delta\varphi^* = \sum_{k=1}^{K} e_k, \qquad E(\tau) = \frac{1}{K}\sum_{k=1}^{K} e_k^2 + \beta_E\, e_{\mathrm{ep}}^2. \tag{3.2}$$

Positive errors indicate overshoot and negative errors lag. The cost squares them so that overshoot in one cycle cannot cancel lag in another, and the weight β_E trades accumulated drift against cycle-wise accuracy. Let $C(\tau) \in \{0, \dots, K\}$ count the cycles completed before failure or timeout and let $\Pi_{\mathcal{S}}$ denote the policies that execute the phases of $\mathcal{S}$ in order in every cycle. The research problem is to find the policy that chains all phases with maximum completion success and, among those, minimum tracking cost:

$$\Pi_{\mathrm{succ}} = \arg\max_{\pi \in \Pi_{\mathcal{S}}} \Pr_{\tau \sim p_\pi}[C(\tau) = K], \qquad \pi^* = \arg\min_{\pi \in \Pi_{\mathrm{succ}}} \mathbb{E}_{\tau \sim p_\pi}[E(\tau) \mid C(\tau) = K]. \tag{3.3}$$

Unless a study states otherwise, $K = 10$ and $\Delta\varphi^* = 60°$, which corresponds to one face of a hexagonal nut per cycle. The policy π^* is learned from fixed hybrid-teacher demonstrations rather than by optimizing (3.3) on hardware, because on-robot optimization of a contact-rich cyclic task is neither safe nor sample-efficient.

The task is partially observable at deployment, so it is a partially observable Markov decision process (POMDP): the MDP of Section 2.1.2 is extended by an observation model $O(\boldsymbol{z}_t \mid \boldsymbol{x}_t)$, and the policy acts on observations $\boldsymbol{z}_t$ instead of states [26]. The simulator state $\boldsymbol{x}_t$ contains the nut pose, the contact state, and the controller parameters. The student observes only an aligned RGB-D image, its proprioception, and two low-dimensional conditioning signals introduced in Section 3.5.2. A single observation is then no longer sufficient to act optimally, because the information that disambiguates the state may lie in earlier observations. The teacher–student structure of the method separates the two difficulties of this POMDP: the teacher solves the control problem with privileged state, and the student learns to perceive and to reproduce the teacher's behavior from sensor data.

3.2 Simulated and Physical System

The method is trained in simulation on a model of the physical hand–arm system and deployed on that system. This section describes the platform, the thread model that makes learning in simulation tractable, and the action contract that teacher, student, and robot share.

3.2.1 Platform

The robot is a seven-DoF torque-controlled Franka Research 3 (FR3) arm with a five-fingered Inspire RH56DFX right hand mounted on its flange (Figure 3.2). The hand has twelve finger joints driven by six motors. The non-thumb fingers each have one actuated flexion joint with a mechanically coupled distal joint, and the thumb has an abduction joint and a coupled flexion chain. Only three hand degrees of freedom are commanded by the policy: thumb abduction, thumb flexion, and index flexion, which together form the pinch used for threading. The remaining fingers hold a curled posture. This restriction narrows the solution space of the dexterous hand to one grasp strategy, as difficulty (b) of Section 1.2 requires. The nut is threaded onto a bolt that stands on a bench in front of the robot. A fixed RGB-D camera provides the observation. Its extrinsic calibration relative to the robot base was measured on the physical cell and is reproduced in simulation.



(a) FR3 arm. **(b)** Inspire RH56DFX right hand.

Figure 3.2: The robot platform, rendered from the simulation models used in this work. In the assembled system the hand is mounted on the arm flange.

Both teacher and student act at 15 Hz, while the low-level controller closes its loop at the physics rate. Simulator and timing settings are listed in Table A.1 in the appendix.

3.2.2 Simplified Thread Simulation

The thread model is the first answer to difficulty (a) of Section 1.2: the skill is learned in simulation, so the simulated thread has to behave like a mating thread in each of the hundreds of parallel environments that PPO needs. The default Isaac Sim collision pipeline does not provide this, because of how it resolves contact.

PhysX, like other GPU-parallel engines, resolves contact through interpenetration (Figure 3.3). Once the gap between two bodies falls below the contact offset, the engine creates a contact. Within one time step the bodies can sink into each other by a penetration depth d, and the solver then pushes them apart along the contact normal with a separating velocity that grows with $d/\Delta t$ and is capped at the maximum depenetration velocity. Contact is therefore never strictly rigid. It tolerates a small overlap, and the repulsive response grows with that overlap.

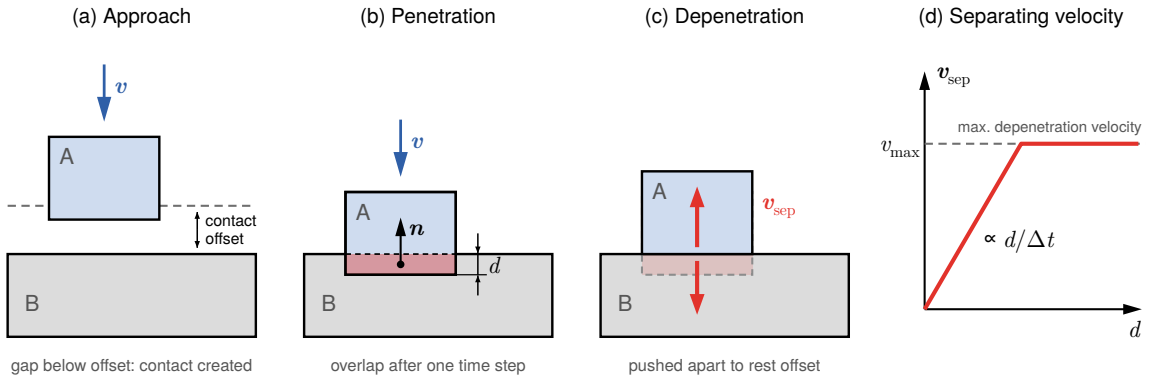


Figure 3.3: How a physics engine resolves contact, shown for PhysX. (a) Body A moves toward body B with velocity $\boldsymbol{v}$. Once their gap falls below the contact offset, the engine creates a contact. (b) Within one time step Δt, A sinks into B by the penetration depth d along the contact normal $\boldsymbol{n}$. (c) The solver pushes both bodies apart along $\boldsymbol{n}$ with a separating velocity $\boldsymbol{v}_{\text{sep}}$ until their separation reaches the rest offset. (d) The separating velocity grows in proportion to $d/\Delta t$ and is capped at the maximum depenetration velocity $v_{\max}$.

For large bodies this tolerance is harmless. For threads it is not, because the pitch of the nuts used in this thesis is only 3 to 4 mm and the flanks and crests are smaller still. The solver resolves an overlap along the normal of the nearest surface, and which surface is nearest depends on how deep the nut has sunk (Figure 3.4). As long as the nut thread overlaps only the loaded flank of the bolt, the normal points back out of the groove and depenetration restores the thread. Because the separating velocity is capped, a fast axial push can carry the nut thread past the bolt crest before the overlap is resolved. The nearest surface is then the opposite flank, and the solver pushes the nut into the next groove. The nut has advanced one pitch without turning.

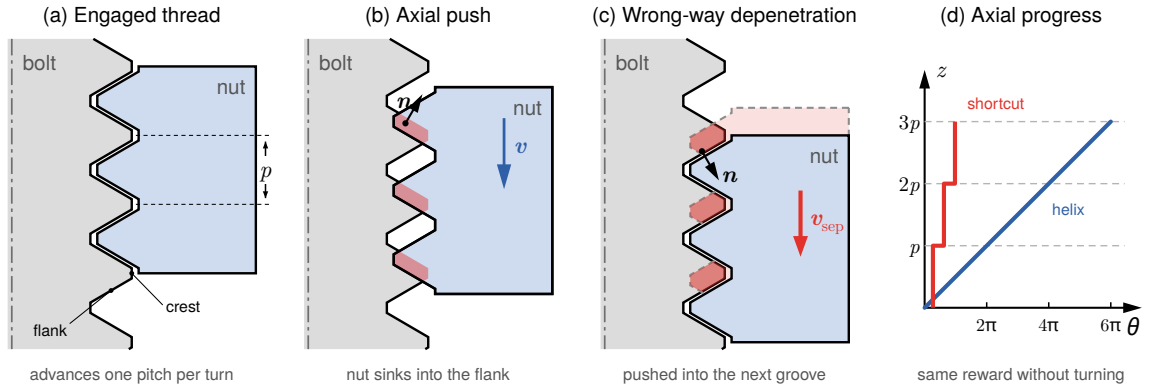


Figure 3.4: How interpenetration can be exploited in thread simulation, shown in an axial section of bolt and nut. (a) The nut thread sits in the bolt groove and advances by one pitch p per turn. (b) A push along the axis with velocity $\boldsymbol{v}$ sinks the nut flanks into the bolt flanks. The contact normal $\boldsymbol{n}$ of the loaded flank points back out of the groove. (c) Once the nut thread has been pushed past the bolt crest, the nearest separating direction is the opposite flank. The solver resolves the overlap with $\boldsymbol{v}_{\text{sep}}$ along the axis and pushes the nut into the next groove. (d) Axial progress z over nut rotation θ. On the helix the nut gains one pitch per full turn. The shortcut gains the same progress, and the same reward for axial progress, almost without turning.

With signed distance field (SDF) mesh colliders on nut and bolt, the simulated thread contact is therefore not accurate enough to reproduce a mating thread, and axial progress of the nut can be obtained by pressing it along the bolt axis. A policy rewarded for axial progress can learn this shortcut, which fails on hardware. A kinematic coupling instead

makes axial progress a consequence of rotation alone, so that progress can only be earned by turning. The thread model used for teacher training therefore replaces the thread contact by a kinematic coupling, illustrated in Figure 3.5. The nut is attached to the bolt by a passive revolute joint about the bolt axis and a coaxial, position-controlled prismatic joint. With prismatic coordinate d_t, unwrapped and direction-corrected revolute coordinate θ_t, thread pitch ρ, and the index b_k of the step at which the current turn started, the prismatic target is

$$d_t^* - d_{b_k} = -\frac{\rho}{2\pi}\left(\theta_t - \theta_{b_k}\right), \tag{3.4}$$

so that one revolution produces one pitch of axial travel. The prismatic joint tracks d_t^* with a stiff drive, whose finite stiffness permits small deviations from the ideal helix while the revolute–prismatic pair constrains lateral motion and keeps the nut near the commanded path. The finger–nut contact forces remain fully simulated: the policy must physically grip and turn the nut. Thread resistance is modeled on the revolute joint by a smoothed Coulomb and viscous friction torque

$$\bar{\tau}_{f,t} = -\tau_c \tanh\left(\frac{\omega_t}{\omega_s}\right) - c_v\,\omega_t, \tag{3.5}$$

with revolute velocity ω_t, Coulomb torque τ_c, viscous coefficient c_v, and smoothing speed ω_s. Table A.1 lists the drive, friction, and pitch values. The model has one deliberate simplification beyond contact: the nut is initialized already engaged with the bolt. Thread engagement from a loose nut is a separate skill with a narrow entry configuration, and it is outside the scope of this thesis. The simplified contact opens a gap between the simulated and the physical thread, which the domain randomization of the teacher (Section 3.3) is meant to cover.

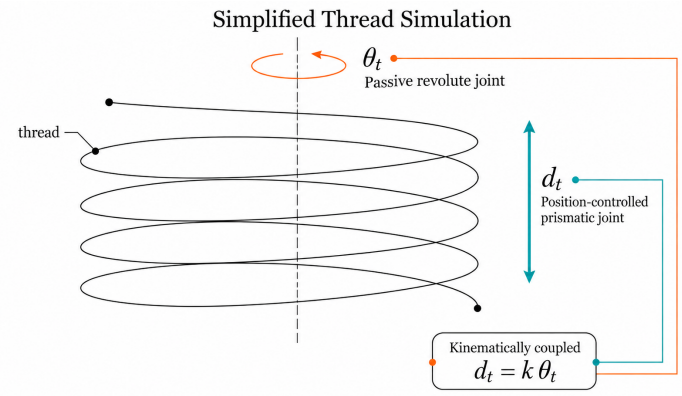


Figure 3.5: Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t through $d_t = k\theta_t$ with $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. The finger–nut contacts remain fully simulated.

3.2.3 Action Contract and Controller

The teacher acts in a 9-D action space. A fixed action decoder, described below, maps each action to a Cartesian target for the hand's grasp frame and three pinch-joint targets. The student and the scripted transitions act in the same space and use the same decoder (Section 3.4.1). The grasp frame has its origin at the midpoint of the thumb and index fingertips. Its x axis is the thumb-to-index direction, along which the pinch closes, and its z axis is the approach direction, fixed at reset in flange coordinates. The first three components set a position offset relative to a target frame placed on the tip of the bolt. Every action component

is normalized to the range from −1 to 1, and the position components are scaled by 50 mm, so the raw position action deviates by at most 50 mm per axis from this frame. Because the target is expressed relative to the bolt, the controller draws the grasped nut toward the bolt wherever nut and robot are, and a zero action does not hold the current pose. The next three components set roll, pitch, and yaw around a neutral grasp orientation, with up to ±57.3° in roll and pitch and a yaw range of $[-180°, 90°]$, and the last three command thumb abduction, thumb flexion, and index flexion. The raw action $\boldsymbol{a}_t$ is smoothed by an exponential moving average,

$$\tilde{\boldsymbol{a}}_t = \boldsymbol{\alpha} \odot \boldsymbol{a}_t + (1 - \boldsymbol{\alpha}) \odot \tilde{\boldsymbol{a}}_{t-1}, \tag{3.6}$$

with $\alpha = 0.0625$ for the arm components and 3α for the finger components, so that a single command moves the filtered action by only a small fraction of its deviation and consecutive targets stay smooth. The filtered action is decoded to a desired pose $\boldsymbol{x}^{\mathrm{des}}$, and the target passed to the controller is clipped to a bounded step from the current pose $\boldsymbol{x}$,

$$\boldsymbol{x}^{\mathrm{tgt}} = \boldsymbol{x} + \mathrm{clip}\left(\boldsymbol{x}^{\mathrm{des}} - \boldsymbol{x}, -\boldsymbol{\delta}, \boldsymbol{\delta}\right), \tag{3.7}$$

with $\delta = 0.02\,\mathrm{m}$ per axis and 0.097 rad per Euler angle, applied at every physics step. One command therefore cannot demand a large jump. Table A.2 in the appendix lists the scales and limits.

The Cartesian target is tracked by the operational-space impedance controller of (2.13). The stiffness and damping gains K_p and K_d are diagonal in world axes, with $K_d = 2\sqrt{K_p}$ for critical damping, and the posture term uses the dynamically consistent nullspace projector of Section 2.1.6. The same law runs in simulation and on the physical FR3. The two implementations differ only in where the model quantities J and M come from and in how gravity is handled, which Table A.2 in the appendix compares. The finger joints are position-controlled.

3.3 Privileged Teacher Training

The privileged teacher answers difficulty (b) of Section 1.2. It learns to approach the nut, acquire the pinch grasp, keep it under load while turning, and stop at the target, all through the commanded hand and arm degrees of freedom of Section 3.2.1. Because it observes privileged simulator state, it solves this control problem without having to perceive the nut (Section 3.1).

The teacher for the learned phases in $\mathcal{M}_{\mathrm{RL}}$ is a recurrent policy, which keeps a hidden state $\boldsymbol{h}_t$ that integrates the history of past observations. It is trained with PPO (Section 2.1.2) in an asymmetric actor–critic [29], in which the critic additionally receives privileged state that exists only in the simulator and cannot be measured on the hardware. This state includes the noise-free poses of the nut and bolt, the arm joint positions, and the randomized controller gains and smoothing factor of the episode (Table 3.1). The actor receives only quantities with a counterpart on the physical system, with sensor noise applied. The critic enters only through the value estimates that shape the policy gradient and is discarded after training, so the privileged state informs the update of the actor without ever being an input to it. The actor observation $\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$ selects the actor-visible components of the simulator state and applies observation noise $\boldsymbol{\xi}_t$. Its 46 dimensions form four groups. The pose group holds the fingertip (grasp-frame) position relative to the bolt frame and its quaternion, the end-effector linear and angular velocity, the fingertip position relative to the held nut, and the nut quaternion. The task group holds the position and quaternion error to the current turning target and a grasp-phase flag that indicates whether the proper grasp has latched. The force

group holds two fingertip forces and seven arm joint torques. The last group is the previous filtered action.

The force group mirrors the sensors of the physical system. Each fingertip channel is the force that the thumb or index pad exerts along its pad normal, computed from the total contact force on the pad regardless of the body in contact, because the physical fingertip sensors observe neither the other force components nor which body presses on them. Each torque channel is the reaction torque that the simulator measures at one arm joint, normalized by the effort limit of that joint, so that contact loads appear in it. The student observes the same two channels, with the noise realization of the teacher's actor (Section 3.5.2). Pose and force inputs are perturbed by noise that models sensor noise and calibration error (Table 3.2). The critic observes clean values of the same quantities and extends them with the arm joint positions, the complete nut and bolt pose, and the sampled controller parameters, for 76 dimensions. Table 3.1 lists the exact components.

Table 3.1: Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. Joint torques are divided by the effort limits of 87 N m (joints 1–4) and 12 N m (joints 5–7).

Input	Component	Dim	Noise (actor)
actor (46-D)	fingertip position relative to bolt frame	3	$\mathcal{N}(0, 0.5\,\text{mm})$ per axis; bolt frame $\mathcal{N}(0, 2\,\text{mm})$ per reset
	fingertip quaternion (WXYZ, w and z zeroed)	4	random-axis angle $\mathcal{N}(0, 0.2°)$
	end-effector linear velocity	3	finite difference of noisy position
	end-effector angular velocity (x and y zeroed)	3	finite difference of noisy quaternion
	fingertip position relative to held nut	3	$\mathcal{N}(0, 0.5\,\text{mm})$ per axis, each body
	held-nut quaternion (WXYZ)	4	random-axis angle $\mathcal{N}(0, 0.2°)$
	target position error	3	inherits nut noise
	target quaternion error (WXYZ, $w \geq 0$)	4	inherits nut noise
	grasp-phase flag	1	none
	thumb / index flexion force	2	gain $U(0.6, 1.4)$, offset $U(-2, 2)$ N per reset; $\mathcal{N}(0, 2\,\text{N})$; ≥ 0
	arm joint torques (normalized)	7	gain $U(0.8, 1.2)$, offset $U(-0.02, 0.02)$ per reset; $\mathcal{N}(0, 0.01)$; in $[-1, 1]$
	previous filtered action (roll, pitch masked)	9	none
critic (76-D)	fingertip position, quaternion, velocities	13	clean
	arm joint positions	7	clean
	held-nut position, position relative to bolt, quaternion	10	clean
	bolt position and quaternion	7	clean
	controller proportional gains	6	sampled per reset
	action smoothing factor	1	sampled per reset
	thumb / index flexion force	2	clean
	arm joint torques (normalized)	7	clean
	position and rotation action thresholds	6	sampled per reset
	target position, quaternion, progress	8	clean
	previous filtered action	9	none

The teacher is optimized with the shaped reward

$$\begin{aligned} r_t &= \sum_{i \in \mathcal{R}} w_i^r R_{i,t} - \sum_{j \in \mathcal{P}} w_j^p P_{j,t}, \\ \mathcal{R} &= \{\text{center}, \text{face}, \text{grasp}, \text{hold}, \text{turn}, \text{target}\}, \\ \mathcal{P} &= \{\text{magnitude}, \Delta a, \text{align}, \text{force}, \text{settle}, \text{wrench}, \text{clear}, \text{contact}, \text{limit}\}. \end{aligned} \tag{3.8}$$

The reward terms follow the three phases of the learned behavior, and Table A.5 in the appendix lists their exact form and weights. During approach, R^{center} rewards motion of the grasp center toward the nut center and R^{face} rewards motion of the thumb and index pads toward two designated opposing faces of the nut. Both pay only for newly achieved progress and stop paying once the grasp has latched, so repeated motion cannot accumulate reward. A proper grasp is simultaneous thumb and index contact near the designated faces with a pinch line nearly perpendicular to the nut axis. R^{grasp} pays a one-time bonus when the proper grasp has held for ten consecutive steps, which latches the grasp, and R^{hold} pays a small reward in every step in which the proper grasp holds under pad pressure. The hold reward of a complete episode is smaller than the target bonus, so holding without turning is worth less than threading. During turning, R^{turn} rewards newly achieved progress toward the helical

target pose and pays only while the proper grasp remains live. At completion, R^{target} pays a bonus once, on the step at which the episode succeeds.

The penalties discourage large commands ($P^{\text{magnitude}}$), abrupt action changes ($P^{\Delta a}$), misalignment of the hand frame with the nut frame (P^{align}), excess grip force (P^{force}), and residual rotation near the target (P^{settle}). P^{wrench} prices the wrench that the thread joint transmits to the nut above small dead bands, except the torque about the bolt axis that threading requires. Its weight rises with the initial-yaw curriculum described below, so that the policy learns to thread before the force limits apply. P^{clear} and P^{contact} keep the arm off the table, and P^{limit} keeps every arm joint away from its hardware limit. In addition, the effort of the three pinch joints is limited, which bounds the grip force that the policy can apply (Table A.1).

Each episode samples a target turn $\theta^* \sim U(55°, 65°)$ and builds the observed target pose at θ^*, so the policy cannot settle on one fixed stopping angle and has to read the target from its observation. An episode succeeds when the clockwise rotation lies within 2.5° of θ^* under a live proper grasp for a number of consecutive steps, so overshoot fails. Success ends the episode, so the target bonus is paid once and cannot be farmed by rocking the nut across the success angle. Once the initial-yaw curriculum is complete, the required hold grows from eight to twelve steps (0.53 s to 0.8 s). An episode also terminates when the nut leaves the bolt, tilts, or releases the thread, when the grasp is lost after it has latched, when an arm joint comes within 0.15 rad of its limit, and at the 30 s time limit. The grasp-loss rule turns grasp retention into a hard constraint rather than a shaped term.

Robustness is obtained by domain randomization of the nut mass, the contact friction, the thread resistance, the arm joint dynamics, the controller parameters of Section 3.2.3, the observations, and the initial state (Table 3.2). The pose noise is small in absolute terms, because the actor's inputs are relative poses that the student will later infer from a calibrated camera. A per-episode offset of the bolt frame models calibration error and forces the teacher to close the loop on the nut rather than on the bolt frame. The initial nut rotation is randomized by an automatic curriculum that widens the yaw range from zero to ±30° in 5° steps whenever the success rate reaches 80 %, and the initial axial height of the nut is randomized as well. The friction, thread, arm-dynamics, and start-pose randomization starts at nominal values and widens with the same curriculum, so that it reaches its full range at ±30°.

Table 3.2: Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling. Multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability. Joint torques are in units of the joint effort limits. Quantities marked † start at their nominal value and widen to the listed range with the initial-yaw curriculum.

Group	Quantity	Range
physics	nut mass offset	$U(-5, 5)$ g per reset
	bolt static friction	$U(0.25, 1.25)$, 128 buckets, once per environment
	nut friction / finger pad friction†	$U(0.6, 1.4)$ / $U(1.0, 2.0)$, once per environment
	thread resistance†	× log-uniform in $[0.25, 4]$ on the Coulomb and viscous terms
	arm joint friction / armature / damping†	$\times U(0.8, 2.0)$ / $\times U(0.7, 1.3)$ / $\times U(0.7, 1.3)$ of the nominal values
controller	proportional gains	multiplicative, $\ell = 0.41$, per axis and reset
	action smoothing factor	$U(0.025, 0.1)$ per reset; fingers 3×, capped at 1
	position / rotation threshold	multiplicative, $\ell = 0.25$ / 0.29
	dead zone	$U(0, [5, 5, 5, 1, 1, 1])$ N, N m, resampled every 2 s
observation	fingertip and nut position	$\mathcal{N}(0, 0.5\,\text{mm})$ per axis and step
	fingertip and nut orientation	random-axis angle $\mathcal{N}(0, 0.2°)$
	bolt frame	$\mathcal{N}(0, 2\,\text{mm})$ per axis and reset
	fingertip forces	gain $U(0.6, 1.4)$, offset $U(-2, 2)$ N per reset; $\mathcal{N}(0, 2\,\text{N})$ per step; ≥ 0
	arm joint torques	gain $U(0.8, 1.2)$, offset $U(-0.02, 0.02)$ per reset; $\mathcal{N}(0, 0.01)$ per step; in $[-1, 1]$
reward	grip-force penalty threshold	$U(5, 10)$ N per reset
initial state	nut yaw	curriculum $0 \rightarrow \pm 30°$ in 5° steps at 80 % success
	nut axial height	$U(0, 20)$ mm below nominal, room for 65° reserved
	bolt position†	$U(-5, 5)$ cm in x and y
	arm start pose†	$U(-3, 3)$ cm per axis, $U(-10, 10)°$ yaw of the grasp frame
task	target turn θ^*	$U(55°, 65°)$ per episode

The demonstrations of this thesis are collected with this single-size M24 teacher, which is

trained from scratch with PPO in 500 parallel environments for 200 epochs (Table A.7). The initial-yaw range reaches ±30° at epoch 125, and the success rate over the last ten epochs is 0.87. At the end of training the required success hold has grown to its final twelve steps. The run takes 3.6 h of wall-clock time. An earlier configuration of the same task, without the force group and without initial-yaw randomization, conditions one teacher on the M24, M30, and M36 nuts through a one-hot code and a round-robin assignment of sizes to environments.

3.4 Hybrid Teacher and Dataset Collection

The privileged teacher turns the nut once. The hybrid teacher answers the data side of difficulty (c) of Section 1.2: it chains the learned turn with scripted release and return, so that each phase is specified by the mechanism that specifies it most simply, and it rolls the chain out in simulation to produce complete multi-cycle demonstrations. This section describes the cyclic rollout, the disturbance augmentation, the vision data collection, and the dataset format.

3.4.1 Cyclic Teacher Rollout in Simulation

Let $m_t \in \mathcal{M}$ denote the active phase, $\boldsymbol{x}_t$ the full simulator state, $\boldsymbol{z}_t^{\pi}$ the noisy actor observation, and $\boldsymbol{h}_t$ the actor's hidden state. During rollout the teacher is queried for its deterministic mean action $\bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t) = \mathbb{E}_{\boldsymbol{a}\sim\pi_{\mathrm{T}}(\cdot|\boldsymbol{z}_t^{\pi},\boldsymbol{h}_t)}[\boldsymbol{a}]$. The hybrid teacher is

$$\pi_{\mathrm{HT}}(\boldsymbol{x}_t, m_t) = \begin{cases} \bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t), & m_t \in \mathcal{M}_{\mathrm{RL}}, \\ \kappa_{m_t}(\boldsymbol{x}_t), & m_t \in \mathcal{M}_{\mathrm{script}}, \end{cases} \tag{3.9}$$

where κ_{m_t} is the scripted controller of the active transition phase. Both branches produce commands in the same filtered, bolt-relative Cartesian action space, so the recorded label is a single 9-D signal throughout the cycle. The scripted Cartesian targets are mapped back through the action decoder of Section 3.2.3, and every label is the filtered target divided by a fixed per-component scale that bounds the scripted transitions to $[-1, 1]$ (Table A.8). This unified label space is what makes the demonstration a coherent supervision signal for one student: at no step does the label change meaning or units.

Release is triggered when the clockwise nut rotation since the start of the cycle comes within 5° of the cycle's target turn, which lies between 55° and 65°. Four Cartesian waypoints follow: the first opens the thumb and index while the arm holds its pose, and the next three retract the hand away from the nut and raise it. A final return segment brings the hand back to the initial grasp pose. The return is physically gated: it counts as complete only when the grasp frame and the hand joints are back within tolerance of their reset values, and a timeout marks the cycle as a transition failure. The position and orientation tolerance is drawn anew for every return, and a return that has settled inside the largest tolerance is accepted, so the demonstrations contain returns of different accuracy. Table A.6 in the appendix lists the thresholds and timings. While the hand is away from the nut the thread joint is held, so the nut cannot drift between cycles. On a completed return the turning reference is rebased without resetting physics. The next target turn is 60° minus the residual by which the total nut rotation misses the nearest multiple of 60°, clamped to the trained range of target turns, so that a short turn is made up in the next cycle and the nut stays on the 60° grid of the hexagon. The frozen threading policy receives the nut orientation with whole 60° steps removed, which the hexagonal nut cannot distinguish, so that its observation

stays within the initial-yaw range of training. The physical nut pose is not reset, so the demonstration contains the axial advance of the nut over ten cycles. The teacher population was first qualified for one period only. Repeated cycles are a property of the hybrid rollout, not of the RL teacher alone. Figure 3.6 shows the coordinator that sequences one cycle.

Two details of the coordinator matter for the quality of the labels. First, the scripted transitions never write physics state: a transition may request that the thread joint be held while the hand is away, but it repairs nothing by teleporting the nut, so every recorded frame is a physically consistent consequence of the recorded commands. Second, the release gate is physical rather than timed: release is decided from the measured nut rotation, and the return is validated against the reset frame, so the phase labels in the recording coincide with physical events. Both properties allow the recorded phase to serve as a conditioning signal at deployment. There, the return is validated against the reset grasp pose with the same tolerances, while the release, for which the physical cell has no nut-angle sensor, is derived from the measured yaw of the grasp frame as a proxy for the nut rotation.

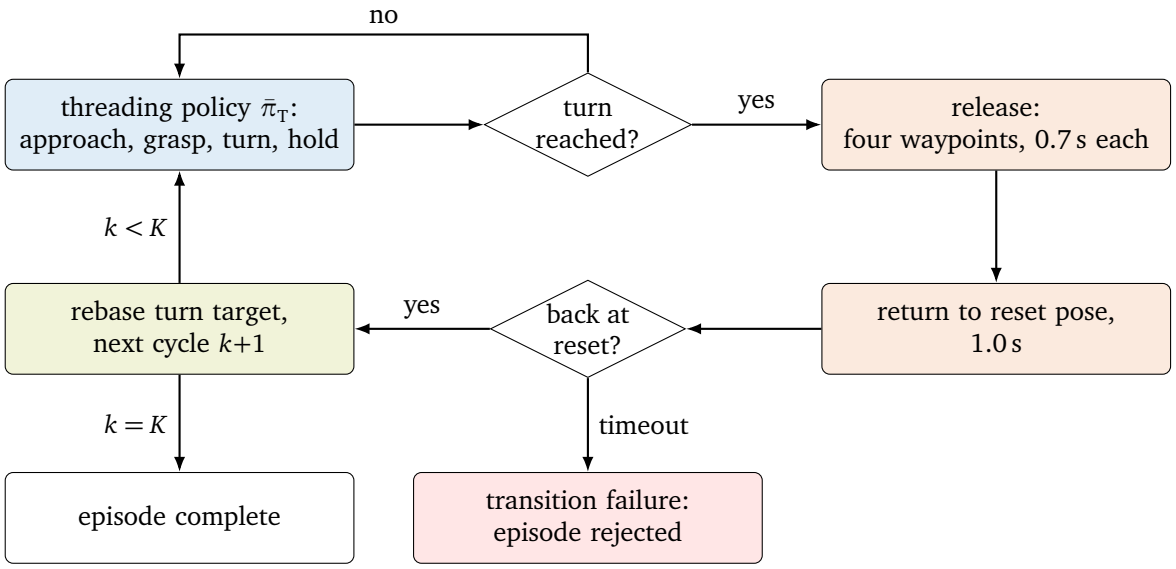


Figure 3.6: Data-collection loop of the hybrid teacher over K threading cycles. The learned policy (blue) owns approach, grasp, turn, and hold. A physical release gate on the nut rotation hands over to the scripted transitions (orange): four waypoints, which open the pinch and retreat, and the gated return. A completed return rebases the turning target and starts the next cycle, and a return that times out rejects the episode. Table A.6 lists the gate thresholds. Every policy, waypoint, and return segment receives one random arm impulse, and every executed command is recorded in the common 9-D label space.

3.4.2 Disturbance Augmentation

Twenty demonstrations contain too little variation for a student that must recover from its own deviations. Trajectory diversity is therefore increased at collection time by applying random impulses to the arm: one impulse is applied to a random arm link shortly after the start of every policy, waypoint, and return segment of every cycle, which gives 30 impulses per ten-cycle demonstration (Table A.9). The teacher, which is a closed-loop controller, reacts to the disturbance, and the recorded recovery is part of the label. The teacher is queried for its mean action and controls every step ($\beta = 1$), so each recovery segment is the teacher's own response to the state that precedes it. Besides the impulses, the demonstrations vary through the task randomization that remains active during collection (Section 3.4.4). Demonstrations

in which the teacher fails to complete all cycles are rejected before materialization. The retained set consists only of explicitly successful episodes.

3.4.3 Vision Data Collection

The student perceives the scene through one RGB-D camera, so the demonstrations are rendered from a simulated copy of the physical camera. The extrinsic pose of the physical camera relative to the robot base is deduced from twelve different poses of a calibrated AprilTag observed by the camera. The resulting pose and the intrinsics are used to render the simulated camera during data collection and enter the student training pipeline. The physical stream is cropped and downscaled to the 320×180 resolution that the simulated camera renders directly. Camera geometry and field of view therefore agree between the two domains up to calibration error. The collection covers this error: at every reset, the pose of the simulated camera is shifted by a uniform offset of up to ± 11.2 mm per axis and rotated by up to $\pm 1.13°$ per axis, sized from the residual of the hand–eye fit (Table A.9). Appearance and depth quality do not, because a rendered scene has one clean appearance and a rendered depth image contains no sensor noise. Texture randomization covers the first difference at collection time, and depth augmentation covers the second during training.

Texture randomization follows DextrAH-RGB [23] and varies the appearance of the rendered scene. At every reset, each environment draws new materials for the tabletop, the nut, and the bolt, each with a random texture, roughness, and metallicity. The dome light, with its texture, orientation, and intensity, and the floor texture are shared by all parallel environments and are redrawn at a reset with a fixed probability. An environment can also replace the far and invalid pixels of the RGB image with a random background image. These pixels lie outside the crop box of the point-cloud encoder (Section 3.5.2), so the background reaches only the image-space encoders of Chapter 4. The robot keeps its nominal appearance. Because the randomization is drawn at reset, the appearance is constant within a ten-cycle demonstration and differs between demonstrations. The offline loader cannot regenerate scene appearance from stored images, so this variation is recorded into the dataset once and reused unchanged by every student trained on it. Table A.9 lists the sampled ranges, and Figure 3.7 shows the resulting variation of the rendered scene.

Depth augmentation corrupts the stored metric depth images during student training. Rendered depth is exact, whereas a physical RGB-D camera returns noisy depth with missing regions, and the depth determines the position of every point that the encoder receives. The corruption is drawn anew for every training batch and acts before back-projection, so each epoch presents a different corruption of the same demonstration. It first perturbs the valid pixels with Gaussian noise whose standard deviation grows with the squared depth, adds a spatially smooth offset field, and quantizes the result. It then invalidates pixels at depth discontinuities, spatially correlated regions, and isolated pixels. Invalid pixels are rejected during back-projection, so an augmented frame yields a noisier point cloud with missing regions. The parameters in Table A.8 approximate a commodity RGB-D camera qualitatively, and Figure 3.8 shows their effect on a depth image next to a depth frame from the physical camera. The dataset of the next subsection stores the randomized RGB images together with the noise-free depth, so the same demonstrations can be trained with or without the augmentation.



Figure 3.7: Three draws of the texture randomization in the simulated workcell. The dome light and the textures of the floor and the tabletop change between draws, while the robot keeps its nominal appearance.

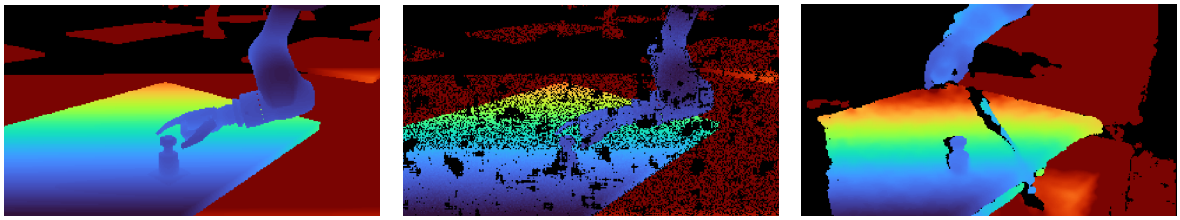


Figure 3.8: Simulated depth with and without augmentation, next to real depth. Left: the noise-free depth of one frame of a simulated demonstration, rendered by the 320×180 head camera. Middle: the same depth image after the full augmentation used in student training, with depth noise, quantization, and dropout of edge pixels, correlated holes, and isolated pixels. The augmentation raises the fraction of invalid pixels in this frame from 21.4 % to 47.3 %. Right: a depth frame streamed from the head camera of the physical robot, shown for comparison. All three images use the same color map, from about 0.8 m (dark blue) to 2 m (dark red), and black marks invalid pixels.

3.4.4 Dataset Format

The demonstrations are collected from the M24 teacher of Section 3.3 with two training mechanisms switched off: success does not end the episode, and the target turn is not sampled but follows the 60° grid of Section 3.4.1. Every environment starts from the nominal pose, without initial yaw and with the nut at the top of the thread, so that ten 60° turns, 1.67 revolutions or 5 mm of axial travel on the 3 mm pitch, fit on the bolt. Only part of the teacher's randomization stays active during collection: the noise of the proprioceptive observations, the friction of the arm joints and of the finger pads, the pose of the head camera, and the visual appearance of the scene. The other quantities, namely the nut mass, the nut and thread friction, the controller parameters, and the bolt and arm start poses, are fixed at their nominal values. The release and return of the scripted transitions run with the settings of Table A.9. A cycle that fails is not discarded together with its episode: it is retried from the checkpoint of the last completed return, at most three times, and the rows of the abandoned attempt are removed from the episode. An episode may last up to 200 s.

Each demonstration is materialized as one episode file with aligned 15 Hz samples. A sample holds the RGB and metric depth images at 320×180 pixels, the 35-D proprioception, the executed 9-D command, the active process phase, the nut pose for privileged ablations, and the timestamp, from which the normalized task progress is derived. Action chunks never span a reset, phase boundaries are retained so that the student learns the transitions, and terminal chunks are padded with the final command so that the last observations of an episode remain conditioning states.

3.5 Student Policy Distillation by Conditional Flow Matching

The student answers the deployment side of difficulty (c) of Section 1.2. The hybrid teacher cannot be deployed, because both its learned policy and its scripted controllers act on privileged simulator state (3.9). The student replaces it with one visuomotor policy that executes every phase of every cycle from sensor data, without a phase switch or separate networks. RQ1 asks whether this unified student sustains consecutive cycles better than a single-cycle policy invoked repeatedly. RQ2 asks which student design distills the demonstrations best, and Chapter 4 compares the design below with its ablations.

3.5.1 Conditional Flow Matching

The action head of the student is a conditional flow-matching model [22], [36], whose principle Section 2.1.5 introduced. This section states what the head is trained to generate and which signals it is conditioned on.

The head generates action chunks. Its training target is the demonstrated chunk of the next H commands, $\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}] \in \mathbb{R}^{H\times 9}$. The chunk is conditioned on the student observation $\boldsymbol{y}_t = (\boldsymbol{O}_t^{\mathrm{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$. The RGB-D image $\boldsymbol{O}_t^{\mathrm{RGBD}}$ of the head camera shows the hand, the nut, and the bolt. The proprioception $\boldsymbol{s}_t$ holds the joint positions, the arm joint velocities, the previous action, the fingertip forces, and the arm joint torques. The normalized task progress $p_t \in [0,1]$ tells the student how far into the trial it is, and the process-phase encoding $\boldsymbol{e}_t^m$ marks whether the policy, the release waypoints, or the return is active. The network of Section 3.5.2 encodes these signals into one condition vector $\boldsymbol{c}_t$, which takes the place of $\boldsymbol{y}$ in (2.9). Training applies the straight-line construction of (2.8) to whole chunks:

$$\theta^* = \arg\min_\theta \mathbb{E}\left\|\boldsymbol{v}_\theta\left(\boldsymbol{U}_t^\lambda, \lambda \mid \boldsymbol{c}_t\right) - \dot{\boldsymbol{U}}_t^\lambda\right\|_2^2, \qquad \boldsymbol{U}_t^\lambda = (1-\lambda)\boldsymbol{U}_t^0 + \lambda \boldsymbol{U}_t^D, \quad \dot{\boldsymbol{U}}_t^\lambda = \boldsymbol{U}_t^D - \boldsymbol{U}_t^0, \quad (3.10)$$

where the expectation is over $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, a Gaussian chunk $\boldsymbol{U}_t^0 \sim \mathcal{N}(\boldsymbol{0}, I)$ of the same shape, and $\lambda \sim U[0,1]$. At run time, the student draws a Gaussian chunk $\boldsymbol{U}^0$ and follows the learned field with N explicit Euler steps, $\boldsymbol{U}^{\lambda+1/N} = \boldsymbol{U}^\lambda + \frac{1}{N}\boldsymbol{v}_\theta(\boldsymbol{U}^\lambda, \lambda \mid \boldsymbol{c}_t)$, with $N = 16$ in simulation (Table A.8 lists the deployment setting). The condition $\boldsymbol{c}_t$ is computed once per chunk, so the image is encoded once and each Euler step evaluates only the Transformer of Section 3.5.2.

A generative head is used because the demonstrations are multimodal. The teacher's recoveries after impulses depend on state that the observation does not fully resolve, so near-identical observations can carry the nominal action in one demonstration and a recovery in another. A regression head averages such labels, whereas the flow head samples one of them (Section 2.1.5). Among generative heads, flow matching is chosen over diffusion because it trains on a plain regression target without a noise schedule and samples in a few deterministic integration steps.

3.5.2 Student Architecture

Figure 3.9 shows the network architecture of the student, which encodes the observation $\boldsymbol{y}_t$ into the condition $\boldsymbol{c}_t$ and predicts the flow velocity $\boldsymbol{v}_\theta$ of the action chunk from it. It consists of a vision encoder, a proprioception encoder, a fusion and conditioning stage, and the velocity network.

The student encodes the visual and proprioceptive streams separately and fuses them by concatenation, following DextrAH-RGB [23], with the visual feature placed before the

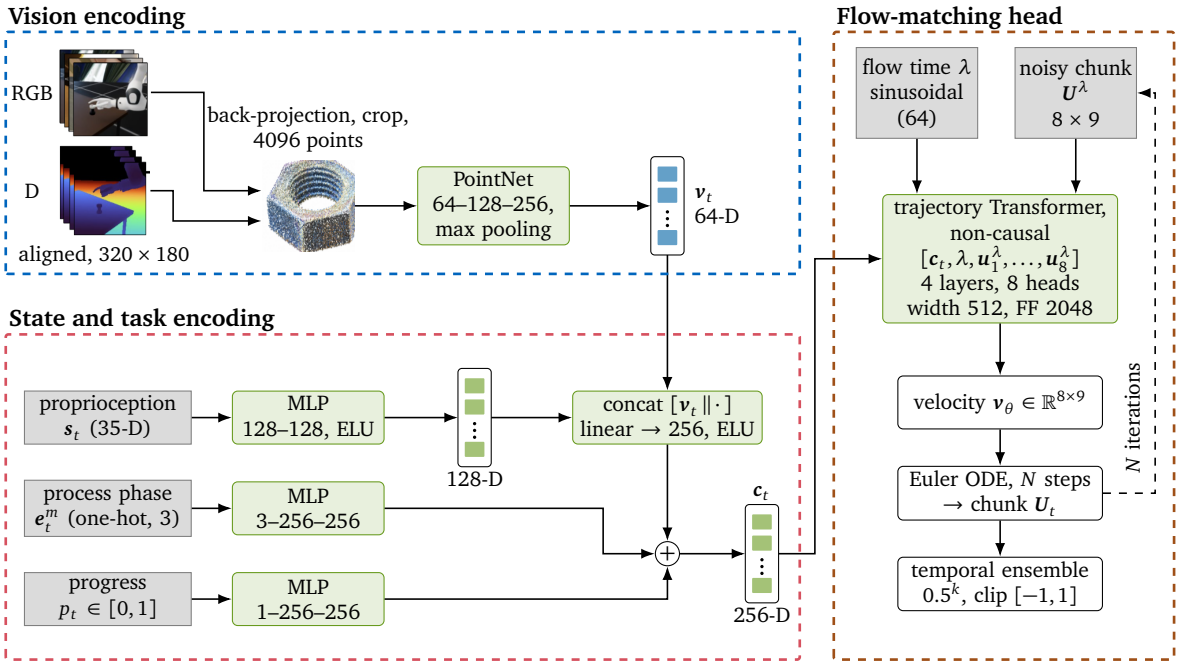


Figure 3.9: Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature. The 35-D proprioception is encoded by an MLP. Late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time.

encoded proprioception. The 35-D proprioception s_t consists of the ten controlled joint positions, the seven arm joint velocities, the previous filtered 9-D action, the two fingertip forces, and the seven arm joint torques, and is encoded by an MLP. The hand joint velocities are omitted. The fingertip forces are divided by 20 N and the joint torques by the joint effort limits. The concatenation of the 64-D visual feature and the proprioceptive feature is projected to the 256-D condition.

The vision encoder of the student is the point-cloud encoder of DP3 [25]. The depth image is back-projected through the calibrated intrinsics into camera-frame points. Points outside a fixed crop box around the workspace or outside the valid depth range are rejected, and the remaining points are normalized by fixed workspace constants and carry the color of their pixel. The encoder takes a fixed set of 4096 points per frame, while the number of valid points varies from frame to frame. Each pixel therefore receives a ranking score, and the 4096 highest-ranked pixels form the input set. Valid pixels always rank above rejected ones, so rejected pixels enter the set only when a frame has fewer than 4096 valid points. These fill-in points stay marked as invalid and are excluded from the pooling below. Among the valid pixels, the score decides which subset is kept. During training it is drawn uniformly at random, so every update sees a different subset of the scene. At evaluation each pixel carries a fixed score from a golden-ratio sequence over the pixel index, which makes the selection deterministic and spreads it across the image without the cost of farthest-point sampling. A compact PointNet with pointwise layers and global max pooling over the valid points yields the 64-D visual feature v_t. The encoder has no pretrained backbone. It is initialized randomly and trained jointly with the flow head, so that its representation is shaped by the action objective alone.

Two low-dimensional conditions, the global trajectory progress and the process phase, are added to the 256-D condition through separate learned embeddings. The global trajectory progress $p_t \in [0,1]$ maps the first timestamp of each demonstration to 0 and the last to 1.

At deployment it advances with elapsed time from the start of the trial, normalized by the median demonstration duration stored in the checkpoint, and is clamped to 1 afterwards. It is a monotonic clock for the complete behavior and nothing more: it is not a teacher-stage selector, it is not periodic, and it does not encode the cycle index. It tells the student how far into a nominally timed trial it is, which disambiguates, for instance, the first and the last approach of a trial that otherwise look alike. The process-phase encoding $\boldsymbol{e}_t^m$ is a three-way one-hot over the policy-controlled phases, the release waypoints, and the return. During training it is derived from the recorded phase labels, and during evaluation it is published by the cyclic coordinator, in simulation from the physical events of Section 3.4.1 and on the robot from the release proxy and return gate, so that a cycle that takes longer than nominal cannot select the wrong phase. The phase encoding is a synchronization aid: it does not switch networks or action contracts, and the student predicts the transition actions itself.

The velocity field $\boldsymbol{v}_\theta$ is parameterized by a non-causal trajectory Transformer. The noisy chunk $\boldsymbol{U}^\lambda$ is split into $H = 8$ action tokens of dimension 9. A condition token embeds $\boldsymbol{c}_t$, and a time token embeds the flow time λ through a sinusoidal encoding. Learned positional embeddings mark the token slots and carry no absolute time. The Transformer applies full self-attention over all tokens, so that every step may attend to the whole noisy chunk when predicting its velocity, and a linear output layer maps each action token to its 9-D velocity. The model keeps an exponential moving average of its parameters, and inference uses the averaged weights. Table A.8 in the appendix lists the network sizes and the training hyperparameters.

The student predicts actions in chunks of $H = 8$ steps, which cover 0.53 s at the 15 Hz control rate. The first action of a chunk is the command that acts on the observed state. At deployment the student replans at every control step, and the command sent to the controller is a temporal ensemble of all still-valid proposals for the current tick with weights 0.5^k for a proposal of age k. A decay of zero recovers strict first-action execution. The ensemble operates in action space and is distinct from the parameter-space EMA. The executed 9-D action is clipped to $[-1, 1]$, decoded to the filtered Cartesian target, and passed to the controller of Section 3.2.3. Because the label scale is fixed, no action statistics are learned.

3.5.3 Offline Training

Algorithm 1 summarizes offline distillation. The dataset consists of the simulated hybrid-teacher rollouts, windows of H consecutive commands are built from each episode without crossing episode boundaries, and the velocity field is trained with (3.10).

Algorithm 1 Offline flow distillation

Require: hybrid teacher π_{HT}, horizon H

1: Collect simulated rollouts $\mathcal{D}_{\mathrm{sim}}$ with π_{HT} under random arm impulses and visual domain randomization
2: Form H-step chunks from $\mathcal{D} = \mathcal{D}_{\mathrm{sim}}$, masking resets and padding terminal chunks
3: **repeat**
4: Sample $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, $\boldsymbol{U}_t^0 \sim \mathcal{N}(\mathbf{0}, I)$, $\lambda \sim U[0,1]$
5: Encode the point cloud and fuse it with proprioception, progress, and phase to obtain $\boldsymbol{c}_t$
6: Update θ by one gradient step on (3.10)
7: Update EMA parameters $\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta$
8: **until** convergence
9: **return** student velocity field $\boldsymbol{v}_{\bar{\theta}}$

With every component of the method defined, Chapter 4 evaluates it in simulation to answer RQ1 and RQ2, and on the physical system to answer RQ3.

Chapter 4

Experiments

This chapter evaluates the method of Chapter 3 in simulation, where every component can be varied under matched conditions, and on the physical system, where the student is deployed zero-shot. It answers the three research questions of Section 1.4. Section 4.1 defines the metrics. Section 4.2 describes the experimental setup, including the physical system and the zero-shot transfer protocol. Section 4.3 reports the teacher and the demonstrations it generates. Section 4.4 defines the ablations, and Section 4.5 reports their results in simulation and on hardware. Section 4.6 evaluates the student on unseen nut sizes and geometries, Section 4.7 under long-horizon stress, and Section 4.8 summarizes the chapter.

4.1 Evaluation Metrics

4.1.1 Evaluation Ladder

A failed closed-loop rollout can originate in the supervised fit, in the command conversion and controller, or in the distribution shift between teacher and student states. The three evaluation modes in Figure 4.1 separate these causes, and each mode makes a stronger claim than the previous one.

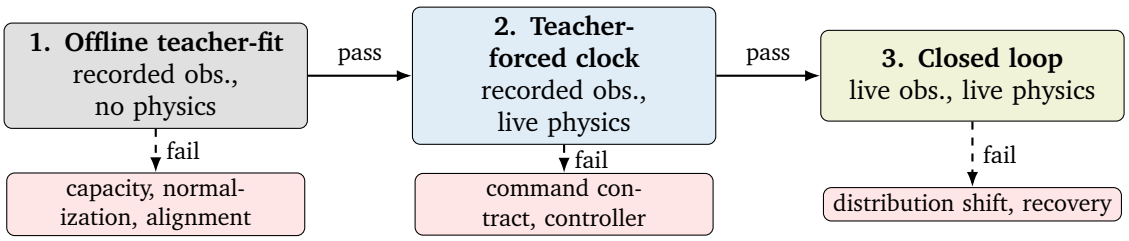


Figure 4.1: Evaluation ladder. Each mode closes one more part of the control loop. A failure in a mode points to the causes listed below it.

In the offline teacher-fit mode, every recorded teacher observation of a demonstration is fed to the student, and the sampled chunk is compared with the recorded command chunk. The reported quantities are the RMSE of the first action and of the full chunk and the per-joint errors over time. Passing shows that the checkpoint can represent the observation-to-action mapping under the chosen alignment. It establishes neither controller correctness nor closed-loop success. In the teacher-forced clock mode, the student's predicted commands are executed in live physics, but the student receives the recorded teacher observations selected on the recorded playback timeline, and for progress-conditioned checkpoints the same timeline

supplies p_t. The realized joints, tool center point, and nut pose are compared with the recording. Passing shows that the learned action map, the command conversion, the controller, and the physics reproduce the nominal behavior once closed-loop observation drift is removed. Only the normal closed-loop mode, in which the student consumes live observations and its errors feed back into later observations, measures task success and compounding error. It is the mode of all results that claim success. The two diagnostic modes are used in that order to localize a failure before expensive retraining: a bad offline fit points to capacity, normalization, or alignment. A good fit with bad teacher-forced execution points to the command contract. Good teacher-forced execution with a bad closed loop points to distribution shift and recovery, which is the regime that motivates online labels or disturbance augmentation.

4.1.2 Metrics

Each condition is evaluated over $N_{\mathrm{tr}} = 10$ trials with up to ten cycles per trial, where each cycle consists of a nominal 60° clockwise turn followed by release and return. A cycle counts as completed when the release has been triggered and the return has been validated against the reset pose within its 16 s timeout (Section 3.4.1). A trial ends at the first transition failure, early termination, or time limit. Three quantities are reported. The mean completed cycle count $\bar{C} = N_{\mathrm{tr}}^{-1} \sum_i C_i$ counts, for trial i, the cycles $C_i \in \{0, \ldots, 10\}$ completed before failure or timeout. Incomplete cycles are excluded, and higher is better. The cyclic tracking error is computed for every completed cycle k of trial i as $e_{i,k} = \Delta\varphi_{i,k} - 60°$, where $\Delta\varphi_{i,k} = \varphi_{i,k} - \varphi_{i,k-1}$ is the clockwise nut rotation. Over all $M = \sum_i C_i$ completed cycles the mean $\mu_{\mathrm{cyc}} = M^{-1} \sum_{i,k} e_{i,k}$ and standard deviation σ_{cyc} are reported, where negative values indicate lag and positive values overshoot, and a mean closer to zero with a smaller deviation is better. If $M = 0$ both statistics are undefined. The ten-cycle tracking error is computed over the set $I_{10} = \{i : C_i = 10\}$ of trials that complete all cycles as $e_{10,i} = \sum_{k=1}^{10} \Delta\varphi_{i,k} - 600°$, reported as mean μ_{10} and standard deviation over I_{10}. All errors are in degrees. In simulation the nut angle is read from the simulator. On hardware it is read from an angular scale mounted under the bolt (Section 4.2).

4.2 Experimental Setup

Teacher and student policies are each trained on a single NVIDIA GeForce RTX 4090 GPU. At inference time, the student runs with CUDA on an NVIDIA GeForce RTX 3060 Mobile GPU. One inference pass, from the point-cloud construction through the sixteen integration steps of the flow head, takes 11.7 ms on average and at most 12.0 ms, less than a fifth of the 67 ms control period at 15 Hz.

The physical workcell, shown in Figure 4.2, is the platform of Section 3.2.1: the FR3 arm with the Inspire RH56DFX right hand and the three commanded pinch degrees of freedom described there. The bolt stands on a 5 cm pedestal on a 60 cm by 80 cm bench, and the robot base is mounted at tabletop height, 46.5 cm from the near table edge, so that the reach envelope lies above the work surface. The finger kinematics of the simulated hand were identified on the physical hand from camera images, as described in Appendix A.3.

The simulated system is a model of this workcell. Appendix A.2 describes the arm and hand assets and how the hand model was fitted to the physical hand.

A fixed Intel RealSense D415 on a stand provides the RGB-D observation at 640 × 480. As in simulation, the stream is center-cropped and downscaled to the 320 × 180 policy input (Figure 4.2). The pose of the camera relative to the robot base was deduced from twelve



(a) Physical workcell.

(b) Live RGB (left) and depth (right) observation.

Figure 4.2: Hardware setup. Left: the FR3 arm with the Inspire RH56DFX hand, the bolts on their holders in front of the robot, and the fixed RealSense D415 on its stand. Right: the RGB image and the metric depth image of the D415 as the student receives them, after center-cropping and downscaling to 320×180.

different poses of a calibrated AprilTag observed by the camera, with a translation fit RMSE of 9.7 mm and a rotation fit RMSE of 0.98° over the calibration poses. The calibrated optical pose and the intrinsics are the ones used to render the simulated camera and to back-project the point cloud, so the student sees the same geometry in both domains.

Robot commands use the same Cartesian impedance law as in simulation, so the student's bounded action reaches the arm through the shared action path of Section 3.2.3. The deployment controller is a direct implementation of the dynamically consistent operational-space controller (2.13) [44], the same controller that is used in training. It uses the rotation-vector orientation error, the dynamically consistent nullspace projector with the arm mass matrix from the robot model and the hand configured as end-effector load, and no Coriolis term, because the simulated arm is not compensated either. The student's commands therefore meet the stiffness under which the teacher acquired its contact behavior. Gravity is compensated by libfranka, which matches the simulation, where gravity is disabled for the arm. The controller runs at the 1 kHz rate of the Franka control interface, while the student commands targets at 15 Hz. The hand is position-controlled through its serial interface and receives its pinch targets once per policy step, also at 15 Hz. The task-space gains equal those of the simulation, 565 N/m and 28 N m/rad with critical damping, and the nullspace stiffness is 10 with critical damping. The targets pass through the same action smoothing as in training and no further low-pass filter, and the controller applies the step limit of Table A.2 against the measured grasp pose in every 1 kHz cycle. On top of the impedance law, the deployment controller adds a Coulomb friction feedforward of 0.3 N m per joint (Section B.2).

The metrics of Section 4.1.2 require the nut angle. It is read from an angular scale printed on a sheet under the bolt pedestal (Figure 4.4), and the unwrapped angle is recorded at the end of every goal-reaching phase. The test-object family consists of M24, M30, and M36 nuts in four profiles each, hexagonal, octagonal, rectangular, and round, with matching bolts and holders (Figure 4.3). Teacher and student training use the hexagonal profiles only. The other three profiles are held out for the generalization study of Section 4.6.

4.2.1 Zero-Shot Transfer Protocol

RQ3 (sim-to-real) asks whether a student trained on simulated demonstrations only transfers zero-shot to the physical system. The student of the full method is trained for 100 epochs on the 36 simulated demonstrations $\mathcal{D} = \mathcal{D}_{\text{sim}}$ of Section 3.4.4 with eight-step chunks. It is

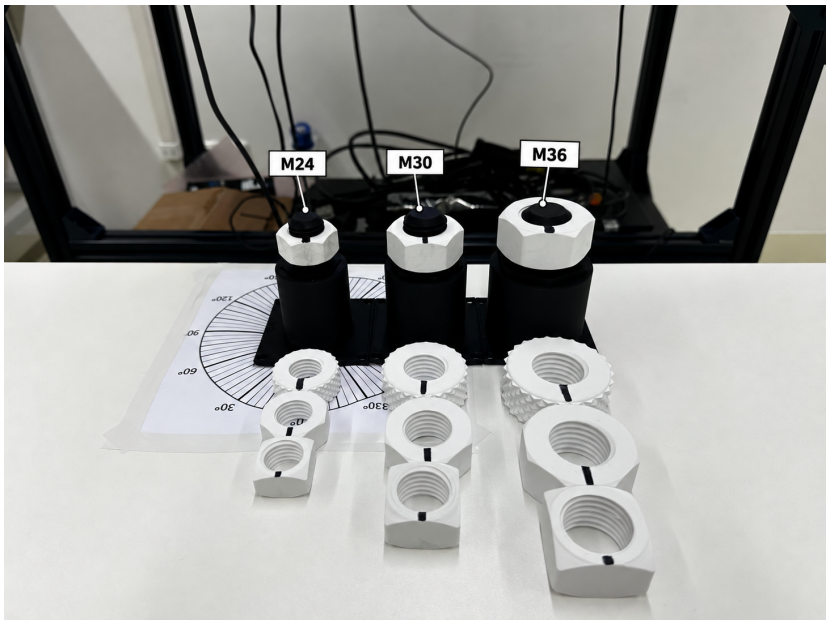


Figure 4.3: The test-object family. The hexagonal nuts of the three sizes M24, M30, and M36 sit on their bolts and holders. In front of each bolt lie the round, octagonal, and rectangular nuts of the same size. The angular scale of Figure 4.4 is printed on the sheet under the holders.

deployed on the physical robot with a prediction horizon of eight and sixteen ODE integration steps. No real-world images, trajectories, or corrections enter its training, and it is not fine-tuned on the robot. The transfer therefore rests entirely on four properties of the pipeline. The calibrated camera described above is the one that rendered the demonstrations. The point-cloud crop box removes the physical background before encoding. The demonstrations were rendered under lighting, texture, and background randomization. The Cartesian targets are executed by the impedance controller described above. The student receives the progress scalar from the elapsed trial time and the process phase from the physical events of the cycle, as in simulation (Section 3.5.2). The progress scalar is normalized by the median demonstration duration of 98.2 s, and the phase is published by the release proxy and the return gate of the cyclic coordinator. The deployed network uses the EMA weights after the final epoch. The D415 depth is aligned to the color image and used raw: the spatial, temporal, and hole-filling filters of the driver are off, and depths outside the 0.01–5 m clipping range of the simulated camera are marked invalid, as in rendering.

Figure 4.5 shows what the point-cloud encoder receives on the physical robot. The upper row overlays the 4096 sampled points on the camera image, and the lower row shows the same points in a side view of the robot base frame. The table forms a flat band at a height of about 0.01 m, and the bolt and its holder rise from it at the position of the bolt in the training scene. The hand appears as a separate cluster that descends toward the nut position. Its recorded grasp point reaches the red box of the training nut and stays close to it in the following seconds. The geometry that enters the encoder therefore agrees with the training scene in position and scale, even though the points come from a physical depth sensor.

Before the student is deployed, the shared action path is characterized on the physical arm, and Appendix B reports these measurements. The same impedance law runs in training and on the FR3, but the two arms behind it differ: the simulated arm has no gearbox friction, whereas the physical joints show 0.2–1 N m of Coulomb friction (Section B.10) and a loss that grows with speed (Section B.6). Commanded to a target, the physical arm returns to it within 1.7–3.3 mm from the same side but rests 6–9 mm from it on average (Section B.9). The

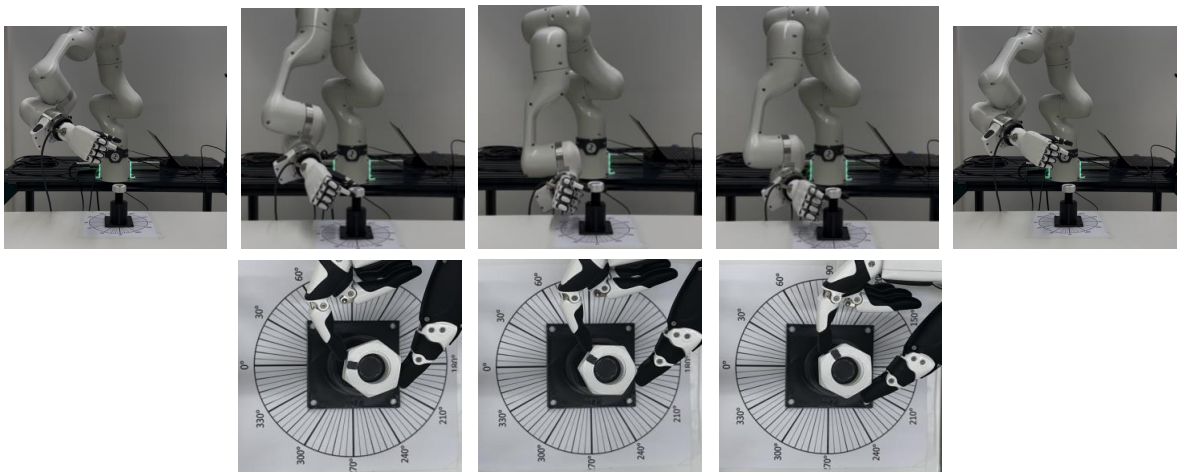


Figure 4.4: One threading cycle on the physical robot. Top, from left to right: approach, grasp, rotation, goal reaching, and release with return to the reset pose. Bottom: top-down views of the angular scale under the bolt during grasp, rotation, and goal reaching. The scale is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors.

transition clip of Section B.8 widens the target step during the scripted release and return. In five hardware runs with it, every return that passed the reset gate did so within 2.0–2.7 s, and the three of these runs that re-homed between cycles completed all 12 of their cycles. Every hardware rollout is first rehearsed in ros-sim, a MuJoCo model of the workcell that runs the same ROS 2 stack and controller as the physical robot.

4.3 Teacher and Demonstrations

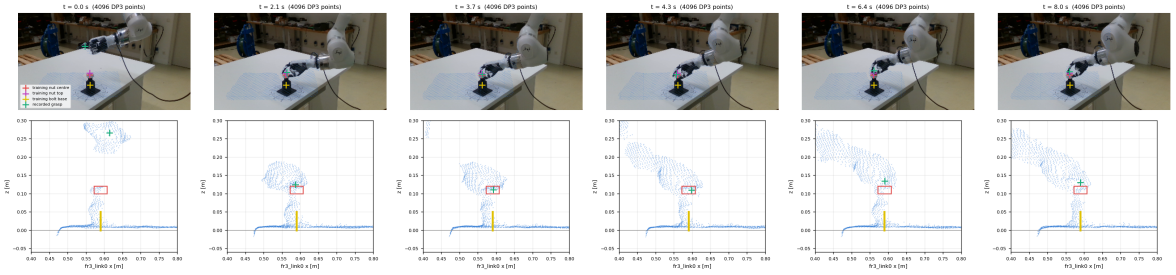
The teacher of Section 3.3 is the starting point of the pipeline, so its result is reported before the students. Table 4.1 summarizes the last ten of its 200 training epochs. The values are training statistics of the stochastic policy under the full randomization of Table 3.2, not a separate evaluation.

Table 4.1: Training statistics of the M24 teacher over its last ten epochs, with the full randomization of Table 3.2 active.

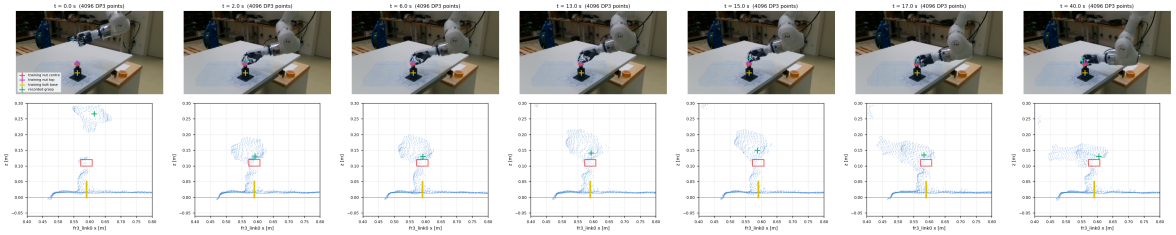
Quantity	Value
threading success rate	0.87
epoch at which the initial-yaw range reaches ±30°	125
proper grasp / grasp latched once	0.72 / 0.65
thumb / index pad force [N]	10.8 / 7.8
pad-force slip fraction	0.063
grasp-loss / joint-guard terminations	0.001 / 0.000
completed episode length [steps]	112

The teacher reaches a success rate of 0.87 under the full initial-yaw range, which the curriculum reached at epoch 125 of 200. The pads press the nut with about 11 N at the thumb and 8 N at the index finger. The loss of a latched grasp and the joint-limit guard end episodes at rates of 0.001 and 0.000. A completed episode lasts 112 steps, or 7.5 s at 15 Hz.

Chained into the hybrid teacher, this policy produced the demonstration dataset of Section 3.4.4. One rollout of 16 parallel environments yielded 36 ten-cycle demonstrations with



(a) Six time steps within the first 8 s of a trial.



(b) Seven time steps over 40 s of a second trial.

Figure 4.5: Point cloud of the physical D415 as the encoder receives it. Upper rows: the 4096 sampled points (blue) over the RGB image, with crosses for the nut centre, the nut top, and the bolt base of the training scene and for the grasp point recorded during the trial. Lower rows: the same points in the x–z side view of the robot base frame, with the training nut as a red box and the bolt as a yellow bar.

explicit teacher success, 14 of the 16 environments contributing at least one. Twenty of the 36 demonstrations contain at least one cycle that was retried from the last completed return, 40 retries in total. The demonstrations hold between 1341 and 1838 frames each, and the median demonstration lasts 98 s.

4.4 Ablations

To investigate the contribution of individual components, the ablation study focuses on three aspects. The first is whether a unified multi-cycle student outperforms the repeated execution of a single-cycle policy, which is RQ1 (long horizon) of Section 1.4. The second and third belong to RQ2 (student design): the contribution of the point-cloud encoder, and the effectiveness of flow matching relative to deterministic regression with an LSTM and a Transformer backbone. Five students are trained on the simulated dataset of Section 3.4.4 and compared under matched training and evaluation budgets, and each ablation changes only the component it names. Ablation (a) replaces the training data and addresses RQ1. Ablation (b) replaces the point-cloud encoder with an image-space encoder, and ablations (c) and (d) replace the flow-matching head with deterministic regression on a recurrent and on a Transformer backbone. These three address RQ2. Ablation (e) is the full method. All five students are evaluated in simulation, and (a), (b), and (e) are also deployed zero-shot on hardware to answer RQ3 (sim-to-real).

(a) The single-cycle student is trained on demonstrations of one cycle and is invoked ten times in a trial. Its temporal context is reset after every cycle.

(b) The ResNet student replaces the point-cloud encoder with a one-branch image-space encoder. An ImageNet-pretrained ResNet-18 stage-2 trunk processes RGB, log depth, and a validity mask. An embedding of the calibrated camera-frame XYZ is added on the

token grid, and attention pooling over the tokens yields the same 64-D feature. Policy, objective, data, and budget are unchanged.

(c) The LSTM student keeps the point-cloud encoder but replaces the flow-matching Transformer with the DextrAH-RGB-style recurrent backbone, a layer-normalized 512-unit LSTM followed by a 512–512–256 ELU MLP. It is trained by deterministic action regression on the same data.

(d) The Transformer regression student keeps the point-cloud encoder and uses a deterministic variant of ACT [32] without the variational objective. It predicts the eight-step chunk from eight learned action queries, and its backbone has the same size as the flow Transformer.

(e) The full method is one flow-matching student with the point-cloud encoder, trained on the complete multi-cycle simulated demonstrations.

4.5 Results

Table 4.2 reports the results on the M24 hexagonal nut in simulation and on hardware. In simulation, the full method (e) completes all ten cycles in every trial, with a cyclic tracking error of $-1.77 \pm 1.02°$ and a ten-cycle error of $-17.7 \pm 3.23°$. The student thus lags the 60° target by less than two degrees per cycle, and the lag accumulates over the trial without its spread drifting. Each ablation removes one property on which this result may depend. The single-cycle student (a) never sees the axial advance of the nut: in a single-cycle demonstration the nut is always grasped at its reset height, whereas over ten cycles it advances down the bolt by one sixth of a pitch per cycle, and the multi-cycle student has to read this height from the point cloud (RQ1, long horizon). The ResNet student (b) keeps everything but the representation. The regression students (c) and (d) keep the data and, for (d), also the backbone, the chunk length, and the temporal ensembling, so that their difference to (e) isolates the generative objective (RQ2, student design). In simulation, the single-cycle student (a) completes 5.5 ± 0.8 cycles with a cyclic error of $-4.16 \pm 3.21°$, and no trial reaches ten cycles. The ResNet student (b) completes 2.3 ± 0.9 cycles and overshoots by $+20.17 \pm 7.80°$ per cycle. The LSTM student (c) diverges before it completes the first cycle. The Transformer regression student (d) completes 1.4 ± 0.8 cycles with an overshoot of $+124.21 \pm 18.50°$, so it fails from the same compounding error after a short delay. Every ablation therefore falls short of the ten cycles of the full method, and the two regression heads fail first.

For the encoder part of RQ2, a representation probe complements the closed-loop ablation (b). One shared pose decoder is trained on the frozen 64-D feature of each encoder to predict the nut position and orientation. A re-grasp needs exactly the orientation of the nut faces relative to the fingers, so the probe tests whether a representation carries the quantity on which the closed loop depends. In simulation, the point-cloud feature yields a position error of 8 mm and an orientation error of 15.16°. The ResNet feature reaches a similar position error of 8.6 mm, but its orientation error of 31.20° is about twice as large. The image-space feature thus preserves the nut position but loses much of the face orientation on which the re-grasp depends, which is consistent with the early failure of (b) in closed loop.

The real-world part of Table 4.2 answers RQ3 on the training nut, and it checks whether the comparisons of RQ1 and RQ2 carry over to hardware. Three of the five students are taken to hardware: the full method (e), the single-cycle student (a), and the ResNet student (b). The single-cycle student tests whether the advantage of multi-cycle distillation persists on the physical system, and the ResNet student tests whether the image-space representation

Table 4.2: Simulation and real-world zero-shot ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target. It is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot. Ablations (c) and (d) are evaluated in simulation only.

Ablation	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
Simulation			
(a) single-cycle student	5.5 ± 0.8	-4.16 ± 3.21	–
(b) ResNet encoder	2.3 ± 0.9	$+20.17 \pm 7.80$	–
(c) LSTM regression	0.0 ± 0.0	–	–
(d) Transformer regression	1.4 ± 0.8	$+124.21 \pm 18.50$	–
(e) full method	10.0 ± 0.0	-1.77 ± 1.02	-17.7 ± 3.23
Real world			
(a) single-cycle student	5.2 ± 1.2	-4.56 ± 5.55	–
(b) ResNet encoder	1.1 ± 0.9	$+24.09 \pm 9.94$	–
(e) full method	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37

transfers as well as the point cloud. The two regression heads (c) and (d) are evaluated in simulation only. On hardware, the full method (e) completes all ten cycles in every trial with a cyclic error of $-4.21 \pm 1.20°$ and a ten-cycle error of $-42.10 \pm 4.37°$. Its lag per cycle is larger than in simulation, while its spread stays similar. The single-cycle student (a) completes 5.2 ± 1.2 cycles. In its typical failure, the fingers slip off the edge of the nut after a few cycles, which shows that the student has learned the turning skill but does not follow the axial advance of the nut. The ResNet student (b) completes only 1.1 ± 0.9 cycles with an overshoot of $+24.09 \pm 9.94°$. Both comparisons from simulation therefore carry over to hardware, and the gap between the image-space encoder and the point cloud widens.

4.6 Generalization across Nut Sizes and Geometries

The training nut alone does not show how far the zero-shot transfer reaches, so the full method is also evaluated on all twelve test objects (Figure 4.3). Table 4.3 reports ten trials per object: the M24, M30, and M36 hexagonal nuts, whose profile the demonstrations show, and the octagonal, rectangular, and round profiles of each size, which it has never seen. The study tests two kinds of generalization. Across sizes, the student must infer the nut size from the point cloud, since it receives no explicit size input. Across profiles, the question is whether the learned pinch-and-re-grasp strategy, which the demonstrations show only on hexagons, transfers to a different face topology.

The student completes all ten cycles in every trial on the three hexagonal nuts. The M30 nut shows the smallest lag, $-3.70 \pm 0.77°$ per cycle, and the M36 nut the largest, $-5.52 \pm 1.34°$. Among the unseen profiles, the octagonal and round nuts are also completed in every trial at all three sizes, and the round nut has the smallest lag at every size. The rectangular profile fails. The M24 rectangular nut completes 3.0 ± 1.2 cycles, and the M30 and M36 rectangular nuts complete none. In the typical failure, the fingers turn the nut by 60° but do not establish contact with the next pair of faces in the following cycle, so the nut stays at its first 60° position. Transfer thus holds when an unseen profile preserves re-grasp configurations compatible with the hexagon, and it breaks when the face topology conflicts with the learned re-grasp.

Table 4.3: Real-world zero-shot transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation. The octagonal, rectangular, and round profiles and the M30 and M36 sizes are absent from the demonstrations.

Size	Profile	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
M24	hexagonal	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37
	octagonal	10.0 ± 0.0	-4.10 ± 1.16	-41.00 ± 4.20
	rectangular	3.0 ± 1.2	$+30.30 \pm 4.50$	–
	round	10.0 ± 0.0	-2.33 ± 0.91	-23.30 ± 3.58
M30	hexagonal	10.0 ± 0.0	-3.70 ± 0.77	-37.00 ± 2.37
	octagonal	10.0 ± 0.0	-3.66 ± 0.58	-36.60 ± 1.74
	rectangular	0.0 ± 0.0	–	–
	round	10.0 ± 0.0	-2.45 ± 0.49	-24.50 ± 1.43
M36	hexagonal	10.0 ± 0.0	-5.52 ± 1.34	-55.20 ± 5.44
	octagonal	10.0 ± 0.0	-5.48 ± 1.22	-54.80 ± 4.75
	rectangular	0.0 ± 0.0	–	–
	round	10.0 ± 0.0	-4.78 ± 1.02	-47.80 ± 2.36

4.7 Long-Horizon Stress

Unlike the protocol of Section 4.1.2, the stress trials have no ten-cycle limit. The student is run on hardware for as many cycles as it sustains, to measure how far its horizon extends beyond the ten cycles of the demonstrations. Every revolution advances the nut by one pitch down the bolt. After enough cycles, the nut therefore leaves the axial range that the ten-cycle demonstrations cover. The stress trials thus measure how far the learned behavior reaches along the bolt. The student sustains between 15 and 22 cycles before it fails, depending on nut size, profile, and test conditions. It therefore extends the demonstrated horizon by at least half, but not indefinitely.

4.8 Summary

In simulation, the full method trained on the demonstrations of the hybrid teacher completes all ten cycles of the M24 task in every trial with a cyclic error of $-1.77 \pm 1.02°$. Every ablation fails earlier. The single-cycle student does not follow the axial advance of the nut (RQ1), and the ResNet encoder and both regression heads fall short of the full student design (RQ2). On hardware, the student transfers zero-shot and completes all ten cycles on the training nut with a cyclic error of $-4.21 \pm 1.20°$ (RQ3). It also completes all ten cycles on the M30 and M36 hexagonal nuts and on the unseen octagonal and round profiles. It fails on the rectangular profile, and in the stress trials it sustains 15 to 22 cycles. Chapter 5 interprets the simulation and hardware results together.

Chapter 5

Discussion

This chapter interprets the results of Chapter 4. Section 5.1 answers the three research questions of Section 1.4 as far as the available results allow. Section 5.2 draws the lesson of the action and observation contracts. Section 5.3 returns to the assembly motivation and asks how far the demonstrated skill is from industrial threading. Section 5.4 states the limitations of the work.

5.1 Interpretation per Research Question

All three research questions presuppose a teacher that produces complete multi-cycle demonstrations, so the teacher result comes first. PPO learns the threading turn once the thread is constrained kinematically and the learned task is one turn from an engaged nut. The kinematic constraint makes the learning problem well posed: it ties axial progress to rotation, so the progress reward can only be earned by turning. The teacher also answers difficulty (b) of Section 1.2.2: with the hand restricted to three commanded pinch degrees of freedom, it acquires the pinch grasp and keeps it under the turning load, and losing the grasp after it has latched ends the episode, which makes grasp retention a hard constraint (Section 3.3). Trained under initial-yaw, axial-height, and controller randomization, the retained teacher, chained into the hybrid teacher, produces the successful ten-cycle demonstrations of the student dataset (Section 3.4.4). The RL teacher solves one turn, and the hybrid rollout extended it to ten consecutive cycles. The actor of the teacher and the student observe the two fingertip forces and the seven arm joint torques, which have counterparts on the physical system (Section 3.3). No experiment of this thesis isolates their contribution.

RQ1 asks whether one student distilled from multi-cycle demonstrations sustains consecutive cycles, and it tests the deployment side of difficulty (c) of Section 1.2.3. The full method completes all ten cycles in every simulated trial and lags the target by less than two degrees per cycle. The content of the data is the likely reason. In a single-cycle demonstration the nut is always grasped at its reset height, so a student trained on it never has to read the height from the point cloud. In a multi-cycle demonstration the nut descends by one sixth of a pitch per cycle, and the student learns to track it. The global progress scalar supports the multi-cycle student, but it does not supply the axial information. That information comes from the coverage of the axial state in the demonstrations. The single-cycle ablation (a) receives the same progress scalar and therefore isolates this effect. It completes 5.5 ± 0.8 cycles in simulation and 5.2 ± 1.2 cycles on hardware, and none of its trials reaches ten cycles (Table 4.2). On hardware, its fingers typically slip off the edge of the nut after a few cycles, which is the failure expected from a student that turns the nut but does not follow its axial advance. The result thus supports the principle that unifying the complete cyclic behavior in one dataset,

rather than invoking a per-cycle policy repeatedly, yields a policy that sustains consecutive cycles.

RQ2 asks which student design makes the unification work. It has two parts, the visual representation and the action head, and each of the ablations (b) to (d) changes one of them. On the representation side, the student does not need to localize the nut to sub-millimeter accuracy. The engagement of the nut on the bolt and the compliance of the impedance controller absorb lateral error. What the student needs is the orientation of the nut faces for the re-grasp and the height of the nut for the approach. A representation that encodes position but not orientation would let the fingers close on the right spot with the wrong face alignment. The point-cloud encoder consumes the metric geometry of nut, bolt, and hand directly. It has about 60 000 parameters and no pretrained image backbone. The pose probe of Section 4.5 tests whether the feature of each encoder carries the face orientation. The image-space encoder of ablation (b) also receives depth and per-pixel coordinates, so (b) tests whether the geometric structure of the representation, rather than the depth input alone, decides the closed-loop result. Random point sampling adds an augmentation that the image encoder does not receive. The closed-loop ablation (b) and the pose probe quantify the difference. In closed loop, the ResNet student completes 2.3 ± 0.9 cycles in simulation and 1.1 ± 0.9 cycles on hardware, and it overshoots the target by $+20.17°$ and $+24.09°$ per cycle (Table 4.2). The probe is consistent with an orientation deficit. The two features locate the nut with similar position errors of 8 mm and 8.6 mm, but the orientation error of the ResNet feature, 31.20°, is about twice that of the point-cloud feature, 15.16°. The image-space representation thus keeps the nut position and loses much of the face orientation that the re-grasp needs.

On the head side, the chunked Transformer regression head (d) shares the backbone, the chunk length, and the temporal ensembling of the flow student, so the difference between (d) and (e) isolates the generative objective. Two mechanisms could favor the flow head. The first is multimodality of the labels. For near-identical observations, the dataset contains the teacher's nominal action in one demonstration and its recovery after an impulse in another. A squared-error head converges to their average, an action that neither demonstration contains, whereas the flow head samples one of them. The second is the structure of the prediction. The flow head predicts a velocity from a noisy chunk toward the data, a denoising problem that may be easier to fit than a direct regression from observation to chunk. Through the sixteen integration steps it refines all eight steps of the chunk jointly, with full attention among them. The recurrent regression head (c) additionally removes chunking. Both regression heads fail first. The LSTM student (c) diverges before it completes the first cycle, and the Transformer regression student (d) completes 1.4 ± 0.8 cycles with an overshoot of $+124.21 \pm 18.50°$ per cycle (Table 4.2). Because (d) differs from (e) only in the objective, the generative objective, and not the backbone or the chunking, separates the full method from this failure. The experiments do not show which of the two mechanisms dominates.

RQ3 asks whether a student trained on simulated data only transfers zero-shot to the physical system, and it tests the transfer side of difficulty (a) of Section 1.2.1. The transfer rests on four properties of the pipeline. The calibrated camera model is meant to make the simulated and real point clouds geometrically comparable. The crop box removes the background, which is where rendered and real images differ most. Lighting, texture, and background randomization, together with image-space color augmentation, are meant to cover the remaining appearance variation of the points that enter the encoder. The shared Cartesian action path executes the student's commands through a controller that reproduces the simulated control law, and the teacher was trained under randomized gains and noisy pose inputs. The same geometric property that RQ2 relies on matters here: inside the crop box the point geometry is identical in both domains up to depth noise and calibration error, whereas

an image-space encoder faces the appearance gap of the full image. The per-point colors still carry the appearance gap of nut, bolt, and hand. The approach requires no real-world data collection and no training on the robot. On hardware, the full method completes all ten cycles in every trial on the training nut, with a cyclic error of $-4.21 \pm 1.20°$ against $-1.77 \pm 1.02°$ in simulation (Table 4.2). The transfer thus preserves task success and the spread of the error, and it increases the lag by about 2.4° per cycle. The student also completes all ten cycles in every trial on the M30 and M36 hexagonal nuts, whose sizes the demonstrations do not contain, with cyclic errors of $-3.70 \pm 0.77°$ and $-5.52 \pm 1.34°$ (Table 4.3). The hardware ablations agree with the geometric argument: the gap between the ResNet student and the point-cloud student widens from simulation to hardware.

5.2 Action and Observation Contracts

The student imitates the teacher's bounded, bolt-relative Cartesian action. This keeps it in the teacher's own action space, expresses every target relative to the bolt tip, and executes the command through the unchanged controller, including the action smoothing that the teacher was trained with. A student should therefore imitate the action the teacher produced, in the units the teacher produced it, and let the controller do the conversion it was designed for.

The observation contract follows the same logic. The student receives the previous filtered action, which the teacher also observes, the positions and velocities of the ten controlled joints, which the teacher actor does not observe, and two low-dimensional conditions that the demonstration provides without privileged state. The global progress scalar assumes a nominally timed trial. For trials that run much longer than the demonstrations, the phase should be inferred from the observation history inside the policy rather than supplied as an external signal. Chapter 6 returns to this as future work.

5.3 What This Means for Robotic Assembly

The motivation of this thesis was assembly with threaded joints (Section 1.1). Measured against an industrial fastening step, the demonstrated skill covers the run-down of an engaged nut and leaves four gaps.

The first gap is the start of the task. Every trial starts with the nut already engaged on the bolt (Section 3.2.2). In assembly the nut arrives loose and has to be seated on the thread first, which is a separate skill with a narrow entry configuration. Engagement is therefore not covered by the demonstrated skill.

The second gap is the end of the task. A trial ends after ten turns of 60°, which advance the nut by one and two thirds of a pitch, 5.0 mm on the M24 nut. The nut is never seated against a part, and it is never tightened to a preload. The simulated thread resists the turn only with a smoothed friction torque whose Coulomb term is 2×10^{-3} N m, which models a free-running nut, and the reward penalizes excess wrist force. A tightening phase would demand that the pinch transmit a much larger torque and that the policy sense when the joint is tight, which the thread model does not represent.

The third gap is the grasp. The student executes one strategy, a thumb–index pinch with three commanded hand joints, learned on the face topology of a hexagon. People change the grasp as the resistance grows (Section 1.2.2), and a tightening phase would likely require such a change. The held-out octagonal, rectangular, and round profiles of Section 4.6 test how far the single strategy carries. It carries to the octagonal and round profiles, on which the

student completes all ten cycles in every trial at all three sizes, and the round nut shows the smallest lag at every size. It does not carry to the rectangular profile. The student completes 3.0 ± 1.2 cycles on the M24 rectangular nut and none on the M30 and M36 rectangular nuts (Table 4.3). In the typical failure, the fingers turn the nut by 60° but do not establish contact with the next pair of faces in the following cycle.

The fourth gap is cycle time. In the demonstrations, release and return take a nearly fixed time in every cycle, since their scripted waypoints are interpolated over 0.7 s per segment and 1.0 s for the final return. Because each cycle turns the nut by 60°, a full revolution takes six cycles, which is far slower than the continuous rotation of a powered spindle.

These gaps do not erase what the result shows. In simulation, a learned policy controlled a dexterous hand through ten cycles of grasp, turn, release, and return, and it re-grasped the nut at the angle and height that the previous cycle left it. It perceives the nut from one fixed camera, and it is trained without any real data. The same hand also picks up a loose nut in simulation. Whether this versatility outweighs the speed of a dedicated tool depends on the application, and the remaining gaps define the work that separates the two (Chapter 6).

5.4 Limitations

Section 5.3 described the task-level gaps to industrial threading. This section states the limitations of the method and of the evidence. The demonstrated behavior is a pinch-grasp re-grasp strategy, and the student has no mechanism to discover a different grasp for a profile that requires one. The demonstrations start at the nominal nut height and cover only the 5.0 mm of axial travel of ten cycles, and the global progress clock is defined over the nominal trial length. The teacher's thread model is a kinematic simplification. In simulation the nut therefore cannot jam or cross-thread, and the thread resists only through a small friction torque on the revolute joint. The randomization ranges that are meant to cover the resulting sim-to-real gap have not yet been fitted to measured statistics of the physical sensor. The camera calibration has a translation fit RMSE of 9.7 mm and a rotation fit RMSE of 0.98° (Section 4.2). It is measured but not deployment-validated, and the dependence of the student on calibration error has not been quantified. The evaluation uses ten trials per condition and reports means and standard deviations without confidence intervals. Chapter 6 summarizes what the thesis achieved and turns these gaps and limitations into future work.

Chapter 6

Conclusion and Future Work

This chapter summarizes what the thesis achieved, answers the research questions, and outlines the work that remains between the demonstrated skill and robotic assembly with threaded joints.

6.1 Conclusion

Threaded joints can be released again and their threads are standardized [1], and fastening a nut is still largely left to people and specialized tools. This thesis asked how a dexterous robot can learn to thread a nut. Threading poses three difficulties at once. The skill has to be learned in simulation and transferred to hardware, although thread contact is hard to simulate. A dexterous hand has a large action space and many ways to solve the task. Threading is a long-horizon cyclic task in which errors compound across cycles and phase transitions.

The thesis answers these difficulties with one principle: decompose during data generation, unify at deployment. During data generation the cycle is decomposed into a turning skill and scripted transitions. A privileged asymmetric actor–critic learns the pinch-grasp turn in a simplified thread simulation in which axial progress follows from rotation. Scripted release and return chain the turns into a hybrid teacher that produces complete multi-cycle demonstrations with physically gated phase boundaries. At deployment the behavior is unified again. One visuomotor student, distilled from the demonstrations by conditional flow matching, executes the whole cycle from an RGB-D camera and proprioception. It was trained on simulated data only, and a calibrated camera model, a metric point cloud cropped to the workspace, visual domain randomization, and a shared Cartesian action path are designed to carry it to hardware.

The evidence is consistent with the principle. In simulation, the distilled student completes all ten cycles in every trial on the M24 hexagonal nut, with a mean cyclic tracking error of $-1.77°$ and a ten-cycle error of $-17.7°$. A single-cycle student invoked ten times, the comparison that tests the principle (RQ1), completes 5.5 ± 0.8 cycles and never reaches ten. On hardware, the student transfers zero-shot and again completes all ten cycles in every trial, with a mean cyclic tracking error of $-4.21°$.

The three research questions are answered as follows.

RQ1 One student distilled from complete multi-cycle demonstrations sustains all ten cycles in simulation and on hardware. A single-cycle policy invoked repeatedly completes 5.5 ± 0.8 cycles in simulation and 5.2 ± 1.2 cycles on hardware, because it does not follow the axial advance of the nut.

RQ2 The flow-matching student with a point-cloud encoder is the only tested design that sustains the cyclic task. An image-space encoder completes 2.3 ± 0.9 cycles in simulation and 1.1 ± 0.9 on hardware, and its feature carries about twice the orientation error of the point-cloud feature. Deterministic LSTM regression diverges before the first cycle, and deterministic Transformer regression completes 1.4 ± 0.8 cycles.

RQ3 The student is trained on simulated data only and deployed without real data or training on the robot. It completes all ten cycles in every trial on the M24, M30, and M36 hexagonal nuts, with mean cyclic tracking errors between $-3.70°$ and $-5.52°$. It transfers to the held-out octagonal and round profiles at all three sizes and fails on the rectangular profile. In stress trials without a cycle limit, it sustains 15 to 22 cycles.

Returning to the motivation of Section 1.1: in simulation, a dexterous hand can now thread a nut over ten consecutive cycles with a learned policy, re-grasping it in every cycle from camera data at the angle and height that the previous cycle left it, and the policy is trained without real data. It does so zero-shot on the physical system as well, on hexagonal nuts of three sizes and on unseen octagonal and round profiles (RQ3). What it cannot yet do is start from a loose nut, tighten the joint, choose a different grasp when the resistance grows, or match the speed of a dedicated tool (Section 5.3). These gaps define the future work.

6.2 Future Work

The first extension is autonomous engagement of a loose nut. It requires grasping the loose nut, transporting it to the bolt, and chaining these steps with threading. Seating the nut on the bolt is a separate skill with a narrow entry configuration. It will most likely require fingertip-force and tactile feedback, and the hardware-equivalent fingertip-force and joint-torque observation channels that are implemented for teacher and student are the starting point.

The second extension is tightening to a preload. A threading cycle that ends with the nut tight needs a thread model whose resistance rises when the nut seats, a grasp that transmits the resulting torque, and a policy that senses when the joint is tight. The force channels that the teacher and the student already observe are the natural inputs for such a policy.

The third extension is a longer horizon. The demonstrations start at the nominal nut height and cover only the 5.0 mm of axial travel of ten cycles. A curriculum that widens this range toward the full usable thread length would extend the horizon of the teacher and of the student with it. The global progress clock should be replaced at the same time, because it binds the student to a nominal trial duration. Inferring the phase from the observation history inside the policy would remove that dependency.

The fourth extension concerns the transfer. The randomization ranges of the simulated camera should be fitted to measured statistics of the physical D415, including depth failures, exposure, motion blur, and camera and control latency. The depth-corruption model that the encoder pretraining already provides can then be applied to the student's training data as well.

The fifth extension concerns the grasp. Training on more than one face topology would extend the re-grasp strategy to profiles that a hexagon does not cover, and other grasp topologies than the thumb–index pinch would let the hand adapt its grasp to the resistance of the thread.

The last extension completes the distillation study. The online flow-matching student is implemented and remains to be evaluated. Collecting multi-step teacher recovery fragments

from states that the student visits would give it on-policy labels without violating the constraint that a chunk label must be produced by the teacher alone. The pending sweeps over the action horizon, the number of integration steps, and the temporal-ensemble decay, and the closed-loop comparison of the point-cloud encoder with the Adapt3R-style encoder, are further ablations that Chapter 4 does not cover.

Appendix A

Implementation Details

A.1 Simulation and Controller

Table A.1: Simulation settings, thread model, and simulated hand of Section 3.2.

Parameter	Value
simulator	Isaac Sim 5.1, Isaac Lab 2.3.2, PhysX
physics rate / policy rate	120 Hz / 15 Hz (one command held over eight physics steps)
prismatic thread drive	stiffness 2×10^5 N/m, damping 200 N s/m, force limit 1000 N
thread friction (3.5)	$\tau_c = 2 \times 10^{-3}$ N m, $c_v = 2 \times 10^{-4}$ N m s/rad, $\omega_s = 0.1$ rad/s
thread pitch ρ	3.0 / 3.5 / 4.0 mm for M24 / M30 / M36
nut carrier mass	1 g
prismatic lower limit	0.2 mm above the physical stop
pinch-joint effort limit	0.5 N m (hand drives otherwise 20 N m)
pad friction coefficient	1.5 nominal, randomized (Table 3.2); robot 0.75, nut 1.0
arm joint dynamics	identified FR3: friction 0.2 N m, armature 0.195 kg m^2, damping 0.003 N m s/rad per joint
joint-limit guard	episode ends when an arm joint is within 0.15 rad of its limit
reset pose	hand wound 45° about the bolt against the threading direction and tilted 30° wrist-up; bolt on the hardware holder

The pinch-joint effort limit bounds the grip force. A replay of a policy trained without the limit gave median pad forces of about 43 N at the thumb and 38 N at the index. Limits of 1.0 N m and 0.3 N m reduced them to about 27/21 N and 12/7 N, whereas 0.1 N m let the grip collapse. The retained limit of 0.5 N m lies between the two working values.

The force channels of the teacher have the following counterparts on the physical system. The simulated joint torques are the reaction torques that PhysX projects onto the arm joint axes, not the commanded torques, so contact loads appear in them. Because the simulated arm carries no gravity load, their counterpart on the FR3 is the external-torque estimate `tau_ext_hat_filtered` of libfranka, not the raw sensor torque. The fingertip forces are the pad contact forces along the pad normal, the only component that the fingertip sensors of the hand measure. In simulation, the Jacobian of the controller is the flange Jacobian shifted to the grasp point by the lever arm $\boldsymbol{r}$, $J_v = J_{v,\text{flange}} - [\boldsymbol{r}]_\times J_{\omega,\text{flange}}$, and the mass matrix is the arm block of the generalized mass matrix, which includes the inertia of the attached hand.

A.2 Robot Assets

The simulated robot is composed of three parts: the FR3 arm, an adapter flange, and the RH56DFX hand. The geometry and colliders of the arm come from the base USD asset of the robot. This asset authors no link inertia, so PhysX derives mass and inertia from the convex-hull colliders, which do not describe the real link masses. The arm dynamics are therefore taken from the fr3_v2 model of MuJoCo Menagerie, which is derived from the

Table A.2: Action decoding and controller of the shared Cartesian contract. The values are nominal. The per-episode randomization of the simulated controller is listed in Table 3.2. In the physical column, "same" marks values shared with simulation.

Quantity	Simulation	Physical FR3
Action decoding		
position offset scale	0.05 m per unit action	same
roll, pitch scale	1 rad per unit action	same
yaw range	$[-180°, 90°]$, inside the wrist joint limits	same
pinch joints	±1 moves by half the joint range; 0 holds the closed grasp	same
smoothing factor	0.0625; fingers 3×	same
step limit from current pose	0.02 m per axis, 0.097 rad per Euler angle, every physics step	same, every 1 kHz cycle
Impedance law (2.13)		
stiffness K_p	$[565, 565, 565]$ N/m, $[28, 28, 28]$ N m/rad, world axes	same
damping K_d	$2\sqrt{K_p}$ (critical)	same
posture gains $K_{p,0}$, $K_{d,0}$	10, 6.32	same
nullspace projector	dynamically consistent	same
wrench dead zone	randomized, Table 3.2	none
torque limit	±100 N m per joint	robot torque-rate limit
control rate	120 Hz (physics rate)	1 kHz
Model quantities		
Jacobian J	PhysX flange Jacobian, shifted to the grasp point	libfranka robot model, shifted to the grasp point
mass matrix M	PhysX generalized mass matrix, arm block	libfranka robot model, hand as end-effector load
gravity	disabled for the arm links	compensated by libfranka
Coriolis term	none	none
grasp-frame velocity	mean thumb and index tip velocity, flange angular velocity	arm-only grasp-point velocity
Force channels (teacher only)		
arm joint torques	PhysX projected joint-reaction torque	`tau_ext_hat_filtered`
fingertip forces	pad contact force along the pad normal	fingertip force sensors of the hand

public `franka_description` URDF of Franka Robotics. A layer on top of the base asset overrides, for links 1 to 7, the mass, the center of mass, and the principal inertia with its principal axes, and for joints 1 to 7 the position limits, which follow the limits of the current Franka system image. Geometry, colliders, the hand, and its mount are unchanged. The joint friction of 0.2 N m, the armature of 0.195 kg m^2, and the viscous damping of 0.003 N m s/rad of the Menagerie model (Table A.1) are not part of the layer and are applied to every environment at runtime, which lets the dataset collection scale the joint friction per environment (Table A.9).

The hand is mounted on the arm flange through an adapter flange of 76 mm diameter and 10 mm thickness. The flange-to-palm transform of the simulated asset is a 10 mm offset along the flange axis and a rotation of 180° about it, as on the physical installation. Mass and inertia of the hand enter the controller of the physical robot as the end-effector load, so that the mass matrix of the deployment controller matches that of the simulated arm with the attached hand (Table A.2).

The hand asset starts from the vendor URDF of the RH56DFX. Its finger couplings, register maps, and joint limits were identified from camera images of the physical hand, and Section A.3 describes the procedure, the fitted parameters, the remaining fingertip errors, and the effect of the fitted model on teacher training.

A.3 Finger Kinematics of the Simulated Hand

The simulated hand of Section 3.2.1 is built from the vendor URDF of the RH56DFX. On the physical hand, one motor drives the proximal flexion joint of each finger and a second motor the thumb abduction, and the distal joints follow through a linkage. The URDF approximates each linkage with a linear mimic coupling, and the hand driver converts each motor register $r \in [0, 1000]$, where 1000 is open and 0 is closed, into a URDF joint angle with a linear map.

Both approximations decide where the fingertips are for a given command. The fingertips in turn define the grasp frame, whose origin is the midpoint of the thumb and index fingertips (Section 3.2.3). This section measures the thumb and index joint angles of the physical hand from camera images, fits the couplings and register maps to them, and compares the fingertip positions of the vendor model and the fitted model with the measured ones.

A.3.1 Method

The thumb has four joints: the abduction q_{t0}, the actuated flexion q_{t1}, and the coupled joints q_{t2} and q_{t3}. The index has the actuated joint q_{i0} and the coupled joint q_{i1}. A coupling is modeled as $q_f = k\,q_l + c$, where q_l is the leading joint and q_f the following one. The vendor model uses $c = 0$.

Capture. Each sweep commands the motor registers directly, without the hand driver, and sets the force threshold of every finger to its firmware maximum so that no finger stops early. Middle, ring, and little finger stay open. The arm holds one pose for the whole sweep. At each stop, the sweep waits until no register has changed by more than two counts for 0.5 s. It then records one 1920×1080 color image of the fixed workcell camera, the per-pixel median of eight aligned depth frames, and the register feedback of all fingers. A flexion sweep closes and reopens the thumb flexion in register steps of 250, with the abduction register at 250, where the thumb flexes nearly perpendicular to the line of sight of the camera. A second pass closes the thumb with the abduction register at 1000. The sweep then closes and reopens the index in steps of 200, and finally follows the pinch path of the policy toward its closed target. An abduction sweep moves the abduction register through six values from 1000 to 250 and back, at a second arm pose that views the abduction. Three sweeps are used: a flexion sweep for fitting (31 stops), a second flexion sweep held out for validation (33 stops), and an abduction sweep (11 stops).

Joint fit per stop. The hand meshes are posed by the forward kinematics of the arm and hand model and projected with the calibrated camera intrinsics and extrinsics. One rigid palm correction per arm pose, shared by all stops of a sweep, first absorbs the residual error of camera calibration and hand mounting. For the fitting sweep, it is $(1.2, 4.5, 1.4)$ mm in translation and $(-1.6, -2.4, 1.4)^\circ$ in rotation. Then, for every stop, the joints of the thumb and of the index are fitted separately and without coupling, each finger by minimizing

$$c(\boldsymbol{q}) = \frac{1}{|V|} \sum_{p \in V} \frac{\min\left(|\hat{d}_p(\boldsymbol{q}) - d_p|,\, \delta\right)}{\delta} + 0.6\, \frac{1}{|O|} \sum_{p \in O} \frac{\min\left(D_p,\, \gamma\right)}{\gamma}. \tag{A.1}$$

Here V is the set of pixels covered by the rendered finger and not occluded by another finger, $\hat{d}_p$ the rendered and d_p the measured depth, O the outline pixels of the rendered finger, and D_p the image distance from pixel p to the nearest edge of the color image. The caps $\delta = 12$ mm and $\gamma = 18$ px bound the influence of missing depth and of unrelated edges. A pixel without measured depth counts with the full cap. The minimization first scans the actuated joint, and for the thumb also the abduction when it is fitted, over ± 0.4 rad around the vendor-model pose in steps of 0.05 rad. A Nelder–Mead search refines the best scan point. A second pass restarts every stop from the solutions of its neighboring stops and keeps the lower cost. In the flexion sweeps, the abduction is not fitted. It is held at the value that the abduction sweep measured for the same register, because the abduction sweep views this joint better.

Model parameters. The couplings are least-squares lines through the fitted joint angles. The thumb couplings use the nine flexion stops at abduction register 250. The index coupling uses the seven stops with $q_{i0} < 0.8$ rad, for the reason given in the results. The register maps are linear in $\tilde{r} = (1 - r/1000)\, q_{max}$, the register mapped onto the URDF joint range $[0, q_{max}]$. The joint limits of the fitted model are the measured range of each joint, rounded outward to 0.01 rad. The upper limit of q_{i1} stays at the URDF value.

Validation. Two checks test the fit. The first renders the hand at known joint angles and fits them from the vendor-model pose. Ten images are rendered at five register settings and two arm poses, with the true joint angles perturbed from the vendor-model pose by Gaussian noise of 0.12 rad per joint and a further 0.1 rad on the abduction and 0.15 rad on q_{i1}. The rendered depth receives Gaussian noise of 1.5 mm and loses 5 % of its pixels. The starting poses are off by 0.09 to 0.15 rad RMSE per joint. The fit recovers the abduction, the actuated flexion joints, and q_{i1} to 0.002 to 0.005 rad RMSE, q_{t2} to 0.011 rad, and q_{t3} to 0.028 rad with a maximum of 0.077 rad. The small distal thumb link is thus the least observable joint. The second check isolates the couplings. Given the measured leading joints, each coupling predicts the following joints, and the resulting fingertip positions in the palm frame are compared with the camera fit. This check runs on the fitting sweep and on the held-out sweep. Stops 30 to 32 of the held-out sweep are excluded, because thumb and index were in contact there and the index then failed to reopen.

A.3.2 Results

Table A.3 lists the fitted parameters next to those of the vendor model. The couplings of the thumb flexion chain and of the index differ from the vendor ratios by 6 to 26 %, and the register maps of all three actuated joints carry offsets. The thumb flexion coupling is nearly exactly linear, with a residual of 0.009 rad. The distal thumb coupling is the weakest fit, with a residual of 0.031 rad and $R^2 = 0.92$. For the actuated joints, opening and closing passes at the same register differ by at most 0.008 rad, so the drives show no measurable hysteresis in free space.

Figures A.1 to A.3 overlay both models on the camera images of six stops per sweep, and Table A.4 summarizes the fingertip errors. The thumb abduction has the largest error. The vendor map places the open thumb at 0 rad, whereas the camera measures 0.258 rad, and the gap closes toward full abduction, where the vendor map gives 1.246 rad and the camera 1.263 rad. Over the six stops, the vendor model misplaces the thumb tip by 18.3 mm on average, and the fitted map stays within 0.016 rad of the camera and within 4.8 mm at the tip. In thumb flexion (Figure A.2), the actuated joint of the physical thumb spans −0.097 to 0.374 rad, whereas the vendor map covers 0 to 0.32 rad, and the vendor coupling bends the outermost joint q_{t3} too far. The fitted model reduces the mean tip error from 8.9 to 2.1 mm.

The index (Figure A.3) shows the smallest register-map error. Its actuated joint lies 0.03 to 0.05 rad below the vendor map over the closing range and 0.10 rad below it when fully open. The vendor coupling of 1.12 overbends the distal joint, whereas the fitted coupling is 0.89. Above $q_{i0} = 0.8$ rad, the measured distal angle departs from the line: at $q_{i0} = 1.03$ rad the camera measures $q_{i1} = 1.02$ rad where the fit predicts 0.94 rad, and at full closure, $q_{i0} = 1.29$ rad, it measures 1.42 rad where the fit predicts 1.17 rad. The coupling is therefore fitted below 0.8 rad. On the pinch path of the fitting sweep, the index reached at most $q_{i0} = 0.52$ rad, inside the fitted range.

The coupling check in the lower half of Table A.4 keeps the measured leading joints and changes only the couplings. On the held-out sweep, the fitted couplings reduce the mean

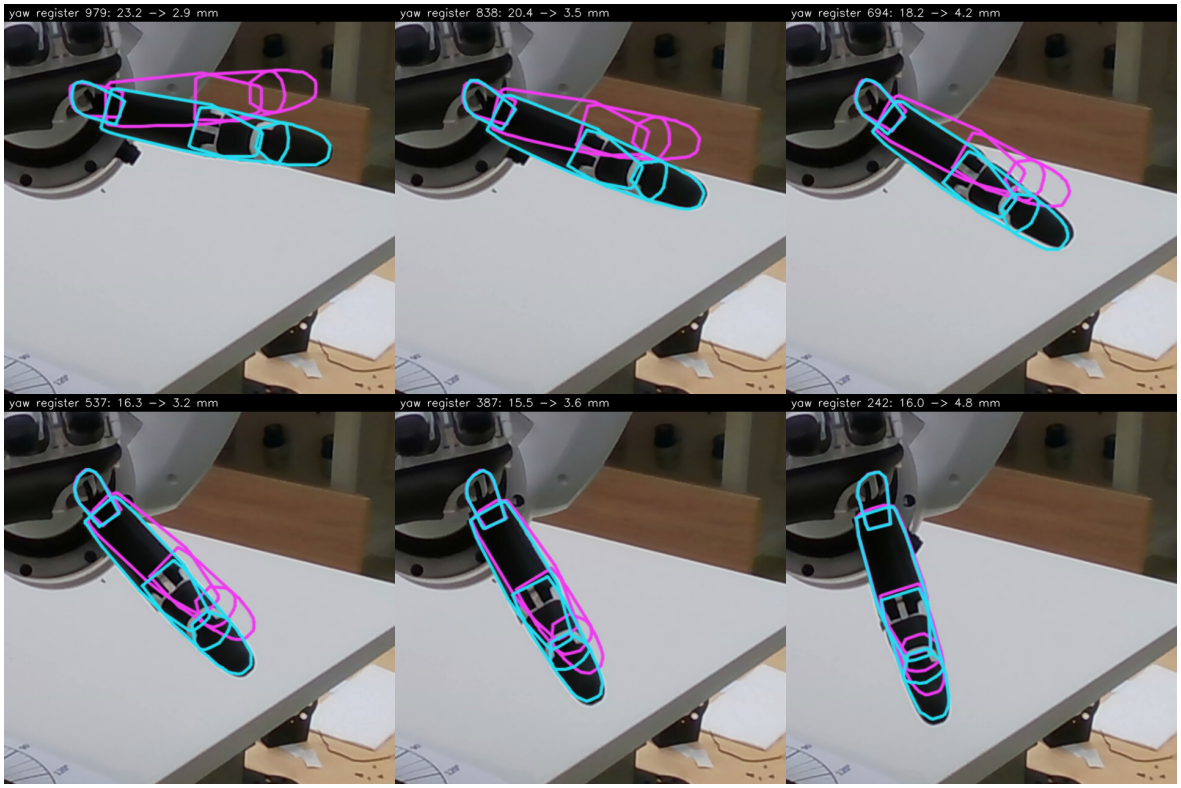


Figure A.1: Thumb abduction sweep on the physical hand, six stops from open (top left, register 979) to fully abducted (bottom right, register 242). Magenta outlines show the vendor model and cyan outlines the fitted model at the register of each stop, both projected into the camera image. Each label gives the fingertip distance of the vendor and the fitted model from the camera fit. Mean over the stops: 18.3 mm for the vendor model, 3.7 mm for the fitted model.

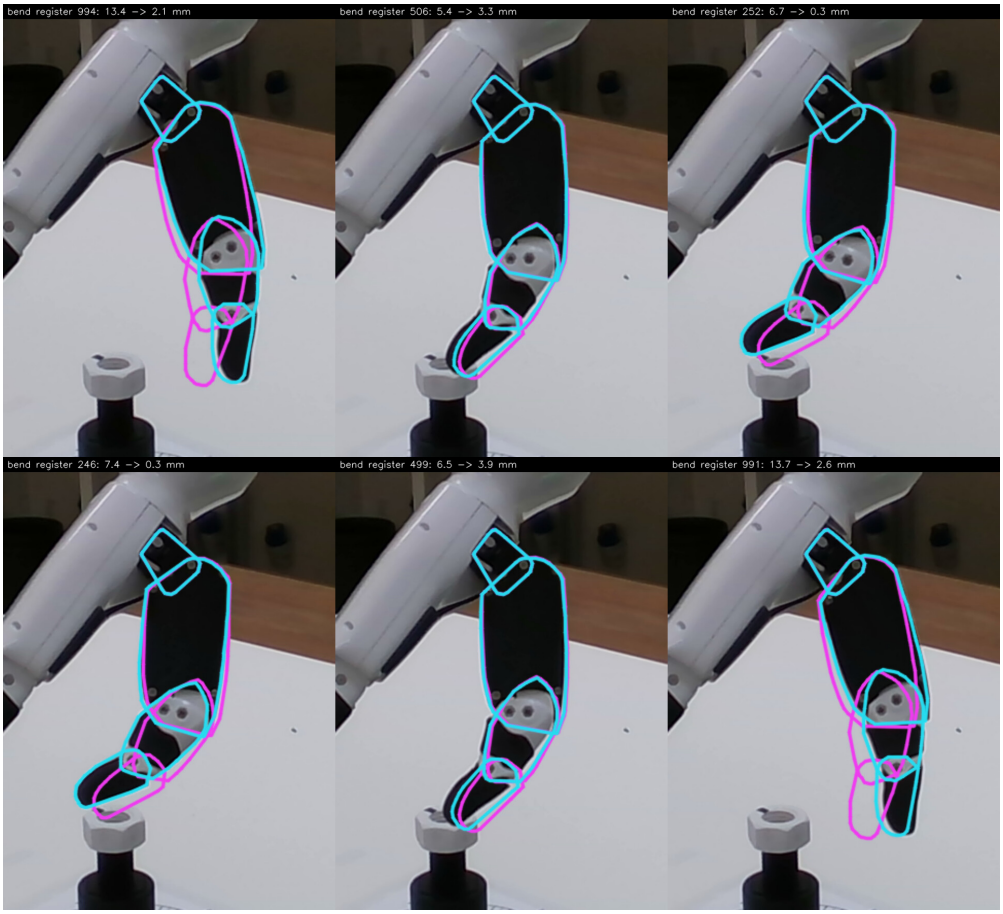


Figure A.2: Thumb flexion sweep at abduction register 250, six stops from open (register 994) through closed and back to open (register 991). Magenta outlines show the vendor model and cyan outlines the fitted model. Mean fingertip distance from the camera fit over the stops: 8.9 mm for the vendor model, 2.1 mm for the fitted model.

Table A.3: Finger kinematics of the vendor model and of the model fitted to the physical hand. The register maps give the joint angle as a function of $\tilde{r} = (1 - r/1000)\, q_{\max}$, with $q_{\max}$ the URDF upper limit of the joint. The vendor maps are those of the hand driver before the fit. The residual is the standard deviation of the coupling fit. Angles in rad.

Quantity	Vendor model	Fitted model	Residual
Register maps			
thumb abduction q_{t0}	$\tilde{r}/0.75$, clipped to $[0, 1.246]$	$1.08\,\tilde{r} + 0.246$	
thumb flexion q_{t1}	$0.73\,\tilde{r} - 0.03$, clipped to $[0, 0.48]$	$\tilde{r} - 0.086$	
index flexion q_{i0}	$\tilde{r}$	$\tilde{r} - 0.05$	
Couplings			
q_{t2}	$1.1425\, q_{t1}$	$1.2092\, q_{t1} + 0.0252$	0.009
q_{t3}	$0.7508\, q_{t2}$	$0.5592\, q_{t2} + 0.0062$	0.031
q_{i1}	$1.1169\, q_{i0}$	$0.8867\, q_{i0} + 0.0275$	0.034
Joint limits			
q_{t0}	$[0, 1.246]$	$[0.25, 1.27]$	
q_{t1}	$[0, 0.48]$	$[-0.10, 0.38]$	
q_{t2}	$[0, 0.358]$	$[-0.11, 0.47]$	
q_{t3}	$[0, 0.278]$	$[-0.07, 0.31]$	
q_{i0}	$[0, 1.333]$	$[-0.11, 1.29]$	
q_{i1}	$[0, 1.527]$	$[-0.02, 1.527]$	

thumb tip error from 5.5 to 3.9 mm. The mean index tip error rises from 1.3 to 1.7 mm, while its maximum falls from 4.2 to 3.8 mm. With the measured leading joints, both index couplings thus place the fingertip within a few millimeters. Most of the 6.9 mm index error of the vendor model in Figure A.3 therefore comes from its register map, not from its coupling.

Table A.4: Fingertip distance from the camera fit in mm, mean / maximum over n stops. Upper part: full models, each driven by the commanded registers, at the six stops of Figures A.1 to A.3. Lower part: couplings only, applied to the measured leading joints, on the thumb flexion stops at abduction register 250, the index stops with $q_{i0} \leq 0.85$ rad, and the pinch stops of the fitting sweep and of the held-out sweep.

Comparison	Fingertip	n	Vendor model	Fitted model
thumb abduction sweep	thumb	6	18.3 / 23.2	3.7 / 4.8
thumb flexion sweep	thumb	6	8.9 / 13.7	2.1 / 3.9
index flexion sweep	index	6	6.9 / 8.2	1.4 / 3.6
couplings, fitting sweep	thumb	20	5.8 / 14.8	4.0 / 13.1
	index	20	1.7 / 4.9	1.5 / 2.3
couplings, held-out sweep	thumb	20	5.5 / 14.9	3.9 / 12.7
	index	20	1.3 / 4.2	1.7 / 3.8

A.3.3 Fitted Model in Simulation

In the simulation, the fitted model replaces the couplings and joint limits of Table A.3, so that a simulated joint angle means the physical joint angle. Geometry, masses, and the pinch ranges of the action decoder are unchanged, so the action scale stays the same. Three further changes make the simulated linkage behave like the physical one. First, each following joint tracks the measured angle of its leading joint instead of the leading joint's target. With targets, a leading joint held at its effort limit left its follower closing on its own, which a rigid linkage cannot do. Second, the drives of the following joints are stiffened to 30 N m/rad and

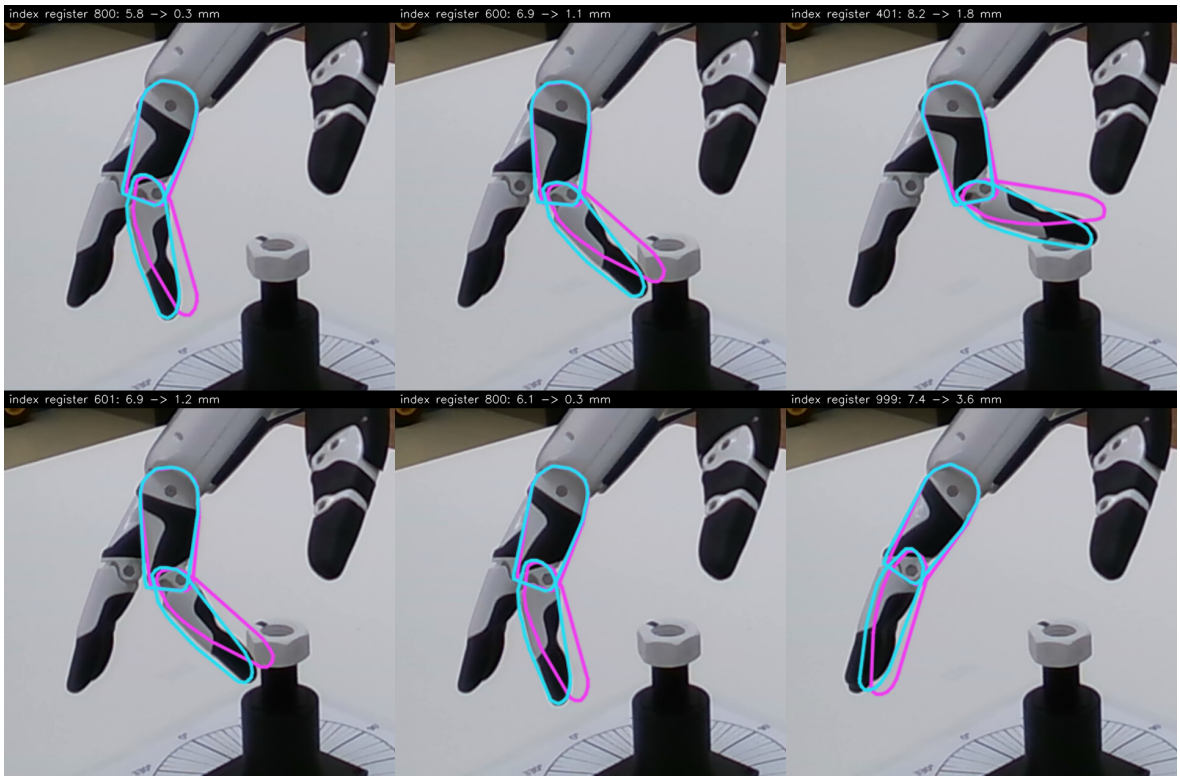


Figure A.3: Index flexion sweep, six stops from register 800 through register 401 and back to open (register 999). Magenta outlines show the vendor model and cyan outlines the fitted model. Mean fingertip distance from the camera fit over the stops: 6.9 mm for the vendor model, 1.4 mm for the fitted model.

3 N m s/rad, from 4.76 and 0.21 for q_{t2} and 0.9 and 0.042 for q_{t3} and q_{i1}. Third, the collision between the palm and the proximal thumb link is filtered. In the vendor model, this contact held the thumb at an abduction of 1.07 rad when the reset commanded 1.185 rad, an angle that the physical thumb reaches.

In a reset probe with eight environments, the fitted model reached the commanded abduction of 1.185 rad. Under a pinch command, its thumb flexion moved from 0.05 to 0.12 rad with q_{t2} within 0.002 rad of its coupling. The vendor model under the same command kept the thumb flexion at 0.025 rad, while q_{t2} closed alone, 0.12 rad away from its coupling. At reset, the two models place the thumb tip 7.7 mm apart and the grasp-frame origin 4.0 mm apart. A teacher trained with the same recipe and seed on both models reached a threading success rate of 0.87 on the fitted model and 0.82 on the vendor model after 200 epochs, averaged over the last ten iterations of one run each. On the fitted model, the grasp was learned about ten iterations later. The teacher and the student of this thesis are trained on the fitted model.

The fit has three limits. It covers free motion only, so the compliance of the linkages under grip load is not measured. The distal thumb coupling is the least certain parameter. The index coupling is linear only below $q_{i0} = 0.8$ rad.

A.4 Teacher Reward

A proper grasp is simultaneous thumb and index contact within 1 cm of the designated faces. Besides success and the time limit, an episode terminates when the nut leaves the bolt by more than 3 cm, tilts by more than 60°, or releases the thread, when an arm joint comes

Table A.5: Reward terms of the threading teacher (3.8). Progress terms pay only for newly achieved progress. The weight of P^{wrench} rises from zero at the first initial-yaw curriculum level to one at its full range.

Term	Definition	Weight
R^{center}	newly achieved fraction of the initial grasp-center–nut-center gap, until the grasp latches	30
R^{face}	newly achieved fraction of the initial pad–face gap, until the grasp latches	30
R^{grasp}	one-time bonus when the proper grasp has held for 10 consecutive steps	180
R^{hold}	per step with proper grasp and both pads pressing with at least 1 N	0.24
R^{turn}	newly achieved fraction of the keypoint distance to the target pose remaining at latch, while the grasp is live	72
R^{target}	one-time bonus on success	300
$P^{\text{magnitude}}$	normalized commanded translation and yaw	10^{-3}
$P^{\Delta a}$	action change	0.1
P^{align}	misalignment of hand and nut frames	0.02
P^{force}	summed pad contact force above a threshold $U(5, 10)$ N, per newton	0.05
P^{settle}	ω_t^2 from 50° of rotation onward	0.25
P^{wrench}	lateral force above 2 N and axial force above 5 N per newton, tilting torque above 0.05 N m per N m	0.05 / 0.05 / 1.0
P^{clear}	squared normalized violation of a 3 cm arm–table clearance	2.0
P^{contact}	arm–table contact force above 2 N, per newton	0.05
P^{limit}	barrier on each arm joint from 0.5 rad of headroom down to the 0.15 rad guard	0.5 (per joint and step at the guard)

within 0.15 rad of its limit, or when the latched grasp is lost. Each promotion of the success-hold curriculum requires a success rate of at least 80 % over completed episodes and at least five episode lengths since the previous change.

A.5 Hybrid Teacher

Table A.6: Thresholds and timings of the scripted transitions of the hybrid teacher (Section 3.4.1).

Parameter	Value
release trigger	nut rotation within 5° of the target turn
release waypoints	4, interpolated over 0.7 s each
return segment	1.0 s
return gate	grasp frame within a tolerance drawn per return between 5 and 15 mm and between 5
transition pose clip	0.05 m and 0.25 rad per axis on release and return rows (policy rows: 0.02 m, 0.097 ra
transition dead zone	task space, drawn per transition up to 7.5 N and 1 N m
return timeout	16 s (transition failure)
target turn clamp	$[55°, 65°]$

A.6 Teacher Hyperparameters

A.7 Student Hyperparameters

A.8 Dataset Summary

Table A.7: PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic).

Parameter	Value
parallel environments	500
epochs	200
rollout horizon	256 steps
minibatch size	512
mini-epochs per update	4
recurrent sequence length	128
discount γ / GAE τ	0.995 / 0.95
learning rate	10^{-4}, adaptive with KL threshold 0.008
clipping ϵ	0.2
critic coefficient	2
entropy coefficient	0
gradient-norm clip	1.0
bounds loss coefficient	10^{-4}
actor / critic network	LSTM 2×1024 (layer norm) $\rightarrow$ MLP 512–128–64, ELU
action distribution	diagonal Gaussian, state-dependent σ
mixed precision	yes

Table A.8: Hyperparameters of the conditional flow-matching student.

Parameter	Value
action contract	9-D filtered Cartesian target divided by $[3,4,16,3,3,3,2,2,2]$, in $[-1,1]$
normalization	none for actions; fingertip forces divided by 20 N, joint torques by the joint effort limits
action horizon H	8 (0.53 s at 15 Hz)
training data	36 ten-cycle episodes, no validation split
target offset	0 (pre-step alignment)
terminal-hold padding	yes
proprioception	35-D (10 q, 7 arm $\dot{q}$, previous action, 2 fingertip forces, 7 arm joint torques)
proprioception encoder	MLP 128–128, ELU
visual feature	64-D (DP3: 4096 XYZRGB points, PointNet 64–128–256, max pool)
point-cloud preprocessing	depth in $[0.10, 2.0]$ m, workspace crop box with 40 mm margin, RGB in $[-1,1]$
encoder initialization	none, trained from scratch jointly with the flow head
condition width	256
progress / phase embeddings	MLP 1–256–256 / 3–256–256, ELU; clock normalized by 98.2 s (1473 steps)
flow-time embedding	sinusoidal, 64-D
velocity network	Transformer, 4 layers, 8 heads, width 512, FF 2048, GELU, pre-norm
probability path	conditional optimal transport (linear interpolant)
optimizer	AdamW, learning rate 10^{-4}, gradient clip 1.0, batch 64, 100 epochs, seed 0
parameter EMA decay	0.999
auxiliary loss	none
RGB augmentation	none
depth augmentation	noise $\sigma = 5\,\text{mm} + 0.002\,\text{m}^{-1} d^2$, smooth offset 3 mm, quantization 1 mm, dropout at edges above 15 mm $p = 0.35$, correlated holes $p = 0.12$ (11 px), pixels $p = 0.01$
sampler	Euler, $N = 16$ steps (online training: 8; CPU deployment: 2)
temporal-ensemble decay	0.5
online variant	auxiliary weight 0.01, encoder learning rate 10^{-5}, 8 Euler steps

Table A.9: Demonstration data of the M24 ten-cycle collection used for the reported students.

Property	Value
teacher	single-size M24 teacher of Section 3.3, mean action, $\beta = 1$
simulated demonstrations	36 successful ten-cycle episodes from one rollout of 16 parallel environments (6400 control steps, seed 1; 14 environments contributed)
frames (15 Hz)	53 408 (per episode 1341 / 1474 / 1838 min / median / max)
cycle rollback	failed cycle retried from the last completed return, at most 3 times; 20 episodes contain a retried cycle, 10 136 rows dropped in total
start state	nominal pose, no initial yaw, nut at the top of the thread
per-cycle target	60° grid, release 5° before the target
task randomization	none: nut mass, nut and thread friction, controller parameters, start poses, and target turn fixed at nominal values
proprioception noise	fingertip pose $\mathcal{N}(0, 0.5\,\text{mm})$ and $\mathcal{N}(0, 0.2°)$ per step; fingertip forces gain $U(0.6, 1.4)$, offset $U(-2, 2)$ N per episode, $\mathcal{N}(0, 2\,\text{N})$ per step; arm joint torques gain $U(0.8, 1.2)$, offset $U(-0.02, 0.02)$ per episode, $\mathcal{N}(0, 0.01)$ per step, in units of the effort limits
joint and pad friction	arm joint friction scaled by $U(0.8, 2.0)$, finger pad friction $U(1.2, 1.8)$, once per environment
head camera pose	per episode, uniform per axis within ±11.2 mm and ±1.13°
release and return	thresholds, clip, and dead zone of Table A.6
impulses per episode	30 (one per policy, waypoint, and return segment), 0.05–0.25 N s, 0.05–0.20 s after segment start
visual randomization	dome light, table, nut/bolt, floor textures, background
material sampling	per environment and reset, roughness $U(0.3, 0.9)$, metallicity $U(0, 0.15)$
dome light and floor	shared, redrawn with $p = 0.3$ per reset, light intensity $U(1000, 4000)$
background image	$p = 0.5$ per environment and reset, replaces pixels at 1.5 m or beyond or without depth
image size	320 × 180 RGB and metric depth (640 × 480 sensor, centered 640 × 360 crop, downscaled by two)

Appendix B

Sim-to-Real Measures in the Arm Controller

The student commands the arm through the operational-space impedance law of (2.13), and the same law runs in training and on the physical FR3 (Section 3.2.3). The two arms behind that law differ. The training arm is a gravity-free PhysX articulation, whereas the physical arm has gearbox friction that depends on joint velocity and posture. This chapter documents what was added to the deployment controller to narrow that gap, and the measurements behind each addition. It also covers *ros-sim*, a MuJoCo simulation of the workcell that runs the same ROS 2 stack and the same C++ controller as the physical robot and is used to rehearse every hardware rollout.

Each section separates two kinds of statement. *Results* are read from logged data. *Interpretation* explains the results and is marked as such. Every result states the condition it was measured under. Wrenches are given in N and N m, pose errors in mm and degrees, and joint quantities in rad and rad/s. "Dead zone" always means the task-wrench dead zone defined in Section B.1.

Table B.1 lists the controller settings that all measurements in this chapter share.

Table B.1: Settings of the deployment Cartesian impedance controller shared by all measurements in this chapter. Hardware runs without friction compensation differ only in that term. ros-sim differs in the model source (built-in kinematic model instead of libfranka), in gravity (off in the scene instead of compensated), in friction compensation (off by default), and in the dead zone, which can only be set there.

Quantity	Value
translational / rotational stiffness K_p	565 N/m / 28 N m/rad
damping K_d	$2\sqrt{K_p}$, i.e. 47.5 N s/m / 10.6 N m s/rad
nullspace	stiffness 10, mass-weighted dynamically consistent projector
Coriolis compensation	off, as in training
policy clip (per axis, 1 kHz, on the policy goal)	0.02 m / 0.097 rad
tracking-fault limits (policy phase)	0.08 m / 0.35 rad
transition-clip ceilings / fault ceilings	0.05 m / 0.25 rad, 0.1 m / 0.5 rad
tool offset in the flange frame	$(-0.058208, -0.028713, 0.173779)$ m
joint torque limits	87 N m (j1–j4), 12 N m (j5–j7)
friction compensation c_j / v_s / τ_s	0.3 N m on every joint / 0.05 rad/s / 0.3 N m
task-wrench dead zone	0 on hardware, selectable in ros-sim

B.1 Task-Wrench Dead Zone in Training

Purpose and implementation. The teacher is trained with a dead zone on the task wrench, which models the unreliability of the controller at low forces. Let $\boldsymbol{w} = K_p\boldsymbol{e} - K_d\dot{\boldsymbol{x}}$ be the task

wrench of (2.13) in the robot base frame and $\boldsymbol{d} \in \mathbb{R}^6_{\geq 0}$ the thresholds. Each component is shrunk toward zero,

$$\tilde{w}_i = \begin{cases} 0, & |w_i| < d_i, \\ \operatorname{sign}(w_i)\,(|w_i| - d_i), & \text{otherwise}, \end{cases} \qquad i = 1, \ldots, 6, \tag{B.1}$$

and $\tilde{\boldsymbol{w}}$ replaces $\boldsymbol{w}$ in $J^{\mathrm{T}}\boldsymbol{w}$. The nullspace term is not affected. The thresholds are drawn as $\boldsymbol{d} \sim U(\mathbf{0}, \boldsymbol{d}_{\max})$ with $\boldsymbol{d}_{\max} = [5, 5, 5\,\mathrm{N}, 1, 1, 1\,\mathrm{N\,m}]$ at every reset and again every 2 s, when they are redrawn for all environments at once. The training arm has gravity disabled. Recipes with physics randomization also add joint friction, scaling a nominal 0.2 N m per joint by a factor from $U(0.8, 2.0)$.

At the gains of Table B.1, the worst-case thresholds correspond to a band of $5/565 = 8.8\,\mathrm{mm}$ in position and $1/28\,\mathrm{rad} = 2.0°$ in orientation, inside which the spring produces no wrench. With zero position error, a pure damping wrench is also removed below $5/47.5 = 0.105\,\mathrm{m/s}$ and $1/10.6 = 0.094\,\mathrm{rad/s}$.

Recipes. Three dataset recipes treat the dead zone differently:

- the *extended-randomization* recipe inherits the teacher's dead zone together with gain noise ($\ell = 0.41$), a random smoothing factor $U(0.025, 0.1)$, the arm joint friction factor $U(0.8, 2.0)$, and nut-depth randomization.
- the *proprio-friction* recipe sets the dead zone to zero, fixes the gains and the smoothing factor at 0.0625, and keeps the joint friction factor $U(0.8, 2.0)$.
- the *return-dead-zone* student adds a dead zone only during release and return. It is drawn per return and per axis from zero to a configured maximum and set back to zero when the arm is handed back to the policy.

Interpretation. The following reading is an analysis of (B.1), not a measurement. On a gravity-free arm, the dead zone is a hole in the stiffness: inside the band no task wrench acts, and the hand floats. In free space this resembles static friction, because a small error is not corrected. In contact the two differ. A frictionless arm inside the band offers no resistance, so contact forces between fingers and nut can move the hand and re-center the grasp. A physical arm resists the same contact forces through joint friction. The dead zone is also memoryless and independent of speed, whereas joint friction acts in joint space, depends on the posture through the Jacobian, and depends on the direction of motion. Sections B.3 to B.10 test which of these properties matter for deployment.

B.2 Controller Additions for Deployment

The deployment controller is a ros2_control plugin whose control law reproduces (2.13). Two terms were added to it for deployment.

Dead zone in ros-sim. The controller implements (B.1) component-wise and applies it to the task wrench before $\boldsymbol{\tau}_{\mathrm{task}} = J^{\mathrm{T}}\tilde{\boldsymbol{w}}$. The thresholds default to zero and can only be set in ros-sim, either to $\boldsymbol{d}_{\max}$ (the worst case) or to six explicit values. Training redraws the thresholds at every reset and every 2 s, whereas ros-sim holds them fixed for a whole run, so the worst case is the upper end of the training distribution. Training expresses the wrench in a world

frame that is the robot base rotated by π about z. This flips the signs of x and y but not the magnitudes, so the base-frame dead zone gives the same result. Two unit tests check that the controller's dead zone matches the training implementation component-wise and that it is off by default.

Joint friction compensation on hardware. Friction compensation adds a Coulomb torque per joint before the torque clamp and the rate limiter,

$$\tau_{f,j} = c_j \left[s_j \operatorname{sign}(\dot{q}_j) + (1 - s_j) \tanh\left(\frac{\tau_{\text{imp},j}}{\tau_s} \right) \right], \qquad s_j = \min\left(1, \frac{|\dot{q}_j|}{v_s} \right), \tag{B.2}$$

where $\tau_{\text{imp}} = \tau_{\text{task}} + \tau_{\text{null}}$ is the impedance torque, c_j the Coulomb coefficient, v_s the velocity scale, and τ_s the torque scale (Table B.1). While a joint moves faster than v_s, the compensation follows its velocity. Near standstill it follows the commanded impedance torque, so a joint that is asked to move receives the breakaway torque, and a joint at equilibrium ($\tau_{\text{imp}} \approx 0$) receives none. The two blend linearly in $|\dot{q}_j|/v_s$. The coefficients are read only when the controller is configured, so switching friction compensation on or off requires a restart of the controller. Four unit tests cover the off state, the moving branch, the standstill branch, and the blend.

The value of 0.3 N m was chosen from two earlier hardware runs, which are not among the runs analyzed in this chapter. According to the notes kept with the controller settings, the yaw clip in those runs asked the wrist joint for about 1 N m, of which about 0.65 N m was lost to joint friction (commanded −1.0 N m, −0.35 N m at the link). The wrist then turned at 22–28°/s instead of 37–41°/s, and the fingers reached the nut 0.3 rad short of the simulated pre-rotation. The moving losses measured in those runs were 0.32–0.44 N m on j2 and j5–j7, and 0.3 N m was chosen to stay below all of them. The same notes report that 0.4 N m per joint raised a student in ros-sim from 0 to 3 cycles in 450 steps.

Logged state. Every controller cycle publishes a state message at 50 Hz and, for recording, at 1 kHz. It carries the goal, the clipped target, and the measured pose, the 6-D error, the wrench after the dead zone, τ_{task}, τ_{null}, the commanded torque, the filtered measured torque, the gravity torque, and the clip and fault limits in effect. In ros-sim, the wrench removed by the dead zone is therefore known exactly in every cycle as $K_p \boldsymbol{e} - K_d \dot{\boldsymbol{x}} - \tilde{\boldsymbol{w}}$.

B.3 Rollouts in ros-sim with and without the Dead Zone

Purpose and method. This comparison tests whether the training dead zone makes ros-sim reproduce the physical arm. The proprio-friction student S_1 was run three times in ros-sim without the dead zone and three times with the worst-case dead zone, without friction compensation and with the same runner settings as on hardware. It was compared with two hardware runs of the same student. One of them used friction compensation; for the other, the setting was not recorded. Every cycle is cut from its first policy sample and aligned on that step. For every observation channel, the RMS distance to the hardware mean is computed over the first 69 policy steps.

Results. The hardware runs completed 3 and 6 cycles, which gives 9 hardware cycles. In ros-sim, five of six runs ended with a re-home failure (a 15 s timeout of the controller manager), so the comparison has 3 cycles without the dead zone, none of them completed, and 4

Table B.2: Per-cycle metrics of the proprio-friction student S_1 on hardware and in ros-sim with and without the worst-case dead zone. Ranges are over cycles. The return error is taken at the last sample of the return, when the re-home gate of 30 mm and 25° is met. Grip width is the median fingertip separation over the last 30 policy steps.

Metric	Hardware	ros-sim, worst-case dead zone	ros-sim, no dead zone
cycles analyzed	9	4	3
policy steps per cycle	84–98	93–94 (112 in one cycle 2)	69–70
return error at the gate	11.5–12.9 mm, 24.8–25.0°	17.2–17.8 mm, 24.5–25.0°	3.2 mm, 14.2–15.3°
lowest grasp height	108.8–113.3 mm	121.2–122.9 mm	114.3–114.7 mm
grip width	31.7–35.2 mm	6.1–8.0 mm (28.2 in one cycle)	6.2 mm
peak j5 angle	1.89–1.98 rad	2.05–2.20 rad	1.68–1.69 rad

Table B.3: RMS distance of observation groups to the hardware mean over the first 69 policy steps of every cycle. "Hardware spread" is the distance of the hardware cycles to their own mean. Joint positions in rad, velocities in rad/s, previous actions in normalized units, pad forces divided by 20 N, torques divided by the joint limit.

Channel	Hardware spread	ros-sim, no dead zone	ros-sim, worst-case dead zone
q1 / q2 / q3	0.007 / 0.012 / 0.005	0.099 / 0.109 / 0.048	0.061 / 0.032 / 0.058
q4 / q5 / q6 / q7	0.015 / 0.015 / 0.016 / 0.019	0.116 / 0.164 / 0.172 / 0.128	0.035 / 0.068 / 0.033 / 0.105
arm joint positions	0.013	0.126	0.061
arm joint velocities	0.025	0.104	0.074
previous actions	0.027	0.295	0.099
pad forces	0.112	0.434	0.405
arm torques	0.016	0.060	0.060

cycles with the dead zone, one of them completed. Table B.2 summarizes the cycle metrics, Table B.3 the observation distances, and Figure B.1 the traces.

With the worst-case dead zone, the policy phase lasts as long as on hardware. The arm joint positions and the previous actions are two to three times closer to the hardware mean than without it, the joint velocities 1.4 times. Joints q2, q4, and q6 come within 0.032–0.035 rad of the hardware mean, q7 improves only from 0.128 to 0.105 rad, and q3 is closer without the dead zone. At the end of the return, the joint offsets from the reset posture agree with hardware on j2, j4, j5, and j6 (hardware −0.28 to −0.30, −0.47 to −0.50, 0.29 to 0.37, 0.36 to 0.43 rad, dead zone −0.31 to −0.32, −0.49, 0.33 to 0.35, 0.40 to 0.42 rad), but have the opposite sign on j1 and j3 (hardware −0.19 to −0.21 and +0.19 to +0.21 rad, dead zone +0.20 to +0.23 and −0.18 to −0.21 rad). Without the dead zone, the return reaches the orientation bound of the gate early and ends at 3.2 mm and 14–15°. With it and on hardware, the return ends exactly at the 25° bound, with 11.5–12.9 mm on hardware and 17.2–17.8 mm in ros-sim.

The dead zone does not reproduce the grasp. With it, the lowest grasp height is 8–14 mm above hardware, and in six of seven ros-sim cycles, with or without the dead zone, the grip closes to 6–8 mm, whereas hardware holds 32–35 mm. The pad forces are as far from hardware with the dead zone as without it. The torque observations are not comparable, because hardware logs the external-torque estimate and ros-sim logs the commanded torque.

Interpretation. The match of the policy length, the wrist joints, and the previous actions indicates that the worst-case dead zone reproduces how the physical arm follows the student's targets in free space. The return comparison shows that hardware and the dead-zone simulation are still turning back at the moment the gate's orientation bound is met, while the simulation without the dead zone has already finished turning. The mirrored offsets of j1 and j3 indicate that the redundant degree of freedom drifts in opposite directions in the two systems. The grasp gap is consistent with the contact argument of Section B.1: the dead

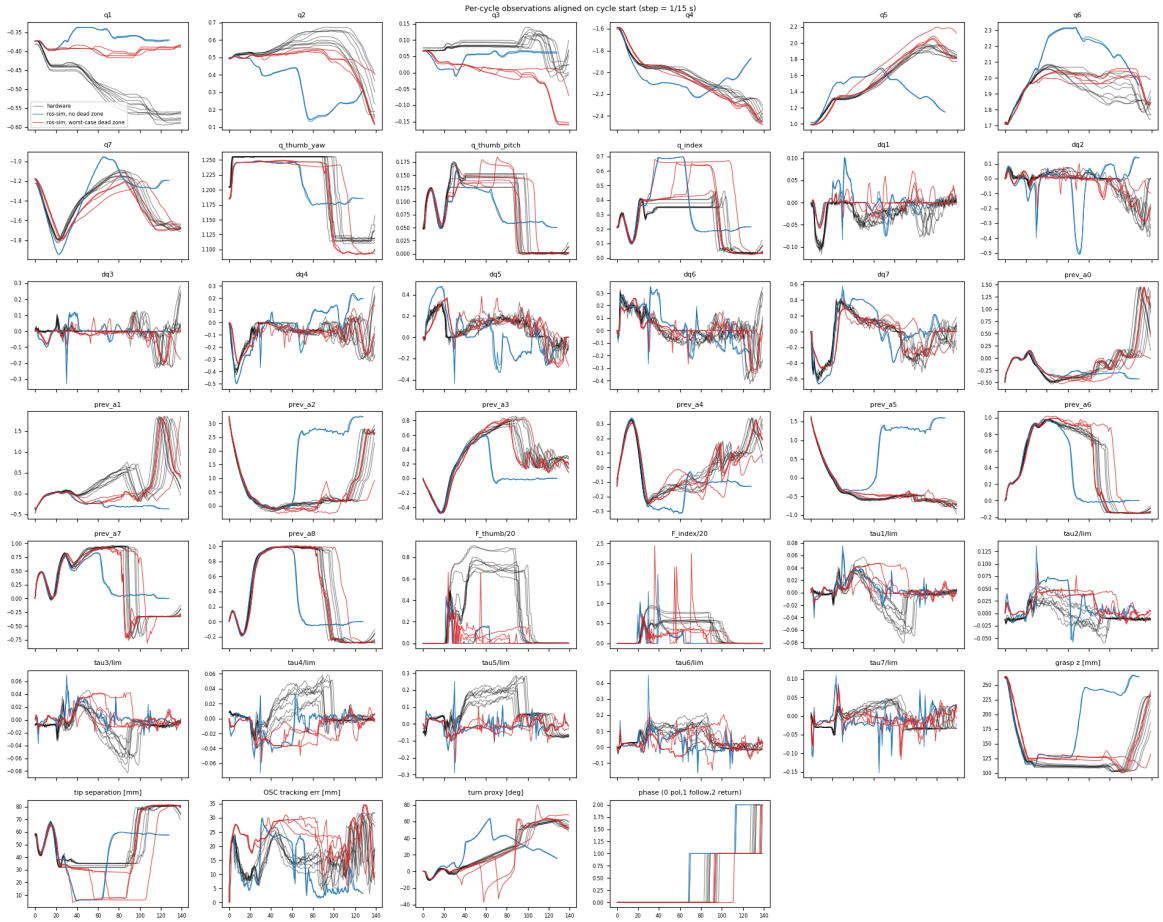


Figure B.1: Observations of the proprio-friction student S_1 per cycle, aligned on the first policy step (one step is 1/15 s). Black: nine hardware cycles of two runs. Blue: ros-sim without the dead zone. Red: ros-sim with the worst-case dead zone of 5 N and 1 N m. Panels show the student's proprioception (joint positions and velocities, previous action, pad forces, arm torques divided by the joint limit) and, in the last row, grasp height, fingertip separation, controller tracking error, the turn proxy, and the process phase.

zone makes free-space motion sticky but leaves the arm compliant against contact forces.

Limitations. The ros-sim side has seven cycles, and only one of them completed. At least one hardware run used friction compensation, and the ros-sim runs did not. The ros-sim arm carries the MuJoCo joint friction of 0.2 N m per joint of the FR3 model it is built from, so the "no dead zone" condition is not frictionless. The run logs do not record whether the dead zone was on, so the assignment of the six ros-sim runs to the two conditions is inferred from the order in which they were run.

B.4 Step-Aligned Comparison of the Return-Dead-Zone Student

Purpose and method. The return-dead-zone student S_2 was trained with the dead zone in release and return only (Section B.1). Its first hardware run was compared step by step with three ros-sim runs of the same checkpoint, one without the dead zone and two with the worst-case dead zone. All runs are aligned on the policy step and cut to the 99 steps of the hardware run, and the mean absolute difference per channel is computed in three windows. Whether friction compensation was active in the hardware run was not recorded.

Results. On hardware, the first grip held from step 24 to about step 60 and was lost at step 63, after which the run was interrupted at step 100. The windows of the comparison are approach (steps 0–21), contact and hold (22–62), and after the slip (63–98). During the approach, the policy actions of all three ros-sim runs differ from hardware by at most 0.035 in normalized units per channel. Table B.4 gives the grasp height and the vertical tracking error, and Figure B.2 shows the grasp position.

Table B.4: Grasp height and vertical controller error (target minus measured) of the return-dead-zone student S_2 per window, hardware against the ros-sim runs of the same checkpoint. Ranges over the steps of each window.

	Contact and hold (22–62)		After the slip (63–98)	
Run	grasp z [mm]	z error [mm]	grasp z [mm]	z error [mm]
hardware	103.4–119.5	−10.3 to 0.3	98.5–104.6	−3.9 to 16.0
ros-sim, no dead zone	114.0–133.0	−20.0 to 0.5	125.8–127.9	−20.0
ros-sim, worst case, run 1	125.0–127.6	−20.0 to −5.8	123.7–125.1	−20.0
ros-sim, worst case, run 2	124.6–132.2	−20.0 to −5.9	126.6–128.8	−20.0

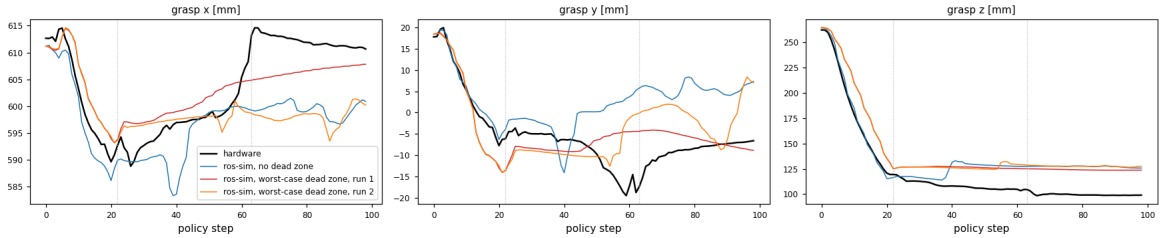


Figure B.2: Grasp position of the return-dead-zone student S_2 in the robot base frame on hardware (black) against ros-sim without the dead zone (blue) and with the worst-case dead zone (red and orange, two runs), aligned on the policy step. Dotted lines mark the first finger contact on hardware (step 22) and the slip (step 63).

After the slip step, the simulated hand is held at 123.7–128.8 mm while its vertical error stays at the −20 mm clip, so the controller pushes down at its limit and the hand does not

descend. On hardware the hand descends to 98.5–119.5 mm over the same steps. In the contact window, joints j4, j5, and j6 of the worst-case dead-zone runs differ from hardware by 0.007–0.042 rad on average, against 0.125–0.207 rad without the dead zone, while j7 differs by 0.066–0.072 against 0.097 rad.

Interpretation. The actions agree until contact, so the student receives equivalent observations in free space. From contact on, the two systems diverge because the simulated nut and the physical nut resist the hand differently. This is a contact gap that the arm controller does not address.

Limitations. One hardware run and three ros-sim runs, all within the first 99 steps of a single cycle.

B.5 Static Dead-Zone and Friction Probe

Purpose and method. The probe measures, at a fixed pose, how large a task-space wrench the physical arm holds without moving, in the units of the training dead zone. The characterization was designed in two parts, of which only part A was built. The probe streams the policy goal along one base axis at a time, out to +15 mm, back, rest, out to −15 mm, back, and rest, at 6 mm/s. Rotations use ±8° at 2°/s. The analysis reads the 1 kHz controller state and evaluates the spring wrench $-K_p(\boldsymbol{x}_{\text{measured}} - \boldsymbol{x}_{\text{target}})$ per axis:

- *rest band*: where the arm settles at the start pose, approached once from each side. Half the difference of the two spring wrenches is the band, half their sum a bias, and the band divided by the stiffness is the stall distance.
- *breakaway*: the spring wrench when the arm first follows the goal in a leg that starts from rest.
- *creep*: the median spring wrench while the arm follows, reported after subtracting the expected damping share.

The rest band is the quantity a stalled return meets. The bias contains payload and gravity model errors. In the two sessions at pose o0, the hand load was set through libfranka on a zero-mass end-effector profile. The session at the return pose did not set it again.

Known-answer test in ros-sim. With the worst-case dead zone, the probe recovers the thresholds as breakaway, 5.18–5.41 N and 1.08–1.22 N m over three runs at two speeds. Without the dead zone, the breakaway at the same pose is 0.05–0.15 N and at most 0.03 N m. The rest band, which only the later version of the probe records, reads 2.82–3.08 N (5.0–5.5 mm) and 1.17–1.30 N m (2.4–2.7°) with the worst-case dead zone. Without the dead zone, at the return pose, it reads 0.28–1.18 N and 0.46–0.64 N m.

Results on hardware. Of six hardware sessions, three ended before the first probe leg with an error. The other three completed. Table B.5 lists their rest bands next to the ros-sim known-answer runs.

At both poses the physical arm holds 0.8–2.1 N and 0.34–1.14 N m, which is 1.5–3.7 mm and 0.7–2.3°. With friction compensation, the band at o0 shrinks on x from 1.72 to 0.92 N and on r_z from 0.77 to 0.52 N m, and changes by at most 0.13 on the other axes. The

Table B.5: Rest band of the static probe per base axis, in N or N m with the corresponding stall distance in mm or degrees in parentheses. Pose o0 places the grasp point at $(0.30, 0.20, 0.30)$ m. The return pose is probed once with the arm in the reset posture and once in the posture in which a return had stalled. Friction compensation, where activated, is 0.3 N m per joint.

System	Pose	Condition	x	y	z	r_x	r_y	r_z
hardware	o0	no friction compensation	1.72 (3.0)	1.25 (2.2)	1.94 (3.4)	0.80 (1.6)	0.82 (1.7)	0.77 (1.6)
hardware	o0	friction compensation activated	0.92 (1.6)	1.12 (2.0)	1.84 (3.3)	0.86 (1.8)	0.71 (1.4)	0.52 (1.1)
hardware	return, reset posture	friction compensation activated	0.90 (1.6)	0.82 (1.5)	1.61 (2.8)	1.14 (2.3)	0.43 (0.9)	0.37 (0.8)
hardware	return, drifted posture	friction compensation activated	1.10 (1.9)	0.97 (1.7)	2.11 (3.7)	1.01 (2.1)	0.47 (1.0)	0.34 (0.7)
ros-sim	o0	worst-case dead zone	2.88 (5.1)	2.82 (5.0)	3.08 (5.5)	1.30 (2.7)	1.17 (2.4)	1.19 (2.4)
ros-sim	return, reset posture	no dead zone	1.18 (2.1)	0.28 (0.5)	0.46 (0.8)	0.46 (0.9)	0.55 (1.1)	0.64 (1.3)

drifted posture raises the translational band by 0.15–0.50 N over the reset posture. Every hardware run has a positive z bias at rest, +1.27 to +2.07 N. Without friction compensation, the creep wrench minus the damping share is 1.71–2.23 N and 0.43–0.60 N m, larger than the breakaway of the same session, 0.86–1.29 N and 0.45–0.79 N m.

Interpretation. At these two poses, the rest band of the physical arm is smaller than the band that the worst-case dead zone produces in the same probe, 2.8–3.1 N and 1.2–1.3 N m. That the arm needs more spring wrench to keep moving than to start moving indicates a speed-dependent drag rather than stick-slip, and the breakaway values on hardware are therefore not a reliable friction estimate. Section B.6 measures this drag directly. The positive z bias is consistent with a payload model error but is not resolved by this test.

Limitations. Two poses, one probe speed, and one session per condition. The payload fit that the same sessions record is ill-conditioned (condition number about 290) and is not used here.

B.6 Lost Wrench against Speed

Purpose and method. A dead zone and joint friction differ most in how the wrench they remove depends on speed. Once the arm moves in ros-sim, the dead zone removes the full threshold d_i at any speed, whereas friction can grow with speed. The probe of Section B.5 was therefore repeated at four goal speeds per axis, 2, 6, 20, and 50 mm/s and 1, 2, 6, and 15°/s, at pose o0 without setting the load. Every leg lasts at least 1.5 s, so the faster levels use longer strokes, up to ±75 mm and ±22.5°. For every moving sample of every leg, the analysis computes the lost wrench along the leg direction, the part of the spring wrench that does not go into the controller's damping,

$$\ell = -K_p\, e - K_d\, v, \tag{B.3}$$

with e the tracking error and v the speed along the leg, both signed in the leg direction. The arm's inertia is ignored. Samples of both directions are folded together, binned by speed, and the bin medians, weighted with the square root of the bin count, are fitted with

$$\ell(v) = F_c \tanh\!\left(\frac{v}{v_0}\right) + b\, v. \tag{B.4}$$

F_c is a Coulomb step smoothed over the speed v_0, and b is a viscous coefficient. The training dead zone corresponds to $F_c = d_i$, $v_0 \to 0$, and $b = 0$. A straight-line fit through the leg ends

is not used, because it assumes that the arm reaches the goal speed within a leg, which the physical arm does not.

Three sessions completed: ros-sim without the dead zone, and hardware without and with friction compensation. The hardware session with friction compensation ran about half an hour after the one without, on a warmer arm. Two further ros-sim sessions ended early. The sweep was not run in ros-sim with a dead zone. For that case, the known-answer test of Section B.5 stands in: there the worst-case dead zone reads back as a breakaway of 5.18–5.41 N and 1.08–1.22 N m.

Results. Table B.6 lists the fitted terms, and Figure B.3 shows the binned losses with the fits for the x and r_x axes. Without friction compensation, the physical arm loses 0.13 N on x at 0.15 mm/s, 1.65 N at 0.76 mm/s, 2.82 N at 1.3 mm/s, 3.63 N at 4.9 mm/s, 5.06 N at 17 mm/s, and 6.97 N at 44 mm/s, with 6.32 N on y at 44 mm/s. ros-sim loses 1.59 N and 1.76 N on x and y at 51 mm/s. With friction compensation, the physical arm loses 4.93 N on x at 43 mm/s and 3.92 N on y at 45 mm/s.

Table B.6: Fit of (B.4) to the lost wrench per axis at pose o0. F_c in N or N m, v_0 in mm/s or deg/s, b in N s/m or N m s/rad. The z axis is left out: on hardware its fitted v_0 sits at the lower bound of the search grid, and its slowest bin already reads 2.8 N, a static offset rather than friction (Section B.5). The last row gives the training dead zone for comparison.

Condition	Term	x	y	r_x	r_y	r_z
hardware, no friction compensation	F_c	2.76	2.43	1.31	1.63	0.81
	v_0	0.96	1.17	0.60	0.89	0.73
	b	97.5	92.3	7.8	1.0	1.8
hardware, friction compensation activated	F_c	1.43	1.44	0.98	1.08	0.40
	v_0	0.96	1.34	0.73	0.96	0.68
	b	84.7	54.3	5.0	1.0	1.3
ros-sim, no dead zone	F_c	1.57	1.47	0.71	0.63	0.35
	v_0	1.43	1.53	0.68	0.68	0.89
	b	0.6	5.8	0.6	0.7	0.3
training, worst-case dead zone	F_c, v_0, b	5, 0, 0	5, 0, 0	1, 0, 0	1, 0, 0	1, 0, 0

The raw signals of a single leg (no friction compensation, x, 2 mm/s) show what the fit summarizes. On the outward leg the lag behind the goal grows from 0.2 mm to 8.3 mm over 6.6 s and is still 7.9 mm at the end of the 7.5 s leg. Over the second half the arm moves at 1.46 mm/s against the goal's 2.00 mm/s, so it never reaches the goal speed. When the goal returns to the start pose and stops, the arm keeps approaching it for about 1.2 s, from 2.6 mm to 2.1 mm short, and then rests.

The joint friction fit of Section B.10 does not account for the viscous part. Its per-joint viscous coefficients, though poorly identified, predict the task-space loss through the power balance $F_{\text{pred}} = \sum_j b_j \dot{q}_j^2 / v$, evaluated on the logged joint velocities of the 20 and 50 mm/s legs. With the coefficients fitted on the day of the sweep, it predicts 0.09–0.13 N at 16–18 mm/s and 0.28–0.35 N at 43–45 mm/s, and with the largest coefficient of each joint over all seven days at most 0.82 N. The fitted viscous term $b\,v$ of Table B.6 is 2.4–4.1 N at the same speeds.

Interpretation. The physical arm does not lose a constant wrench once it moves. Its loss rises over the first millimeter per second to 2.4–2.8 N and 1.3–1.6 N m about r_x and r_y, and

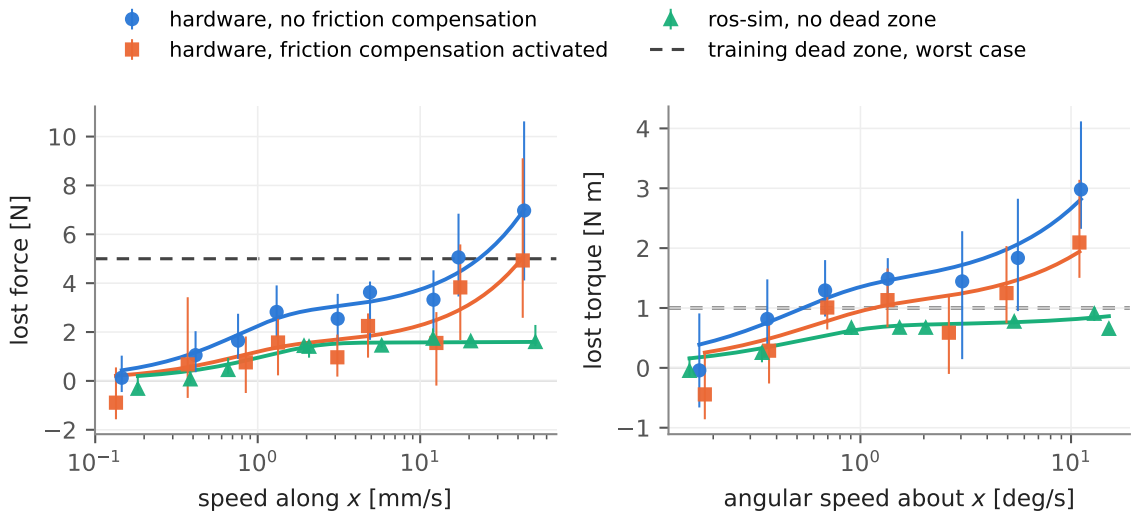


Figure B.3: Lost wrench (B.3) against speed at pose o0, translation along x (left) and rotation about x (right), on hardware without and with friction compensation and in ros-sim without the dead zone. Markers are bin medians, bars the 25th to 75th percentile, lines the fits of (B.4). Dashed: the worst-case training dead zone, which removes its full threshold at any speed.

keeps rising with speed at 92–98 N s/m, about twice the controller's own translational damping of 47.5 N s/m. Because the step is spread over $v_0 \approx 1$ mm/s rather than being sharp, a small error is corrected slowly rather than held. The arm creeps, as the single leg shows, instead of sticking and slipping. ros-sim reproduces a smaller step, 1.5 N, from the 0.2 N m joint friction of its MuJoCo model, and no viscous term. The training dead zone is a sharp step without a viscous term. In translation, its worst case of 5 N is larger than the hardware step of 1.4–2.8 N, and in rotation its 1 N m lies within the hardware range of 0.4–1.6 N m. At policy speeds the viscous term dominates: at 44 mm/s the physical arm loses about 7 N on x without friction compensation, against 1.6 N in ros-sim. The joint viscous coefficients of the same day account for at most a seventh of the fitted viscous term, and the largest coefficients over all days for at most 0.27 of it, so its source remains open. With friction compensation, F_c halves on x and y and b drops from 92–98 to 54–85 N s/m. Because that session ran on a warmer arm, this is an upper bound on the effect of friction compensation.

Limitations. One pose, one session per condition, and the warm-up confound between the two hardware sessions. Inertia is neglected in (B.3), which affects the bins near the turnarounds. The z axis is distorted by a static offset and is not evaluated.

B.7 Why the Return Stalls

Purpose and method. On hardware, the Cartesian return to the reset pose stops short of it, whereas the joint-space re-home lands within a few millimeters. The torques at such a stall were decomposed for two hardware runs. In the second, the return gate of 3 mm and 2° never passed, so the return ran for its full 16.9 s. The logged state is averaged from 4 s into the return onward, and the position spring and the rotation spring are mapped separately to joint torques through J^{T}.

Results. In the second run, the arm settled 3.7 s into the return at 16–17 mm and 7° with all joints below 0.006 rad/s, and then crept to 14.5 mm and 5.9°. Averaged over the rest, the error is 15.8 mm and 6.4°, the spring force $(8.33, -3.18, 0.82)$ N, and the spring torque $(-0.60, 1.86, 2.46)$ N m. Table B.7 gives the joint torques.

Table B.7: Joint torques at the return stall of the second run, from the position spring and the rotation spring separately, their sum, and the commanded torque recorded at rest, in N m.

	j1	j2	j3	j4	j5	j6	j7
position spring	−2.10	−0.93	−1.72	3.64	0.93	2.22	−0.13
rotation spring	2.46	0.99	1.63	−2.02	−2.11	−2.11	0.18
sum (task torque)	0.36	0.06	−0.10	1.62	−1.18	0.10	0.05
recorded command	0.85	0.00	0.16	1.45	−1.18	−0.08	0.15

The two springs oppose each other on j1–j6. What remains is 1.6 N m on j4 and −1.2 N m on j5, and neither joint moves. The first run shows the same pattern: a 13.7 mm position error with 7.7 N and a rotation spring of about 3.5 N m reach 3.8 N m at single joints, but the net torque is at most 1.5 N m. The exact nullspace projector contributes at most 0.04 N m there, so the posture term does not move the arm back either.

Interpretation. The grasp point lies 185 mm from the flange, close to $\sqrt{28/565} = 0.22$ m. For a rotation of the hand about a point near the wrist, the position spring and the rotation spring therefore nearly cancel, and the effective stiffness about the flange is $28 - 565 \cdot 0.185^2 \approx 8.7$ N m/rad, a third of the rotational gain. A small friction torque on j4 and j5 holds a large combined error in this direction. Because j4 and j5 do not move under the residual torque, their static friction at this posture is at least 1.5 and 1.2 N m. The probe of Section B.5 excites one task axis at a time and never this coupled direction, which explains why it reads a small band while the return stalls at about 9 N. The law has no integral term, in training as on hardware, so any error held by friction persists.

Limitations. Two stalls at one return pose. The static friction bounds are lower bounds inferred from the absence of motion.

B.8 Transition Clip

Purpose and implementation. The policy clip of 0.02 m and 0.097 rad bounds the target step against the measured pose in every 1 kHz cycle, and so it also bounds the spring wrench to $565 \cdot 0.02 = 11.3$ N and $28 \cdot 0.097 = 2.7$ N m per axis. During the scripted transitions of the cycle, waypoint following and the return to reset, a goal may now widen the clip and the fault limits up to the ceilings of Table B.1. The hardware runs below use a transition clip of 0.05 m and 0.25 rad, the value of the training configuration.

Clip speed limit. With critical damping, a target clipped to δ ahead of the arm drives it at a steady speed of at most $K_p\delta/K_d = \delta\sqrt{K_p}/2$. This gives 0.24 m/s and 0.26 rad/s for the policy clip and 0.59 m/s and 0.66 rad/s for the transition clip. These are derived values for a pure translation or rotation of a lossless arm. Section B.9 measures the speed while the policy clip is active: 178 mm/s in ros-sim and 95–128 mm/s on the physical arm, whose loss grows with speed (Section B.6).

Method and data. Six hardware runs of the proprio-friction student S_1 used the transition clip. Runs 1–3 recorded the 1 kHz controller state, which is aligned with the rollout log to evaluate each transition phase. Runs 4–6 have the 15 Hz rollout log only. Runs 1 and 2 did not re-home between cycles, and runs 3–6 did. The return gate was 15 mm and 10° in run 1 and 20 mm and 10° afterward.

Results. Table B.8 gives the transition phases of runs 2 and 3, and Figure B.4 shows run 2. In earlier runs with the policy clip in the transitions, orientation had stopped at 16–20° during the return. With the transition clip, every return of runs 1–5 that passed the gate did so within 2.0–2.7 s, and no run hit a tracking fault. With re-homing, 12 of 12 cycles completed in runs 3–5. Without re-homing, the position error at the end of the return grew from cycle to cycle, 13.0, 15.6, and 17.3 mm in run 2, while the arm posture drifted within every cycle, about −0.2 rad on j1 and +0.2 rad on j3, and the drift carried into the next cycle. With re-homing it stayed at 12.5–13.5 mm. In run 1, cycle 2 did not pass the 15 mm gate and stopped at 15.2 mm and 6.7° after 16.9 s. Run 6 ended in cycle 1 when the grip collapsed during the turn.

Table B.8: Transition phases with the transition clip (0.05 m, 0.25 rad, fault limits 0.1 m and 0.5 rad) on hardware, runs 2 and 3, three cycles each. Ranges over cycles. Errors are against the clipped target.

Quantity	Waypoint following	Return to reset
duration	2.87 s	2.27–2.67 s
fraction of the phase with orientation clipped	0.26–0.30	0.53–0.59
fraction of the phase with position clipped	0	0
peak position error	0.041–0.057 m	0.056–0.067 m
peak orientation error	0.406–0.441 rad	0.358–0.364 rad

Interpretation. The wider clip raises the spring wrench that a transition can apply, and this fixed the orientation stall of the return. It does not fix the position residual, which is the coupled stall of Section B.7. The gate fired while the arm was still closing at 3–5 mm/s and 10–16°/s, so the residuals at the gate are not resting points. The growth without re-homing follows the accumulated drift of j1 and j3, which the nullspace spring of stiffness 10 does not undo against friction.

Limitations. The peak orientation error of 0.441 rad leaves a margin of 0.06 rad to the 0.5 rad fault limit. Only runs 1–3 have 1 kHz data, and no per-phase summary is available for run 1. The ros-sim recordings of the same comparison are not evaluated here.

B.9 Target Accuracy and Repeatability

Purpose and method. A repeatability test streams the policy goal between two grasp positions in the base frame, A at $(0.30, 0.20, 0.30)$ m and B at $(0.30, 0.40, 0.30)$ m, at several orientations, and approaches every target from both sides. The goal moves at up to 0.5 m/s, faster than the policy clip lets the arm follow, so the arm runs into the clip on the way. At each arrival, the goal stops on the target and the rest error is the mean over the last 0.5 s of the hold. From the arrivals of one target, the test computes the *accuracy* (distance of the mean rest position from the target, ISO 9283 style), the *repeatability* per approach side, the *hysteresis* (distance between the mean rest positions of the two sides, from friction or a dead

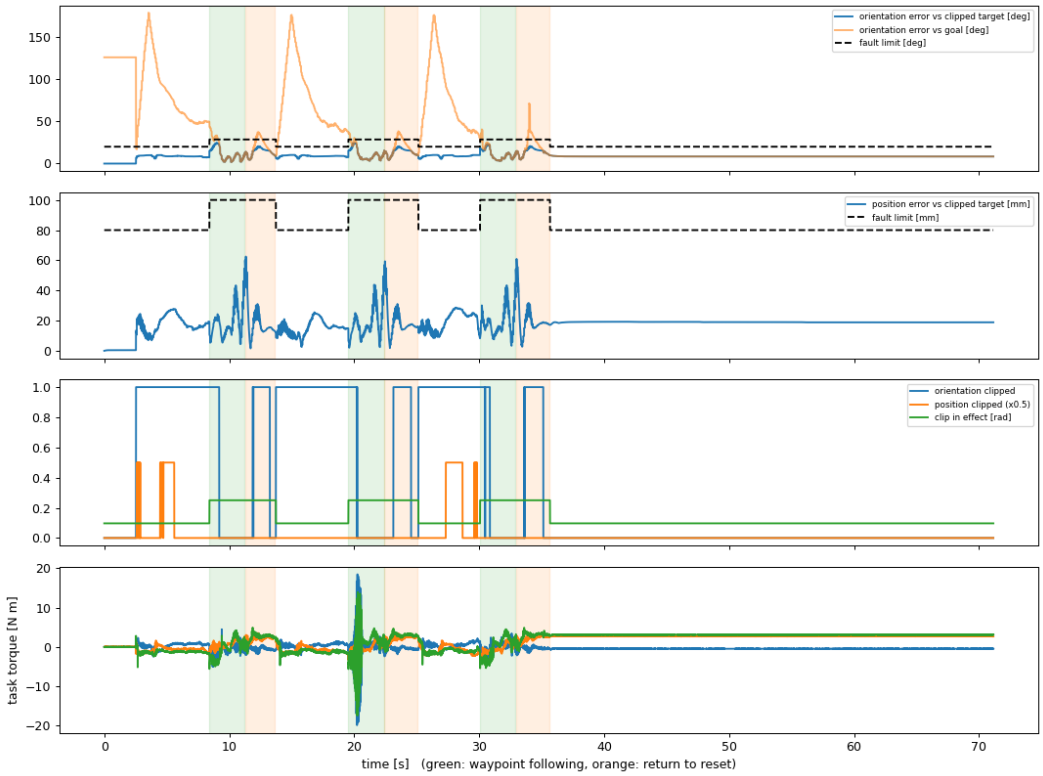


Figure B.4: Transition clip on hardware, run 2 (three cycles without re-homing), from the 1 kHz controller state. From top to bottom: orientation error against the clipped target and against the goal with the fault limit, position error with the fault limit, clip flags and the orientation clip in effect, and the task torque. Green bands mark waypoint following, orange bands the return to reset.

zone), and the *bias* (their mean, from a load or model error). It also counts the arrivals inside 5 mm and 5°. In the earlier runs every hold lasted a fixed 2 s. In the later runs, a hold lasts until the arm has moved less than 0.2 mm and 0.1° over 0.5 s, at least 2 s and at most 10 s, and every arrival records whether it settled.

ros-sim. Five ros-sim runs were made, all without the dead zone. Three completed, one stopped near a singularity, and one was rejected because a joint ended 0.22 rad from the start posture. Table B.9 lists the completed runs.

Table B.9: Accuracy and repeatability of the controller in ros-sim without the dead zone, two positions × four orientations, each approached from both sides. All arrivals ended inside 5 mm and 5°.

Run	Arrivals	Accuracy (mean)	Worst repeatability	Hysteresis (mean)	Peak speed
1	32	1.07 mm	2.22 mm / 2.60°	2.15 mm	0.257 m/s
2	16	1.05 mm	2.16 mm / 2.53°	2.19 mm	0.332 m/s
3	16	1.27 mm	1.32 mm / 0.35°	1.31 mm	0.228 m/s

Hardware runs. Three early hardware attempts stopped before their first arrival. In one, the controller rejected a goto step of 3.03 rad, one was interrupted, and one was rejected because the start posture was 0.71 rad from the checked one. Of five later runs, four completed and one was rejected at the start, because the forward kinematics of the measured joints disagreed with the robot's end-effector pose. The two runs used for accuracy were made about

ten minutes apart, one without and one with friction compensation. Each has 40 arrivals, two positions × orientations o0 (flange down) and o1 (30° yaw) × two sides × five repeats, and every arrival settled. Two earlier runs, again one without and one with friction compensation, have 80 arrivals at four orientations with a fixed 2 s hold. They are used below only for arrival speed, not for accuracy. Table B.10 gives the results of the two settled runs. The error 1 s after the goal stopped and the median speed of the grasp point while the position clip was active were computed from the 50 Hz pose samples.

Table B.10: Accuracy and repeatability of the physical arm, 40 settled arrivals per run at A and B, orientations o0 and o1, both sides, five repeats. Per-target values are listed as A/o0, A/o1, B/o0, B/o1. Friction compensation, where activated, is 0.3 N m per joint.

Quantity	No friction compensation	Friction compensation activated
hold after the goal stopped	2.00–2.49 s	2.00–2.02 s
accuracy (mean over targets)	8.54 mm	6.42 mm
arrivals inside 5 mm and 5°	0 of 40	5 of 40, all at A/o0
hysteresis (mean)	6.07 mm	3.78 mm
hysteresis per target	10.4, 1.5, 0.3, 12.1 mm	5.1, 2.0, 0.2, 7.8 mm
repeatability per approach side	2.2–3.3 mm	1.7–2.8 mm
z bias per target	−3.5, −2.7, −4.2, −5.6 mm	−2.8, −1.2, −5.6, −5.4 mm
load implied by the z bias (mean, range)	0.23 kg (0.16–0.32)	0.22 kg (0.07–0.32)
error 1 s after the goal stopped (median)	18.2 mm	6.9 mm
speed while the position clip is active (median)	95 mm/s	128 mm/s

Within a target and side, the rest error changes from the first to the fifth repeat by −0.5 to +2.6 mm without friction compensation and by −0.6 to +2.1 mm with it, and grows in six of eight cases in both runs. Over the same five repeats, the rest posture drifts by −0.24 to −0.42 rad on j1 and +0.20 to +0.37 rad on j3. The earlier runs give the same order between the two settings. The median error 1 s after the goal stopped is 19.9 mm without and 7.6 mm with friction compensation, and the median speed with the position clip active is 86 and 126 mm/s. In ros-sim (run 1 of Table B.9) it is 178 mm/s.

Interpretation. The physical arm is precise but not accurate: per approach side, it returns within 1.7–3.3 mm, but it rests 6–9 mm from the target on average. Part of the repeatability spread is the drift of the redundant joints j1 and j3 over the repeats, the same drift as in Section B.8. The hysteresis averages 4–6 mm but ranges from 0.2 to 12 mm between targets, large at A/o0 and B/o1 and small at B/o0 and A/o1, in both runs. Joint friction maps to task space through the posture-dependent Jacobian, so its band depends on the target. A per-axis task-space band, such as the dead zone, cannot represent this. Friction compensation reduces the mean hysteresis by 38 % and the mean accuracy error by 25 %, and it makes the arm arrive faster (6.9 instead of 18.2 mm after 1 s). It does not change the downward z bias of 1–6 mm, which corresponds to about 0.2 kg of unmodeled load. That bias is therefore a payload or gravity-model error, not friction.

The clip-limited speed follows from the loss curve of Section B.6. With the clip at $\delta = 0.02$ m, the spring wrench is at most $K_p\delta = 11.3$ N, and in steady motion it balances $K_d v + F_c + b\,v$. With the y fits of Table B.6, this gives 63 mm/s without friction compensation, 97 mm/s with it, and 184 mm/s in ros-sim. ros-sim moves at that speed (178 mm/s), and the physical arm moves about 30 mm/s faster than predicted but in the same order. The loss curve was measured at a single pose and on single axes, whereas the transits combine both. Between runs with the same setting, made 40 to 60 minutes apart, the clip-limited speed changes by 10 % (86 and 95 mm/s) and 2 % (126 and 128 mm/s), and the error after 1 s by 1.7 and 0.7 mm. Switching friction compensation on changes the same quantities by

a factor of 1.3 to 2.6. Any effect of warm-up is therefore smaller than the effect of friction compensation. It was not measured separately.

Limitations. Two positions, two orientations, and one settled run per setting. The two earlier runs have unsettled holds and are not used for accuracy. Arrivals from the same side follow the same transit, so the repeatability per side does not include the spread of different approach paths.

B.10 Joint Friction Fitted from Rollouts

Purpose and method. Ordinary policy rollouts already contain a friction signal. The joint torque sensor of the FR3 sits between gearbox and link, so the difference between the desired torque $\tau_{J,d}$ and the measured link-side torque minus gravity is torque lost in the gearbox. This holds also in contact, because external forces act on the link side. The lost torque is computed from the logged desired and measured joint torques of the hardware rollouts (low-pass filtered with 0.02 s) and fitted per joint and day, on samples with $|\dot{q}_j| > 0.01$ rad/s, with

$$\tau_{J,d,j} - \left(\tau_{J,j} - g_j\right) = \tau_{c,j}\,\mathrm{sign}(\dot{q}_j) + b_j\dot{q}_j + o_j, \tag{B.5}$$

with Coulomb friction $\tau_{c,j}$, viscous coefficient b_j, and offset o_j. Friction compensation, when activated, is part of $\tau_{J,d}$, so the fit measures the total friction either way.

Results. Table B.11 lists the Coulomb terms of 56 runs on seven days over two weeks.

Table B.11: Coulomb friction τ_c per joint of the physical FR3, in N m, fitted from hardware rollouts per day with (B.5). Days are numbered in chronological order over two weeks. The last row gives the correlation between the lost torque and $\mathrm{sign}(\dot{q})$ over the days. For j1 it is undefined on two days.

Day	Runs	j1	j2	j3	j4	j5	j6	j7
1	6	0.32	0.86	0.40	0.50	0.58	0.30	0.26
2	6	0.31	0.88	0.29	0.56	0.59	0.33	0.26
3	11	0.35	0.87	0.42	0.55	0.58	0.29	0.24
4	5	0.54	0.93	0.48	0.64	0.43	0.37	0.24
5	17	0.58	0.97	0.45	0.60	0.54	0.23	0.24
6	8	0.60	0.82	0.37	0.64	0.60	0.32	0.26
7	3	0.61	0.89	0.33	0.65	0.60	0.32	0.21
correlation		0.70–0.87	0.93–0.97	0.58–0.88	0.54–0.96	0.90–0.98	0.55–0.93	0.79–0.95

The Coulomb friction is 0.21–0.97 N m depending on the joint. It is largest on j2 (0.82–0.97 N m), and on j1 it roughly doubled from 0.31–0.35 N m on days 1–3 to 0.54–0.61 N m from day 4 on. The other joints vary by at most 0.19 N m over the seven days. The viscous coefficients scatter between −0.01 and 4.86 N m s/rad. They are not used as a friction model, only for the rough comparison in Section B.6.

Interpretation. The physical joints have 0.2–1 N m of Coulomb friction. The training recipes with joint friction draw 0.16–0.4 N m per joint (0.2 N m scaled by $U(0.8, 2.0)$). Joints j2, j4, and j5 exceed that range on every day, and j6 and j7 lie inside it. The uniform friction compensation of 0.3 N m lies below the fitted friction on j1–j5 but above it on j7 (0.21–0.26 N m) and on j6 on two days (0.23 and 0.29 N m). This disagrees with the rationale

that 0.3 N m stays below every measured moving loss, which rested on two earlier runs (Section B.2). A per-joint compensation from Table B.11 would remove this inconsistency, but it has not been tried.

Limitations. The rollouts are logged at 15 Hz, the rotor inertia is ignored during accelerations, and the joints stay below about 1 rad/s, so the viscous term is not identifiable. The two hardware runs of Section B.3, the second stall run of Section B.7, and the transition-clip runs of Section B.8 are not part of the fit, although they contain the desired torque.

B.11 Summary

Training models the imperfection of the arm as a sharp, memoryless, speed-independent step in task space, the dead zone of up to 5 N and 1 N m. The physical FR3 under the same impedance law behaves differently in four measured respects. First, it loses a Coulomb step smoothed over about 1 mm/s, 2.4–2.8 N and 0.8–1.6 N m without friction compensation, plus a viscous term of 92–98 N s/m, about twice the controller's damping (Table B.6). Second, the low-speed part is smaller than the worst-case dead zone in translation and similar in rotation: the static probe reads a rest band of 0.8–2.1 N and 0.3–1.1 N m (Table B.5), and the joint Coulomb friction fitted from 56 rollouts over seven days is 0.2–1 N m (Table B.11). Third, at policy speeds the viscous part dominates: at 44 mm/s the arm loses about 7 N against 1.6 N in ros-sim, and with the position clip active it moves at 95–128 mm/s against 178 mm/s in ros-sim. Fourth, the loss depends on the posture. The hysteresis between approach sides ranges from 0.2 to 12 mm between targets (Table B.10), and in the coupled direction of a rotation about the wrist the same friction holds a return about 15 mm and 6° from its target (Section B.7). Because the arm creeps rather than sticks, the error it ends with depends on how long it is given.

A task-space dead zone gets the order of magnitude of the rest band right, which is why the worst-case dead zone in ros-sim reproduces the free-space motion and the return of the physical arm (Section B.3). It misses the speed dependence, the posture dependence, and the creep. In contact it leaves the hand free to move inside the band, where the physical arm resists (Sections B.1 and B.4). The deployment controller therefore adds three measures: joint friction compensation on hardware, which reduces the hysteresis by 38 % and the accuracy error by 25 % (Section B.9); the training dead zone as an option in ros-sim; and the wider transition clip, which fixed the orientation stall of the return (Section B.8). The physical arm remains offset 6–9 mm from its targets, including a downward z bias of 1–6 mm that friction compensation does not change and that points to a payload or gravity-model error. The remaining gap between ros-sim and hardware lies mostly in contact and grasping, which the arm controller does not address.

Bibliography

[1] R. G. Budynas and J. K. Nisbett, *Shigley's Mechanical Engineering Design*, 11th ed. New York: McGraw-Hill Education, 2020, ISBN: 978-0-07-339821-1.

[2] G. Niemann, H. Winter, B.-R. Höhn, and K. Stahl, *Maschinenelemente 1: Konstruktion und Berechnung von Verbindungen, Lagern, Wellen*, German, 5th ed. Berlin, Heidelberg: Springer Vieweg, 2019, ISBN: 978-3-662-55481-4. DOI: 10.1007/978-3-662-55482-1

[3] A. Bicchi and V. Kumar, "Robotic grasping and contact: A review," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, IEEE, 2000, pp. 348–353, ISBN: 0-7803-5886-4. DOI: 10.1109/ROBOT.2000.844081

[4] A. I. Weinberg, A. Shirizly, O. Azulay, and A. Sintov, "Survey of learning-based approaches for robotic in-hand manipulation.," *Frontiers in robotics and AI*, vol. 11, p. 1 455 431, 2024, ISSN: 2296-9144. DOI: 10.3389/frobt.2024.1455431

[5] T. Chen, J. Xu, and P. Agrawal, "A system for general in-hand object re-orientation," Nov. 2021.

[6] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. V. Wyk, A. Zhurkevich, B. Sundaralingam, Y. Narang, J.-F. Lafleche, D. Fox, and G. State, "Dextreme: Transfer of agile in-hand manipulation from simulation to reality," Jan. 2024. [Online]. Available: https://arxiv.org/pdf/2210.13702

[7] L. Tang and Y.-B. Jia, "Robotic fastening with a manual screwdriver," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2023, pp. 5269–5275, ISBN: 979-8-3503-2365-8. DOI: 10.1109/ICRA48891.2023.10161139

[8] L. Tang, Y.-B. Jia, and Y. Xue, "Robotic manipulation of hand tools: The case of screwdriving," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2024, pp. 13 883–13 890, ISBN: 979-8-3503-8457-4. DOI: 10.1109/ICRA57147.2024.10610831

[9] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, "Simtoolreal: An object-centric policy for zero-shot dexterous tool manipulation," Feb. 2026.

[10] H. Moravec, *Mind Children: The Future of Robot and Human Intelligence*. Cambridge, MA: Harvard University Press, 1988, ISBN: 978-0-674-57616-2.

[11] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, pp. 1238–1274, 11 Sep. 2013, ISSN: 0278-3649. DOI: 10.1177/0278364913495721

[12] A. Petrenko, A. Allshire, G. State, A. Handa, and V. Makoviychuk, "Dexpbt: Scaling up dexterous manipulation for hand-arm systems with population based training," May 2023.

[13] Y. Narang, K. Storey, I. Akinola, M. Macklin, P. Reist, L. Wawrzyniak, Y. Guo, A. Moravanszky, G. State, M. Lu, A. Handa, and D. Fox, "Factory: Fast contact for robotic assembly," May 2022.

[14] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu, "Sequential dexterity: Chaining dexterous policies for long-horizon manipulation," Oct. 2023.

[15] M. Noseworthy, B. Tang, B. Wen, A. Handa, C. Kessens, N. Roy, D. Fox, F. Ramos, Y. Narang, and I. Akinola, "Forge: Force-guided exploration for robust contact-rich manipulation under uncertainty," Jan. 2025.

[16] B. Tang, M. A. Lin, I. Akinola, A. Handa, G. S. Sukhatme, F. Ramos, D. Fox, and Y. Narang, "Industreal: Transferring contact-rich assembly tasks from simulation to reality," May 2023.

[17] E. Hsieh, W.-H. Hsieh, Y.-J. Wang, T. Lin, J. Malik, K. Sreenath, and H. Qi, "Learning dexterous manipulation skills from imperfect simulations," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026. arXiv: 2512.02011. [Online]. Available: https://arxiv.org/abs/2512.02011

[18] T. G. W. Lum, K. Kedia, C. K. Liu, and J. Bohg, *Play2Perfect: What matters in dexterous play pretraining for precise assembly?* 2026. arXiv: 2606.26428. [Online]. Available: https://arxiv.org/abs/2606.26428

[19] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," Mar. 2011.

[20] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," Mar. 2024.

[21] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," Jan. 2026.

[22] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," Feb. 2023.

[23] R. Singh, A. Allshire, A. Handa, N. Ratliff, and K. V. Wyk, "Dextrah-rgb: Visuomotor policies to grasp anything with dexterous hands," Feb. 2025.

[24] T. G. W. Lum, M. Matak, V. Makoviychuk, A. Handa, A. Allshire, T. Hermans, N. D. Ratliff, and K. V. Wyk, "Dextrah-g: Pixels-to-action dexterous arm-hand grasping with geometric fabrics," Oct. 2024.

[25] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations," Sep. 2024.

[26] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018, ISBN: 9780262039246.

[27] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3-4, pp. 229–256, 1992. DOI: 10.1007/BF00992696

[28] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations (ICLR)*, 2016. arXiv: 1506.02438. [Online]. Available: https://arxiv.org/abs/1506.02438

[29] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, *Asymmetric actor critic for image-based robot learning*, 2017. arXiv: 1710.06542. [Online]. Available: https://arxiv.org/abs/1710.06542

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, 2017. arXiv: 1707.06347. [Online]. Available: https://arxiv.org/abs/1707.06347

[31] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, Morgan Kaufmann, 1999, pp. 278–287.

[32] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. DOI: 10.15607/RSS.2023.XIX.016

[33] K. Black, M. Y. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," in *Advances in Neural Information Processing Systems*, 2025. arXiv: 2506.07339.

[34] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 6840–6851.

[35] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 684–10 695. DOI: 10.1109/CVPR52688.2022.01042

[36] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, "Flow matching guide and code," Dec. 2024.

[37] T. Lin, K. Sachdev, L. Fan, J. Malik, and Y. Zhu, "Sim-to-real reinforcement learning for vision-based dexterous manipulation on humanoids," Sep. 2025.

[38] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987. DOI: 10.1109/JRA.1987.1087068

[39] R. Martín-Martín, M. A. Lee, R. Gardner, S. Savarese, J. Bohg, and A. Garg, "Variable impedance control in end-effector space: An action space for reinforcement learning in contact-rich tasks," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 1010–1017.

[40] L. Villani and J. D. Schutter, "Force control," in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds., 2nd ed., Cham: Springer, 2016, ch. 9, pp. 195–220. DOI: 10.1007/978-3-319-32552-1_9

[41] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge: Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[42] W. K. Chung, L.-C. Fu, and T. Kröger, "Motion control," in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds., 2nd ed., Cham: Springer, 2016, ch. 8, pp. 163–194. DOI: 10.1007/978-3-319-32552-1_8

[43] A. Dietrich, C. Ott, and A. Albu-Schäffer, "An overview of null space projections for redundant, torque-controlled robots," *The International Journal of Robotics Research*, vol. 34, pp. 1385–1400, 11 Sep. 2015, ISSN: 0278-3649. DOI: 10.1177/0278364914566516

[44] E. Z. R. S. Lab, *Robot dynamics lecture notes*, 2019.

[45] O. M. Manyar, S. V. Narayan, R. Lengade, and S. K. Gupta, "Physics-informed learning to enable robotic screw-driving under hole pose uncertainties," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2023, pp. 2993–3000, ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342151

[46] T. Lin, Z.-H. Yin, H. Qi, P. Abbeel, and J. Malik, "Twisting lids off with two hands," Oct. 2024.

[47] P. Koczy, M. C. Welle, and D. Kragic, "Learning dexterous in-hand manipulation with multifingered hands via visuomotor diffusion," Mar. 2025.

[48] Y. Xue, L. Tang, and Y.-B. Jia, "Dynamic finger gaits via pivoting and adapting contact forces," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2023, pp. 8784–8791, ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342156

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90

[50] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133

[51] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660. DOI: 10.1109/CVPR.2017.16

[52] A. Wilcox, M. Ghanem, M. Moghani, P. Barroso, B. Joffe, and A. Garg, "Adapt3R: Adaptive 3d scene representation for domain transfer in imitation learning," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, PMLR, 2025, pp. 1486–1514. [Online]. Available: https://proceedings.mlr.press/v305/wilcox25a.html

[53] O. Zhang, I. Akinola, R. Scalise, M. G. Castro, P. Yin, T. Widmer, M. Macklin, C. Amevor, D. Turpin, D. V. Gelder, J. Carius, A. Handa, G. State, Y. Guo, and Y. Narang, "Alphanist: Emergent assembly skills from binary task rewards," Oct. 2026.

[54] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, pp. 3521–3526, 13 2017.

[55] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu, "Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164, PMLR, 2022, pp. 406–416. [Online]. Available: https://proceedings.mlr.press/v164/lee22a.html

[56] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100, PMLR, 2020, pp. 66–75. [Online]. Available: https://proceedings.mlr.press/v100/chen20a.html

[57] W. Wang, F. Wei, L. Zhou, X. Chen, L. Luo, X. Yi, Y. Zhang, Y. Liang, C. Xu, Y. Lu, J. Yang, and B. Guo, "UniGraspTransformer: Simplified policy distillation for scalable dexterous robotic grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 12 199–12 208.